AUBO 机器人 linux_C++_sdk 学习资料

（仅供内部学习）

（AUBOPE V4.3.5）

目录

一、 获取 linux C++ SDK 包 8
二、 编程环境 8
三、 打开 SDK 工程 9
四、 linux C++ SDK 文件构成 9
五、 运行 SDK 例子 10
六、 构建自己的开发程序（基于 Qt Creator） 11
1. 创建新项目 11
2. 导入 Aubo_linux_C++依赖库 12
3. 添加程序内容 13
4. 编译、运行 15
七、 linux_C++接口函数示例 17
机械臂登录与断开 17
1. 登录 robotServiceLogin 17
2. 退出登录 robotServiceLogout 18
3. 当前连接状态 robotServiceGetConnectStatus 18
4. 握手 robotServiceRobotHandShake 19
机械臂上电与断电 19
5. 机械臂上电 rootServiceRobotStartup 19
6. 机械臂断电 rootServiceRobotShutdown 21
信息推送 21
7. 使能实时关节状态推送 robotServiceSetRealTimeJointStatusPush 21
8. 实时关节信息推送 robotServiceRegisterRealTimeJointStatusCallback 21
9. 使能实时路点推送 robotServiceSetRealTimeRoadPointPush 23
10. 实时路点信息推送 robotServiceRegisterRealTimeRoadPointCallback 23
11. 使能实时末端速度推送 robotServiceSetRealTimeEndSpeedPush 25
12. 实时末端速度推送 robotServiceRegisterRealTimeRoadPointCallback 25
13. 机械臂事件实时推送 robotServiceRegisterRobotEventInfoCallback 25
运动模块 26
14. 初始化运动属性 robotServiceInitGlobalMoveProfile() 26
15. 设置机械臂关节型运动最大加速度 robotServiceSetGlobalMoveJointMaxAcc 27
16. 设置机械臂关节型运动最大速度 robotServiceSetGlobalMoveJointMaxVelc 27
17. 获取机械臂关节型运动最大加速度 robotServiceGetGlobalMoveJointMaxAcc 28
18. 获取机械臂关节型运动最大速度 robotServiceGetGlobalMoveJointMaxVelc 29

19. 设置末端型运动最大加速度 robotServiceSetGlobalMoveEndMaxLineAcc 29
20. 设置末端型运动最大速度 robotServiceSetGlobalMoveEndMaxLineVelc 30
21. 获取末端型运动最大加速度 robotServiceGetGlobalMoveEndMaxLineAcc 30
22. 获取末端型运动最大速度 robotServiceGetGlobalMoveEndMaxLineVelc 31
23. 设置末端型运动旋转最大角加速度 robotServiceSetGlobalMoveEndMaxAngleAcc 31
24. 设置末端型运动旋转最大角速度 robotServiceSetGlobalMoveEndMaxAngleVelc 31
25. 获取末端型运动旋转最大角加速度 robotServiceGetGlobalMoveEndMaxAngleAcc 31
26. 获取末端型运动旋转最大角速度 robotServiceGetGlobalMoveEndMaxAngleVelc 31
27. 清空路点容器 robotServiceClearGlobalWayPointVector 32
28. 添加全局路点 robotServiceAddGlobalWayPoint 32
29. 添加全局路点 robotServiceAddGlobalWayPoint 33
30. 获取全局路点容器 robotServiceGetGlobalWayPointVector 34
31. 设置轨迹运动中的交融半径 robotServiceSetGlobalBlendRadius 35
32. 获取交融半径值 robotServiceGetGlobalBlendRadius 36
33. robotServiceGetTrackPlaybackCycle 36
34. robotServiceSetTrackPlaybackCycle 36
35. 设置圆运动圈数 robotServiceSetGlobalCircularLoopTimes 36
36. 获取圆运动圈数 robotServiceGetGlobalCircularLoopTimes 37
37. 设置相对偏移属性 robotServiceSetMoveRelativeParam 37
38. 设置相对偏移属性 robotServiceSetMoveRelativeParam 39
39. 设置无提前到位 robotServiceSetNoArrivalAhead 45
40. 设置提前到位距离模式 robotServiceSetArrivalAheadDistanceMode 47
41. 设置提前到位时间模式 robotServiceSetArrivalAheadTimeMode 48
42. 设置提前到位交融半径模式 robotServiceSetArrivalAheadBlendDistanceMode 50
43. 设置示教坐标系 robotServiceSetTeachCoordinateSystem 52
44. 关节运动 robotServiceJointMove 53
45. 关节运动 robotServiceJointMove 55
46. 跟随模式轴动 robotServiceFollowModeJointMove 56
47. 直线运动 robotServiceLineMove 58
48. 直线运动 robotServiceLineMove 59
49. 旋转运动 robotServiceRotateMove 61
50. 轨迹运动 robotServiceTrackMove 66
51. 保持当前姿态通过相对位移直线运动至目标位置 robotMoveLineToTargetPositionByRelative 69
52. 保持当前姿态通过相对位移关节运动至目标位置 robotMoveJointToTargetPositionByRelative 76
53. 直线方式运动至给定位置 robotMoveLineToTargetPosition 82
54. 轴动方式运动至给定位置 robotMoveJointToTargetPosition 87
55. 示教运动开始 robotServiceTeachStart 90

56. 示教停止 robotServiceTeachStop 96
57. 运动的启动、停止、暂停和继续控制 rootServiceRobotMoveControl 96
58. 快速停止 robotMoveFastStop 97
离线轨迹运动 98
59. 添加离线轨迹路点 robotServiceOfflineTrackWaypointAppend 98
60. 清空离线轨迹路点 robotServiceOfflineTrackWaypointClear 98
61. 启动离线轨迹运行 robotServiceOfflineTrackMoveStartup 98
62. 停止离线轨迹运行 robotServiceOfflineTrackMoveStop 98
TCP 转 CAN 透传 98
63. 进入 tcp 转 can 透传模式 robotServiceEnterTcp2CanbusMode 98
64. 退出 tcp 转 can 透传模式 robotServiceLeaveTcp2CanbusMode 99
65. 发送坐标数据到关节 can 总线 robotServiceSetRobotPosData2Canbus 99
工具接口 100
66. 正解 robotServiceRobotFk 100
67. 逆解 robotServiceRobotIk 102
68. 工具标定 robotServiceToolCalibration 104
69. 检查用户坐标系是否合理 robotServiceCheckUserCoordinate 104
70. 用户坐标系标定 robotServiceUserCoordinateCalibration 104
71. robotServiceOriMatrixToQuaternion 104
72. Base 坐标系转 User 坐标系 baseToUserCoordinate 104
73. F_B 坐标系转 T_B 坐标 111
74. User 坐标系转 Base 坐标系 userToBaseCoordinate 113
75. User 坐标系下位置参数转 Base 坐标系 userCoordPointToBasePoint 120
76. flange 姿态转 Tool 姿态 endOrientation2ToolOrientation 122
77. Tool 姿态转 flange 姿态 toolOrientation2EndOrientation 123
78. 根据位置获取目标路点信息 getTargetWaypointByPosition 125
79. 四元数转欧拉角 quaternionToRPY 128
80. 欧拉角转四元数 RPYToQuaternion 129
81. 根据错误号返回错误信息 getErrDescByCode 129
机械臂控制接口 130
82. 机械臂控制 rootServiceRobotControl 130
83. 返回电源状态 robotServicePowerControl 130
84. 机械臂释放刹车 robotServiceReleaseBrake 130
末端工具接口 130
85. 设置无工具动力学参数 robotServiceSetNoneToolDynamicsParam 130
86. 设置工具的动力学参数 robotServiceSetToolDynamicsParam 131
87. 获取工具的动力学参数 robotServiceGetToolDynamicsParam 132

88. 设置无工具运动学参数 robotServiceSetNoneToolKinematicsParam 132
89. 设置工具的运动学参数 robotServiceSetToolKinematicsParam 133
90. 获取工具的运动学参数 robotServiceGetToolKinematicsParam 134
机械臂相关属性获取与设置 135
91. 获取机械臂当前工作模式 robotServiceGetRobotWorkMode 135
92. 设置机械臂当前工作模式 robotServiceSetRobotWorkMode 135
93. 获取重力分量 robotServiceGetRobotGravityComponent 136
94. 获取当前碰撞等级 robotServiceGetRobotCollisionCurrentService 136
95. 设置机械臂碰撞等级 robotServiceSetRobotCollisionClass 137
96. 获取设备信息 robotServiceGetRobotDevInfoService 137
97. 设置最大加速度 robotServiceSetRobotMaxACC 139
98. 碰撞恢复 robotServiceCollisionRecover 139
99. 获取机械臂当前运行状态 robotServiceGetRobotCurrentState 139
100. 获取 Mac 通信状态 robotServiceGetMacCommunicationStatus 140
101. 判断真实机械臂是否存在 robotServiceGetIsRealRobotExist 140
102. 获取 Joint6 旋转 360° 使能标志 robotServiceGetJoint6Rotate360EnableFlag 140
103. 获取机械臂关节状态 robotServiceGetRobotJointStatus 141
104. 获取机械臂诊断信息 robotServiceGetRobotDiagnosisInfo 142
105. 获取机械臂当前关节角信息 robotServiceGetJointAngleInfo 145
106. 获取实时路点信息 robotServiceGetCurrentWaypointInfo 145
安全 IO 相关 147
107. 使机械臂回初始位 robotServiceSetRobotAtOriginPose 147
108. 通知接口板上位机暂停状态 robotServiceSetRobotOrpePause 147
109. 通知接口板上位机停止状态 robotServiceSetRobotOrpeStop 147
110. 通知接口板上位机错误 robotServiceSetRobotOrpeError 147
111. 解除系统紧急停止输出信号 robotServiceClearSystemEmergencyStop 147
112. 解除缩减模式错误 robotServiceClearReducedModeError 147
113. 防护重置成功 robotServiceRobotSafetyguardResetSucc 147
接口板 IO 147
114. 获取接口板 IO 配置信息 robotServiceGetBoardIOConfig 147
115. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus 149
116. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus 150
117. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus 151
118. 设置接口板 IO 状态 robotServiceSetBoardIOStatus 152
119. 设置接口板 IO 状态 robotServiceSetBoardIOStatus 152
查看联机模式 153
120. 是否在联机模式 robotServiceIsOnlineMode 153

121. 是否在联机主模式 robotServiceIsOnlineMasterMode 153
安全配置 154
122. 获取机械臂安全配置 robotServiceGetRobotSafetyConfig 154
123. 设置机械臂安全配置 robotServiceSetRobotSafetyConfig 155
124. 获取机械臂安全状态 robotServiceGetOrpeSafetyStatus 155
工具 IO 接口 155
125. 设置工具端电源电压类型 robotServiceSetToolPowerVoltageType 155
126. 获取工具端电源电压类型 robotServiceGetToolPowerVoltageType 156
127. 获取工具端的电源电压 robotServiceGetToolPowerVoltageStatus 156
128. 设置工具端电源电压类型 and 所有数字量 IO 的类型 robotServiceSetToolPowerTypeAndDigitalIOType 157
129. 设置工具端数字量 IO 的类型 robotServiceSetToolDigitalIOType 158
130. 获取工具端所有数字量 IO 的状态 robotServiceGetAllToolDigitalIOStatus 159
131. 根据地址设置工具端数字量 IO 的状态 robotServiceSetToolDOStatus 160
132. 根据名称设置工具端数字量 IO 的状态 robotServiceSetToolDOStatus 160
133. 根据名称获取工具端 IO 的状态 robotServiceGetToolIoStatus 161
134. 获取工具端所有 AI 的状态 robotServiceGetAllToolAIStatus 162
固件升级 164
135. robotServiceUpdateRobotBoardFirmware 164
136. robotServiceGetBoardFirmwareUpdateResultService 164
137. robotServiceGetRobotEthernetDeviceName 164
设置关节补偿 164
138. 设置关节碰撞补偿 robotServiceSetRobotJointOffset 164
传送带跟踪 164
139. 设置编码器复位 robotServiceSetConveyorEncoderReset 164
140. 启动传送带 robotServiceSetConveyorStartup 164
141. 停止传送带 robotServiceSetConveyorStop 164
142. 设置传送带方向 robotServiceSetConveyorDir 164
143. 设置手眼标定结果关系 robotServiceSetRobotCameraCalib 164
144. 设置传送带线速度 robotServiceSetConveyorVelc 164
145. 设置编码器距离关系 robotServiceSetEncoderValPerMeter 164
146. 设置传送带起始窗口上限 robotServiceSetStartWindowUpstream 165
147. 设置传送带起始窗口下限 robotServiceSetStartWindowDownstream 165
148. 设置传送带跟踪轨迹下限 robotServiceSetConveyorTrackDownstream 165
149. robotServiceAppendObject2ConveyorTrackQueue 165
150. robotServiceEnableConveyorTrack 165
151. robotServiceGetConveyorEncoderVal 165
152. 设置传送带跟踪的最大速度 robotServiceSetRobotConveyorTrackMaxVelc 165
153. 设置传送带跟踪的最大加速度 robotServiceSetRobotConveyorTrackMaxAcc 165

154. 设置传送带跟踪的系统延时时间 robotServiceSetRobotConveyorSystemDelay 165
155. 设置机械臂工具 robotServiceSetRobotTool 165
八、 错误代码 165

一、 获取 linux C++ SDK 包

SDK 版本为 V1.2.2。
http://download.aubo-robotics.cn:28080/aubo/download/dev/sdk/v1.2.2/linux-cplusplus/

Directory Listing For /dev/sdk/v1.2.2/linux-cplusplus/ - Up To /dev/sdk/v1.2.2

Filename	Size	Last
AuboInterfaceExample.tar.gz	47514.4 kb	Mon, 10 Dec 2018



二、 编程环境

基于 AUBOPE 虚拟机自带的 Qt Creator。打开 Qt Creator 有两种方式：

1、进入虚拟机，通过快速启动栏打开 Qt Creator。

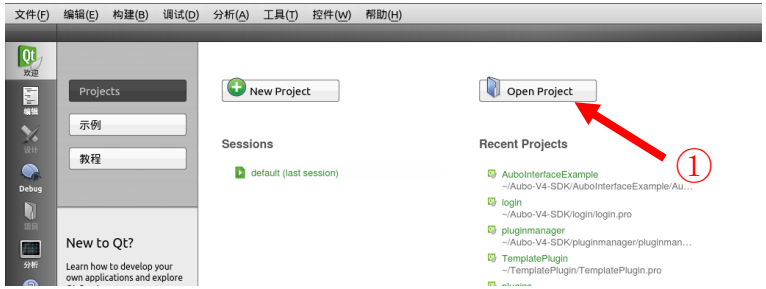


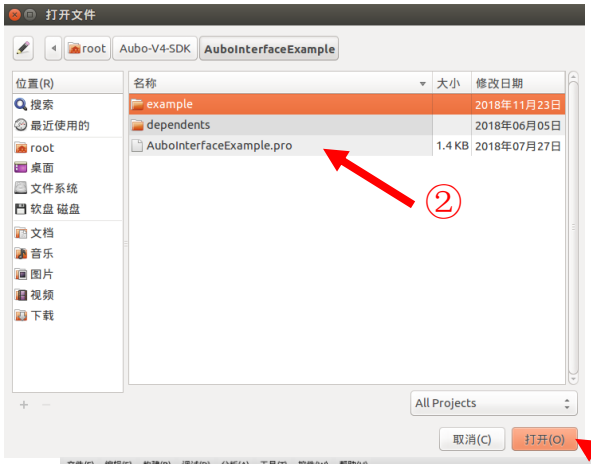
2、【computer】->【usr】->【share】->【applications】->【Qt Creator(Opensource)】。

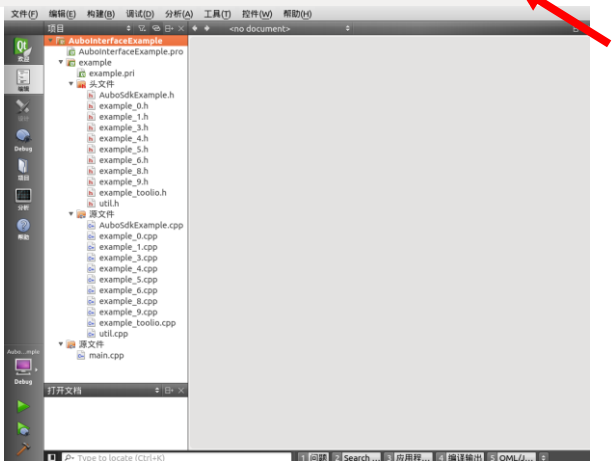


三、 打开 SDK 工程

- 解压缩 SDK 包
- 打开 Qt Creator
- Open Project，打开 AuboInterfaceExample.pro，打开







四、 linux C++ SDK 文件构成

- 《dependents》:AUBO 依赖库。

- 《example》:共 9 个例子，基本覆盖机器人的所有控制接口。
- AuboInterfaceExample.pro：项目文件。
- main.cpp：主函数，运行某个例子（不需要运行的例子注释掉）。

9 个 example

函数名	说明
example_0	工程模板。登录，初始化，上电，断电，退出登录。
example_1	回调函数的方式获取实时路点，末端速度，机械臂的事件　关于机械臂相关状态的获取
example_3	关节运动
example_4	直线运动
example_5	轨迹运动
example_6	直线或关节方式运动至目标位置
example_8	正逆解
example_9	IO 状态获取、IO 控制
example_toolio	工具 IO 操作

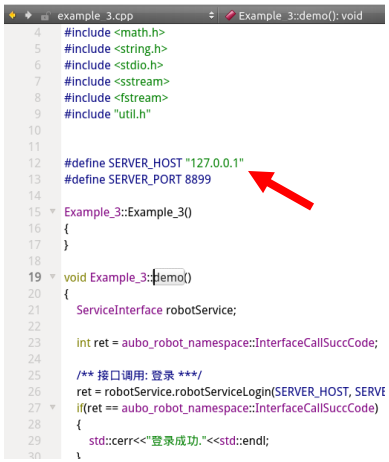
五、　运行 SDK 例子

运行 example_3 关节运动。

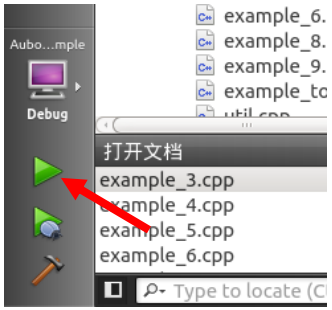
1. 取消 main.cpp 中关节运动函数的注释

```
26    //    Example_1::demo();
27    //    Example_1::getJointStatus();
28
29        //案例3:关节运动
30        Example_3::demo();
31
32        //案例4:直线运动
33    //    Example_4::demo();
```

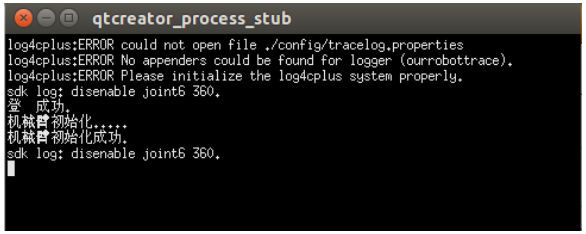
2. 打开 example_3.cpp，修改 IP（机器人 IP）。



3. 点击绿色三角运行按钮，运行 SDK 工程。



4. 终端打印出信息，机械臂动作。

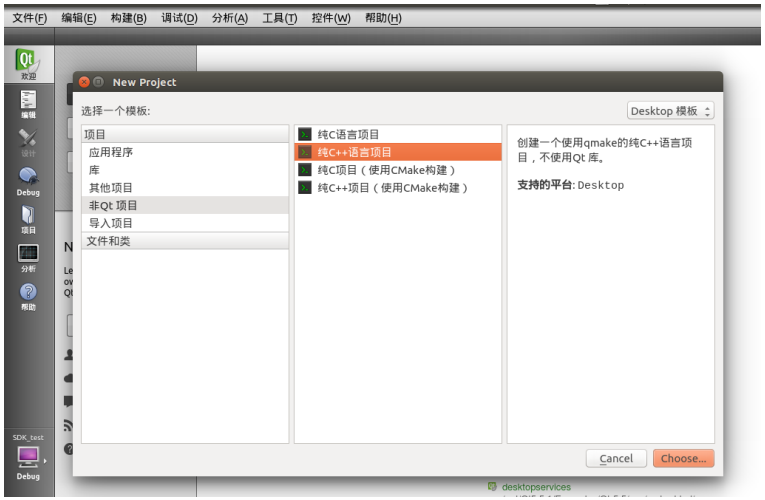


六、 构建自己的开发程序（基于 Qt Creator）

仅供参考。

1. 创建新项目

1. 【欢迎】->【New Project】->【非 Qt 项目】->【纯 C++语言项目】->【Choose】。本例程的工程名为“SDK_test”。



2. 导入 Aubo_linux_C++依赖库

2. 把【AuboInterfaceExample】->【dependents】文件夹拷贝到 SDK_test.pro 同一路径下。



3. 修改 SDK_test.pro 文件，添加以下内容：

```
#********lib select *********************************
unix{
    #32bit OS
    contains(QT_ARCH, i386){
        CONFIG += c++11
        DEFINES += _GLIBCXX_USE_CXX11_ABI=0
        INCLUDEPATH += $$PWD/dependents/robotSDK/inc
        LIBS += $$PWD/dependents/protobuf/linux-x32/lib/libprotobuf.a
        LIBS += $$PWD/dependents/robotController/lib-linux32/libour_alg_i5p.a
        LIBS += -L$$PWD/dependents/log4cplus/linux_x32/lib -llog4cplus
        LIBS += -L$$PWD/dependents/robotSDK/lib/linux_x32/ -lauborobotcontroller
        LIBS += -lpthread
```

```
    }
    #64bit OS
    contains(QT_ARCH, x86_64){
        CONFIG += c++11
        INCLUDEPATH += $$PWD/dependents/robotSDK/inc
        LIBS += $$PWD/dependents/protobuf/linux-x64/lib/libprotobuf.a
        LIBS += $$PWD/dependents/robotController/lib-linux64/libour_alg_i5p.a
        LIBS += -L$$PWD/dependents/log4cplus/linux_x64/lib -llog4cplus
        LIBS += -L$$PWD/dependents/libconfig/linux_x64/lib/ -lconfig
        LIBS += -L$$PWD/dependents/robotSDK/lib/linux_x64/ -lauborobotcontroller
        LIBS += -lpthread
    }
}
```

3. 添加程序内容



4. 向 SDK_test 中添加新文件 -> C++ -> C++ Class，类名填写为“login_out”

5. 编写《login_out.h》如下：

```
#ifndef LOGIN_OUT_H
#define LOGIN_OUT_H

#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

class login_out
{
public:
    login_out();
```

```
    static void robot_login();
};

#endif // LOGIN_OUT_H
```

6. 编写《login_out.cpp》如下：

```
#include "login_out.h"
#include <unistd.h>

#define SERVER_HOST "127.0.0.1"
#define SERVER_PORT 8899

ServiceInterface robotService;
login_out::login_out()
{
}

void login_out::robot_login()
{
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr<<"login 成功."<<std::endl;
    }
    else
    {
        std::cerr<<"login 失败."<<std::endl;
    }

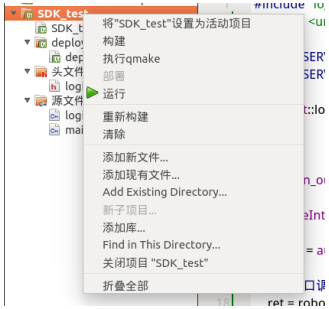
    sleep(2);

    /** 机械臂 Shutdown **/
    ret = robotService.robotServiceRobotShutdown();
    if( ret != aubo_robot_namespace::InterfaceCallSuccCode )
    {
        std::cerr << "robot shutdown failed!" << std::endl;
```

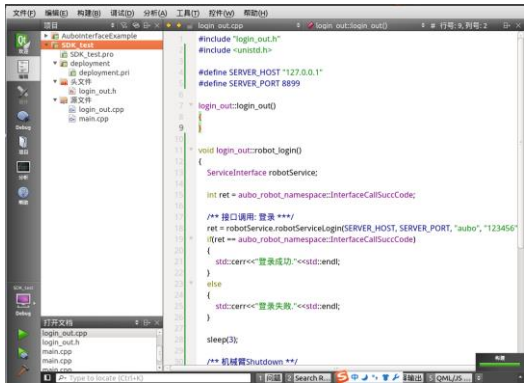
```
    }
    /** 接口调用: 退出登录  **/
    ret = robotService.robotServiceLogout();
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr<<"logout 成功."<<std::endl;
    }
    else
    {
        std::cerr<<"logout 失败."<<std::endl;
    }
}
```

4. 编译、运行

7. 右键【SDK_test】->【构建】



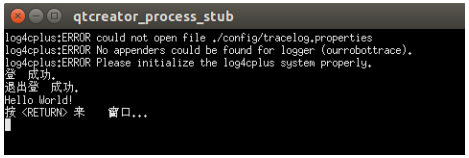
8. 编译器右下角出现绿色进度条，表示构建成功。



9. 点击绿色三角，运行程序



10. 弹出控制台



七、 linux_C++接口函数示例

使用 linux_C++_SDK 过程中经常需要参考以下两个头文件：
11. 《AuboRobotMetaType.h》：定义了 AUBO 相关的数据类型，如路点、关节状态、坐标系类型等等。
12. 《serviceinterface.h》：定义了所有的接口函数原型。

机械臂登录与断开

1. 登录 robotServiceLogin

```
int  robotServiceLogin(const char* host, int port, const char *userName, const char* possword);
```

函数功能	该函数用于与机械臂服务器建立网络连接，该函数调用成功，是使用其他接口的前提。
参数描述	1、 host 机械臂服务器 IP 地址，即控制器的 IP; 2、 port 默认为 8899 3、 userName 用户名默认为： AUBO 4、 possword 密码默认为:123456
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：登录到机器人，成功则打印“login 成功”；否则打印“login 失败”。
	`#define SERVER_HOST "127.0.0.1"`
	`#define SERVER_PORT 8899`
	
	`ServiceInterface robotService;`
	`int ret = aubo_robot_namespace::InterfaceCallSuccCode;`
	`/** 接口调用: 登录 ***/`
	`ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");`
	`if(ret == aubo_robot_namespace::InterfaceCallSuccCode)`
	`{`
	`  std::cerr<<"login 成功."<<std::endl;`
	`}`
	`else`
	`{`
	`  std::cerr<<"login 失败."<<std::endl;`
	`}`

输出	login成功.

2. 退出登录 robotServiceLogout

int robotServiceLogout();	
函数功能	退出登录
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：退出登录机器人，成功则打印“logout 成功”。
	`/** 接口调用：退出登录 **/`
	`ret = robotService.robotServiceLogout();`
	`if(ret == aubo_robot_namespace::InterfaceCallSuccCode)`
	`{`
	`std::cerr<<"logout 成功."<<std::endl;`
	`}`

输出	logout成功.

3. 当前连接状态 robotServiceGetConnectStatus

void robotServiceGetConnectStatus(bool &connectStatus);	
函数功能	该函数用于查看与机械臂服务器的连接状态
参数描述	connectStatus 是一个传出参数，网络处于连通状态返回 true;否则返回 false.
返回值	

示例	示例讲解：查看当前登录状态。
	`//获取连接状态`
	`bool m_connectStatus;`
	`robotService.robotServiceGetConnectStatus(m_connectStatus);`
	`std::cout << "current connect status is " << m_connectStatus << std::endl;`

输出	current connect status is 1

4. 握手 robotServiceRobotHandShake

机械臂上电与断电

5. 机械臂上电 rootServiceRobotStartup

```
int rootServiceRobotStartup(const aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam, uint8 collisionClass, bool readPose,bool staticCollisionDetect, int boardBaxAcc, aubo_robot_namespace::ROBOT_SERVICE_STATE &result, bool IsBolck = true);
```

函数功能	该函数用于启动机械臂，包括上电，松刹车，设置碰撞等级，设置动力学参数等，该函数完成需要的时间比较长，因此可以通过设置是否阻塞来调整函数返回的时间，当设置为非阻塞是，函数调用后立即返回，调用结果通过事件来通知。当设置为阻塞时，参数返回值代表接口是否被调用成功， result 表示机械臂启动结果。
参数描述	toolDynamicsParam：动力学参数，如果末端夹持工具，此参数应该根据具体的来设定；如果末端未夹持工具，将此参数的各项设置为 0。 collisionClass：碰撞等级 readPose：是否允许读取位置，默认是 true staticCollisionDetect:默认为 true boardBaxAcc： result：传出参数，初始化结果，具体参考 ROBOT_SERVICE_STATE 类型。机械臂启动结果只有 result==ROBOT_SERVICE_WORKING 表示机械臂启动成功，否则表示启动失败。
	`/** * @brief 机械臂启动完成状态 */ enum ROBOT_SERVICE_STATE{ ROBOT_SERVICE_READY=0, ROBOT_SERVICE_STARTING, ROBOT_SERVICE_WORKING, ROBOT_SERVICE_CLOSING, ROBOT_SERVICE_CLOSED, ROBOT_SETVICE_FAULT_POWER, ROBOT_SETVICE_FAULT_BRAKE, ROBOT_SETVICE_FAULT_NO_ROBOT };`
	IsBolck：是否阻塞，默认 true。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	示例讲解：机械臂上电，工具动力学参数为 0，碰撞等级 6。打印启动结果 result。

```cpp
if( m_connectStatus == true )
 {
     /** 如果是连接真实机械臂，需要对机械臂进行初始化  **/
     aubo_robot_namespace::ROBOT_SERVICE_STATE result;

     //工具动力学参数
     aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
     toolDynamicsParam.payload = 0;
     toolDynamicsParam.positionX = 0;
     toolDynamicsParam.positionY  = 0;
     toolDynamicsParam.positionZ = 0;
     toolDynamicsParam.toolInertia.xx = 0;
     toolDynamicsParam.toolInertia.xy = 0;
     toolDynamicsParam.toolInertia.xz = 0;
     toolDynamicsParam.toolInertia.yy = 0;
     toolDynamicsParam.toolInertia.yz= 0;
     toolDynamicsParam.toolInertia.zz = 0;
     //机械臂上电
     ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6        /*碰撞等级*/,
                                                true     /*是否允许读取位姿  默认为true*/,
                                                true,    /*保留默认为true */
                                                1000,    /*保留默认为1000 */
                                                result); /*机械臂初始化*/
     if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
     {
         std::cerr<<"机械臂初始化成功."<<std::endl;
     }
     else
     {
         std::cerr<<"机械臂初始化失败."<<std::endl;
     }
     //打印机械臂启动结果
     std::cout << "robot startup result is " << result << std::endl;
 }
```

输出



6. 机械臂断电 rootServiceRobotShutdown

```
int  robotServiceRobotShutdown(bool IsBolck = true);
```

函数功能	机械臂断电
参数描述	IsBolck：是否阻塞，默认 true。IsBolck 调用接口时是否阻塞：设置阻塞时，直到机械臂关机后函数返回；设置非阻塞时，立即返回，结果通过事件推送来返回。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：机械臂断电。
	`/** 机械臂 Shutdown **/`
	`ret = robotService.robotServiceRobotShutdown();`
	`if( ret != aubo_robot_namespace::InterfaceCallSuccCode )`
	`{`
	`std::cerr << "robot shutdown failed!" << std::endl;`
	`}`

输出	

信息推送

7. 使能实时关节状态推送 robotServiceSetRealTimeJointStatusPush

8. 实时关节信息推送 robotServiceRegisterRealTimeJointStatusCallback

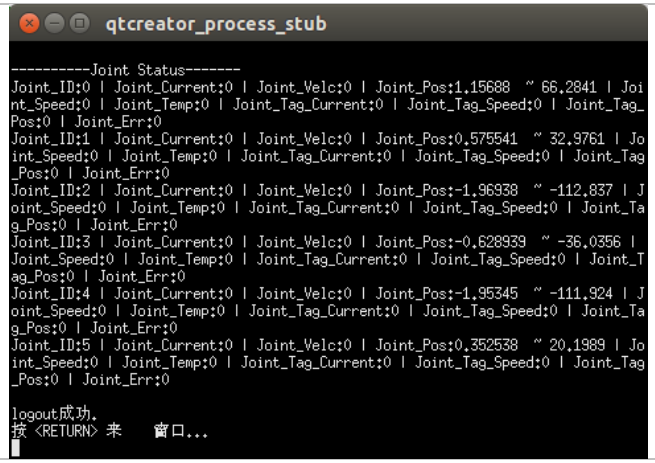
```
int  robotServiceRegisterRealTimeJointStatusCallback(RealTimeJointStatusCallback ptr, void  *arg);
```

函数功能	该函数用于注册获取“实时关节信息的回调函数”到系统，成功注册后,服务器会通过回调函数实时推送当前的关节状态信息。频率 30ms。

参数描述	1、 ptr 为获取实时关节状态信息的回调函数指针，当 ptr==NULL 时，相当于取消回调函数的注册;
	`typedef struct PACKED`
	`{`
	`int jointCurrentI; /**< Current of driver 关节电流*/`
	`int jointSpeedMoto; /**< Speed of driver 关节速度*/`

	``` float jointPosJ; /**< Current position in radian 关节角*/ float jointCurVol; /**< Rated voltage of motor. Unit: mV 关节电压*/ float jointCurTemp; /**< Current temprature of joint 当前温度*/ int jointTagCurrentI; /**< Target current of motor 电机目标电流*/ float jointTagSpeedMoto; /**< Target speed of motor 电机目标速度*/ float jointTagPosJ; /**< Target position of joint in radian 目标关节角 */ uint16 jointErrorNum; /**< Joint error of joint num 关节错误码 */ }JointStatus; ``` 2、arg 这个参数系统不做任何处理，只是进行缓存，当系统调用已注册回调函数时该参数会通过回调函数的参数传回。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：关节状态实时回调。
	``
	void login_out::RealTimeJointStatusCallback(const aubo_robot_namespace::JointStatus *jointStatusPtr, int size,void *arg)
	{
	    (void)arg;
	
	    std::cout<<std::endl<<"----------Joint Status-------" << std::endl;
	    for(int i=0; i<size; i++)
	    {
	        std::cout<<"Joint_ID:" <<i<<"
		" ;
		        std::cout<<"Joint_Current:" <<jointStatusPtr[i].jointCurrentI<<"
			";
			        std::cout<<"Joint_Velc:" <<jointStatusPtr[i].jointSpeedMoto<<"
				";
				        std::cout<<"Joint_Pos:" <<jointStatusPtr[i].jointPosJ<<" "<<" ~ "<<jointStatusPtr[i].jointPosJ*180.0/3.1415926 << "
					";
					        std::cout<<"Joint_Speed:" <<jointStatusPtr[i].jointCurVol<<"
						";
						        std::cout<<"Joint_Temp:" <<jointStatusPtr[i].jointCurTemp<<"
							";
							        std::cout<<"Joint_Tag_Current:" <<jointStatusPtr[i].jointTagCurrentI<<"
								";
								        std::cout<<"Joint_Tag_Speed:" <<jointStatusPtr[i].jointTagSpeedMoto<<"
									";
									        std::cout<<"Joint_Tag_Pos:" <<jointStatusPtr[i].jointTagPosJ<<"
										";
										        std::cout<<"Joint_Err:" <<jointStatusPtr[i].jointErrorNum <<std::endl;
										    }
										    std::cout<<std::endl;
										}
										
										    //机械臂实时关节信息推送
										    robotService.robotServiceRegisterRealTimeJointStatusCallback(login_out::RealTimeJointStatusCallback,NULL);
										``

输出	

9. 使能实时路点推送 robotServiceSetRealTimeRoadPointPush

10.实时路点信息推送 robotServiceRegisterRealTimeRoadPointCallback

```
int  robotServiceRegisterRealTimeRoadPointCallback(const RealTimeRoadPointCallback ptr, void  *arg);
```

函数功能	该函数用于注册获取“实时路点信息的回调函数”到系统，成功注册后,服务器会通过回调函数实时推送机械臂当前路点信息。
参数描述	1、 ptr 为获取实时路点信息的回调函数指针，当 ptr==NULL 时，相当于取消回调函数的注册; 2、这个参数系统不做任何处理，只是进行缓存，当系统调用已注册回调函数时该参数会通过回调函数的参数传回。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：路点信息实时回调。
	`void login_out::RealTimeWaypointCallback(const aubo_robot_namespace::wayPoint_S *wayPointPtr, void *arg)`
	`{`
	`  (void)arg;`
	`  aubo_robot_namespace::wayPoint_S wayPoint = *wayPointPtr;`
	`  std::cout<<std::endl<<"start-------------路点信息---------------"<<std::endl;`
	`  //位置信息`
	`  std::cout<<"位置信息: ";`
	`  std::cout<<"x:"<<wayPoint.cartPos.position.x<<"  ";`
	`  std::cout<<"y:"<<wayPoint.cartPos.position.y<<"  ";`
	`  std::cout<<"z:"<<wayPoint.cartPos.position.z<<std::endl;`

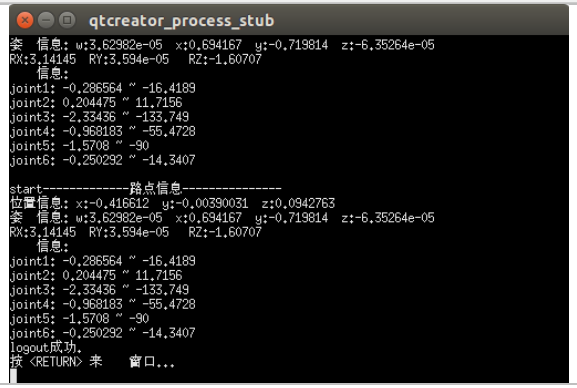
```
    //姿态信息
    std::cout<<"姿态信息: ";
    std::cout<<"w:"<<wayPoint.orientation.w<<"  ";
    std::cout<<"x:"<<wayPoint.orientation.x<<"  ";
    std::cout<<"y:"<<wayPoint.orientation.y<<"  ";
    std::cout<<"z:"<<wayPoint.orientation.z<<std::endl;

    aubo_robot_namespace::Rpy tempRpy;
    robotService.quaternionToRPY(wayPoint.orientation,tempRpy);
    std::cout<<"RX:"<<tempRpy.rx<<"  RY:"<<tempRpy.ry<<"   RZ:"<<tempRpy.rz<<std::endl;

    //关节信息
    std::cout<<"关节信息: "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout<<"joint"<<i+1<<": "<<wayPoint.jointpos[i]<<" ~ "<<wayPoint.jointpos[i]*180.0/M_PI<<std::endl;
    }
}

    //机械臂实时路点推送
    robotService.robotServiceRegisterRealTimeRoadPointCallback(login_out::RealTimeWaypointCallback,NULL);
```

输出

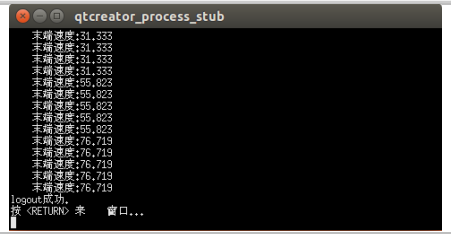


11.使能实时末端速度推送 robotServiceSetRealTimeEndSpeedPush

12.实时末端速度推送 robotServiceRegisterRealTimeRoadPointCallback

`int robotServiceRegisterRealTimeEndSpeedCallback(const RealTimeEndSpeedCallback ptr, void *arg);`	
函数功能	该函数用于注册获取“实时末端速度的回调函数”到系统，成功注册后,服务器会通过回调函数实时推送末端速度信息。
参数描述	1、 ptr 为获取末端速度信息的回调函数指针，当 ptr==NULL 时，相当于取消回调函数的注册; 2、这个参数系统不做任何处理，只是进行缓存，当系统调用已注册回调函数时该参数会通过回调函数的参数传回。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：末端速度实时回调。
	`void login_out::RealTimeEndSpeedCallback(double speed, void *arg)`
	`{`
	`  (void)arg;`
	`  std::cout<<"实时末端速度:"<<speed<<std::endl;`
	`}`
	
	`//机械臂实时末端速度推送`
	`robotService.robotServiceRegisterRealTimeEndSpeedCallback(login_out::RealTimeEndSpeedCallback,NULL);`

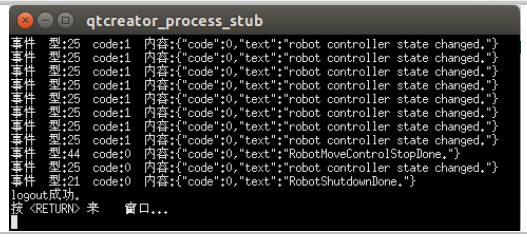
输出	

13.机械臂事件实时推送 robotServiceRegisterRobotEventInfoCallback

`int robotServiceRegisterRobotEventInfoCallback(RobotEventCallback ptr, void *arg);`	
函数功能	该函数用于注册获取“的回调函数”到系统，成功注册后,服务器会通过回调函数实时推送事件信息。
参数描述	1、 ptr 为获取事件信息的回调函数指针，当 ptr==NULL 时，相当于取消回调函数的注册; 2、这个参数系统不做任何处理，只是进行缓存，当系统调用已注册回调函数时该参数会通过回调函数的参数传回。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：机械臂事件实时回调。
	`void login_out::RealTimeEventInfoCallback(const aubo_robot_namespace::RobotEventInfo *pEventInfo, void *arg)`
	`{`
	`(void)arg;`
	`aubo_robot_namespace::RobotEventInfo eventInfo = *pEventInfo;`
	`std::cout<<"事件类型:"<<eventInfo.eventType <<" code:"<<eventInfo.eventCode<<" 内容:"<<eventInfo.eventContent<<std::endl;`
	`}`
	
	`//机械臂实时事件推送`
	`robotService.robotServiceRegisterRobotEventInfoCallback(login_out::RealTimeEventInfoCallback,NULL);`

输出	

运动模块

14.初始化运动属性 robotServiceInitGlobalMoveProfile()

`int robotServiceInitGlobalMoveProfile();`	
函数功能	对运动属性进行初始化，将个属性设置为初始值
参数描述	1.关节型运动的最大速度和最大加速度，默认每个关节的最大加速度为 25 度每秒方，默认每个关节的最大速度为 25 度每秒； 2.末端型运动的最大线速度和最大线加速度，默认最大线加速度为 0.03 米每秒方，默认最大线速度为 0.03 米每秒； 3.末端型运动的最大角速度和最大角加速度，默认最大角加速度为 100 度每秒方，默认最大角速度为 100 度每秒； 4.轨迹运动的路点容器，默认容器为空； 5.轨迹运动的交融半径，默认交融半径为 0； 6.轨迹运动中圆轨迹的圈数，默认为 0； 7.运动属性至偏移量属性，默认为无偏移； 8.设置示教运动的坐标系，默认为基座标系；
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//初始化运动属性` `robotService.robotServiceInitGlobalMoveProfile();`
输出	

15. 设置机械臂关节型运动最大加速度 robotServiceSetGlobalMoveJointMaxAcc

`int robotServiceSetGlobalMoveJointMaxAcc (const aubo_robot_namespace::JointVelcAccParam &moveMaxAcc);`	
函数功能	设置关节型运动最大加速度 关节型运动包含： * 关节运动； * 示教运动中的关节示教（JOINT1，JOINT2，JOINT3，JOINT4，JOINT5，JOINT6） * 轨迹运动下的（JIONT_CUBICSPLINE，JOINT_UBSPLINEINTP）
参数描述	设置关节型运动的最大加速度,关节型运动的最大加速度是 180°/s2，单位 rad
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//设置关节运动最大加速度`
	`aubo_robot_namespace::JointVelcAccParam m_set_jointacc;`
	`m_set_jointacc.jointPara[0] = 30*M_PI/180;`
	`m_set_jointacc.jointPara[1] = 30*M_PI/180;`
	`m_set_jointacc.jointPara[2] = 30*M_PI/180;`
	`m_set_jointacc.jointPara[3] = 30*M_PI/180;`
	`m_set_jointacc.jointPara[4] = 30*M_PI/180;`
	`m_set_jointacc.jointPara[5] = 30*M_PI/180;`
	`robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);`

输出	

16. 设置机械臂关节型运动最大速度 robotServiceSetGlobalMoveJointMaxVelc

`int robotServiceSetGlobalMoveJointMaxVelc(const aubo_robot_namespace::JointVelcAccParam &moveMaxVelc);`	
函数功能	设置关节型运动最大速度 关节型运动包含： * 关节运动；

	*　　示教运动中的关节示教（JOINT1，JOINT2，JOINT3，JOINT4，JOINT5，JOINT6） *　　轨迹运动下的（JIONT_CUBICSPLINE，JOINT_UBSPLINEINTP）
参数描述	设置关节型运动的最大速度,关节型运动的最大速度是 180° /s，角度单位 rad
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//设置关节运动最大速度`
	`aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;`
	`m_set_jointvelc.jointPara[0] = 15*M_PI/180;`
	`m_set_jointvelc.jointPara[1] = 15*M_PI/180;`
	`m_set_jointvelc.jointPara[2] = 15*M_PI/180;`
	`m_set_jointvelc.jointPara[3] = 15*M_PI/180;`
	`m_set_jointvelc.jointPara[4] = 15*M_PI/180;`
	`m_set_jointvelc.jointPara[5] = 15*M_PI/180;`
	`robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);`

输出	

17.获取机械臂关节型运动最大加速度 robotServiceGetGlobalMoveJointMaxAcc

```
void  robotServiceGetGlobalMoveJointMaxAcc (aubo_robot_namespace::JointVelcAccParam        &moveMaxAcc);
```

函数功能	获取关节型运动最大加速度 关节型运动包含： *　　关节运动； *　　示教运动中的关节示教（JOINT1，JOINT2，JOINT3，JOINT4，JOINT5，JOINT6） *　　轨迹运动下的（JIONT_CUBICSPLINE，JOINT_UBSPLINEINTP）
参数描述	moveMaxAcc 是一个传出参数，表示关节运动最大速度，单位 rad/s2
返回值	

示例	`//获取关节运动最大加速度`
	`aubo_robot_namespace::JointVelcAccParam m_get_jointacc;`
	`robotService.robotServiceGetGlobalMoveJointMaxAcc(m_get_jointacc);`
	`std::cout << "joint max acc = { " << m_get_jointacc.jointPara[0] << ","`
	`<< m_get_jointacc.jointPara[1] << ","`
	`<< m_get_jointacc.jointPara[2] << ","`
	`<< m_get_jointacc.jointPara[3] << ","`
	`<< m_get_jointacc.jointPara[4] << ","`
	`<< m_get_jointacc.jointPara[5] <<" }"`
	`<< std::endl;`

输出	joint max acc = { 0.523599,0.523599,0.523599,0.523599,0.523599,0.523599 }

18. 获取机械臂关节型运动最大速度 robotServiceGetGlobalMoveJointMaxVelc

```
void  robotServiceGetGlobalMoveJointMaxVelc(aubo_robot_namespace::JointVelcAccParam        &moveMaxVelc);
```

函数功能	获取关节型运动最大速度 关节型运动包含： *　　关节运动； *　　示教运动中的关节示教（JOINT1，JOINT2，JOINT3，JOINT4，JOINT5，JOINT6） *　　轨迹运动下的（JIONT_CUBICSPLINE，JOINT_UBSPLINEINTP）
参数描述	moveMaxVelc 是一个传出参数，表示关节运动最大加速度，单位 rad/s
返回值	

示例	`//获取关节运动最大速度`
	`aubo_robot_namespace::JointVelcAccParam m_get_jointvelc;`
	`robotService.robotServiceGetGlobalMoveJointMaxVelc(m_get_jointvelc);`
	`std::cout << "joint max velc = { " << m_get_jointvelc.jointPara[0] << ","`
	`<< m_get_jointvelc.jointPara[1] << ","`
	`<< m_get_jointvelc.jointPara[2] << ","`
	`<< m_get_jointvelc.jointPara[3] << ","`
	`<< m_get_jointvelc.jointPara[4] << ","`
	`<< m_get_jointvelc.jointPara[5] <<" }"`
	`<< std::endl;`

输出	joint max velc = { 0.261799,0.261799,0.261799,0.261799,0.261799,0.261799 }

19. 设置末端型运动最大加速度 robotServiceSetGlobalMoveEndMaxLineAcc

```
int   robotServiceSetGlobalMoveEndMaxLineAcc (double  moveMaxAcc);
```

函数功能	设置末端型运动最大加速度 末端型包含：　直线运动（MODEL）； *　　示教运动中的位置示教和姿态示教（MOV_X，MOV_Y，MOV_Z，ROT_X，ROT_Y，ROT_Z）； *　　轨迹运动下的（ARC_CIR，CARTESIAN_MOVEP，CARTESIAN_CUBICSPLINE，CARTESIAN_UBSPLINEINTP）
参数描述	末端型运动最大加速度，最大 2m/s2。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//设置末端型运动最大加速度`
	`double m_set_end_lineacc = 0.3;`
	`robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);`

输出	

20.设置末端型运动最大速度 robotServiceSetGlobalMoveEndMaxLineVelc

`int robotServiceSetGlobalMoveEndMaxLineVelc (double moveMaxVelc);`	
函数功能	设置末端型运动最大速度 末端型包含：直线运动（MODEL）; * 示教运动中的位置示教和姿态示教（MOV_X，MOV_Y，MOV_Z，ROT_X，ROT_Y，ROT_Z）; * 轨迹运动下的（ARC_CIR， CARTESIAN_MOVEP， CARTESIAN_CUBICSPLINE， CARTESIAN_UBSPLINEINTP）
参数描述	末端型运动最大速度，最大 2m/s。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//设置末端型运动最大速度`
	`double m_set_end_linevelc = 0.1;`
	`robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);`

输出	

21.获取末端型运动最大加速度 robotServiceGetGlobalMoveEndMaxLineAcc

`void robotServiceGetGlobalMoveEndMaxLineAcc (double &moveMaxAcc);`	
函数功能	获取末端型运动最大加速度 末端型包含：直线运动（MODEL）; * 示教运动中的位置示教和姿态示教（MOV_X，MOV_Y，MOV_Z，ROT_X，ROT_Y，ROT_Z）; * 轨迹运动下的（ARC_CIR， CARTESIAN_MOVEP， CARTESIAN_CUBICSPLINE， CARTESIAN_UBSPLINEINTP）
参数描述	moveMaxAcc 是一个传出参数，末端型运动最大加速度，最大 2m/s2。
返回值	
示例	`//获取末端型运动最大加速度`

	double m_get_end_lineacc; robotService.robotServiceGetGlobalMoveEndMaxLineAcc(m_get_end_lineacc); std::cout << "end line max acc = " << m_get_end_lineacc << std::endl;
输出	end line max acc = 0.3

22. 获取末端型运动最大速度 robotServiceGetGlobalMoveEndMaxLineVelc

```
void  robotServiceGetGlobalMoveEndMaxLineVelc(double  &moveMaxVelc);
```

函数功能	获取末端型运动最大速度 末端型包含： 直线运动（MODEL）; * 示教运动中的位置示教和姿态示教（MOV_X，MOV_Y，MOV_Z，ROT_X，ROT_Y，ROT_Z）; * 轨迹运动下的（ARC_CIR， CARTESIAN_MOVEP， CARTESIAN_CUBICSPLINE， CARTESIAN_UBSPLINEINTP）
参数描述	moveMaxVelc 是一个传出参数，末端型运动最大速度，最大 2m/s。
返回值	

示例	//获取末端型运动最大速度
	double m_get_end_linevelc;
	robotService.robotServiceGetGlobalMoveEndMaxLineVelc(m_get_end_linevelc);
	std::cout << "end line max velc = " << m_get_end_linevelc << std::endl;

输出	end line max velc = 0.1

23. 设置末端型运动旋转最大角加速度 robotServiceSetGlobalMoveEndMaxAngleAcc

24. 设置末端型运动旋转最大角速度 robotServiceSetGlobalMoveEndMaxAngleVelc

25. 获取末端型运动旋转最大角加速度 robotServiceGetGlobalMoveEndMaxAngleAcc

26. 获取末端型运动旋转最大角速度 robotServiceGetGlobalMoveEndMaxAngleVelc

27.清空路点容器 robotServiceClearGlobalWayPointVector

`void robotServiceClearGlobalWayPointVector();`	
函数功能	清除路点容器，通常用于新的轨迹运动之前
参数描述	
返回值	
示例	`//清空全局路点容器` `robotService.robotServiceClearGlobalWayPointVector();`
输出	

28.添加全局路点 robotServiceAddGlobalWayPoint

`int robotServiceAddGlobalWayPoint(const aubo_robot_namespace::wayPoint_S &wayPoint);`	
函数功能	添加路点，用于轨迹运动
参数描述	flange_center 在 Base 坐标系下的路点信息,wayPoint_S 可只传 jointpos(系统可计算出 position 和 orientation);而不能只传 position 和 orientation,否则表示零位。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`aubo_robot_namespace::wayPoint_S wp1;`
	`wp1.jointpos[0] = 60.443151*M_PI/180;`
	`wp1.jointpos[1] = 42.275463*M_PI/180;`
	`wp1.jointpos[2] = -97.679737*M_PI/180;`
	`wp1.jointpos[3] = -49.990510*M_PI/180;`
	`wp1.jointpos[4] = -90.007372*M_PI/180;`
	`wp1.jointpos[5] = 62.567046*M_PI/180;`
	`wp1.cartPos.position.x = -0.199819;`
	`wp1.cartPos.position.y = -0.598529;`
	`wp1.cartPos.position.z = -0.431393;`
	`aubo_robot_namespace::Rpy rpy_tmp;`
	`rpy_tmp.rx = 179.977173*M_PI/180;`
	`rpy_tmp.ry = -0.027977*M_PI/180;`
	`rpy_tmp.rz = -92.123917*M_PI/180;`
	`robotService.RPYToQuaternion(rpy_tmp,wp1.orientation);`
	
	`//添加路点`

	robotService.robotServiceAddGlobalWayPoint(wp1);
输出	

29.添加全局路点 robotServiceAddGlobalWayPoint

int robotServiceAddGlobalWayPoint(const double jointAngle[aubo_robot_namespace::ARM_DOF]);	
函数功能	添加路点，用于轨迹运动
参数描述	关节角形式路点
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	double wp1[6] = {};
	wp1[0] = 60.443151*M_PI/180;
	wp1[1] = 42.275463*M_PI/180;
	wp1[2] = -97.679737*M_PI/180;
	wp1[3] = -49.990510*M_PI/180;
	wp1[4] = -90.007372*M_PI/180;
	wp1[5] = 62.567046*M_PI/180;
	
	double wp2[6] = {};
	wp2[0] = 83.411541*M_PI/180;
	wp2[1] = 39.625360*M_PI/180;
	wp2[2] = -103.796807*M_PI/180;
	wp2[3] = -53.491856*M_PI/180;
	wp2[4] = -90.021641*M_PI/180;
	wp2[5] = 85.530279*M_PI/180;
	
	double wp3[6] = {};
	wp3[0] = 81.206455*M_PI/180;
	wp3[1] = 28.381980*M_PI/180;
	wp3[2] = -129.233955*M_PI/180;
	wp3[3] = -67.700289*M_PI/180;
	wp3[4] = -90.019516*M_PI/180;
	wp3[5] = 83.325883*M_PI/180;
	
	//清空全局路点容器
	robotService.robotServiceClearGlobalWayPointVector();
	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp1);

	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp2);
	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp3);

输出	

30.获取全局路点容器 robotServiceGetGlobalWayPointVector

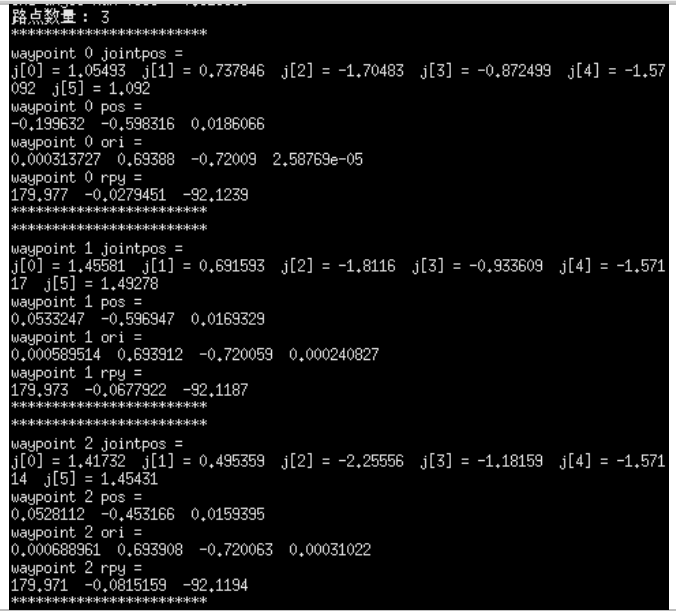
```
void  robotServiceGetGlobalWayPointVector(std::vector<aubo_robot_namespace::wayPoint_S> &wayPointVector);
```

函数功能	获取路点容器
参数描述	为传出参数，表示全局路点容器
返回值	
示例	

```
    //获取路点列表
    std::vector<aubo_robot_namespace::wayPoint_S> waypointVector;
    robotService.robotServiceGetGlobalWayPointVector(waypointVector);
    std::cout << "路点数量:  " << waypointVector.size() << std::endl;
    for( int i = 0; i < waypointVector.size(); i++)
    {
        std::cout << "************************" << std::endl;
        //打印路点关节角
        std::cout << "waypoint " << i << " jointpos = " << std::endl;
        for(int j = 0; j < 6; j++)
        {
            std::cout << "j[" << j << "] = " << waypointVector[i].jointpos[j] << "  ";
        }
        std::cout << std::endl;
        //打印路点位置参数
        std::cout << "waypoint " << i << " pos = " << std::endl;
        std::cout << waypointVector[i].cartPos.position.x << "  ";
        std::cout << waypointVector[i].cartPos.position.y << "  ";
        std::cout << waypointVector[i].cartPos.position.z << "  ";
        std::cout << std::endl;
        //打印路点姿态参数，四元数
        std::cout << "waypoint " << i << " ori = " << std::endl;
        std::cout << waypointVector[i].orientation.w << "  ";
        std::cout << waypointVector[i].orientation.x << "  ";
        std::cout << waypointVector[i].orientation.y << "  ";
```

```
        std::cout << waypointVector[i].orientation.z << "  ";
        std::cout << std::endl;
        //打印路点姿态参数，欧拉角
        std::cout << "waypoint " << i << " rpy = " << std::endl;
        aubo_robot_namespace::Rpy rpy_waypointVector;
        robotService.quaternionToRPY(waypointVector[i].orientation,rpy_waypointVector);
        std::cout << rpy_waypointVector.rx*180/M_PI << "  ";
        std::cout << rpy_waypointVector.ry*180/M_PI << "  ";
        std::cout << rpy_waypointVector.rz*180/M_PI << "  ";
        std::cout << std::endl;
        std::cout << "************************" << std::endl;
    }
```

输出



31. 设置轨迹运动中的交融半径 robotServiceSetGlobalBlendRadius

```
int   robotServiceSetGlobalBlendRadius(float value);
```

函数功能	设置轨迹运动如 movep 的交融半径
参数描述	0.0 ~ 0.05，单位 m
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//设置交融半径`
	`float blendradius = 0.05;`
	`std::cout << robotService.robotServiceSetGlobalBlendRadius(blendradius) << std::endl;`

输出	`set blendradius ret is 0`

32. 获取交融半径值 robotServiceGetGlobalBlendRadius

```
float robotServiceGetGlobalBlendRadius();
```

函数功能	获取轨迹运动如 movep 的交融半径
参数描述	
返回值	返回交融半径值，单位 m

示例	`//设置交融半径`
	`float blendradius = 0.05;`
	`std::cout << "set blendradius ret is " << robotService.robotServiceSetGlobalBlendRadius(blendradius) << std::endl;`
	
	`//获取交融半径值`
	`float blend;`
	`blend = robotService.robotServiceGetGlobalBlendRadius();`
	`std::cout << "交融半径值为: " << blend << std::endl;`

输出	`set blendradius ret is 0` `交融半径值为:0.05`

33. robotServiceGetTrackPlaybackCycle

34. robotServiceSetTrackPlaybackCycle

35. 设置圆运动圈数 robotServiceSetGlobalCircularLoopTimes

```
void  robotServiceSetGlobalCircularLoopTimes(int times);
```

函数功能	设置圆运动圈数，在轨迹类型为 ARD_CIR 时有效
参数描述	times = 0 时，表示圆弧运动； times > 0 时，表示圆运动，且表示圆的圈数
返回值	
示例	`//设置圆运动圈数` `robotService.robotServiceSetGlobalCircularLoopTimes(3);`
输出	

36. 获取圆运动圈数 robotServiceGetGlobalCircularLoopTimes

```
int   robotServiceGetGlobalCircularLoopTimes();
```

函数功能	获取圆运动圈数，在轨迹类型为 ARD_CIR 时有效
参数描述	
返回值	返回圆运动圈数 times = 0 时，表示圆弧运动； times > 0 时，表示圆运动，且表示圆的圈数

示例	`//设置圆运动圈数`
	`robotService.robotServiceSetGlobalCircularLoopTimes(3);`
	`//获取圆运动圈数`
	`int looptimes;`
	`looptimes = robotService.robotServiceGetGlobalCircularLoopTimes();`
	`std::cout << "loop times = " << looptimes << std::endl;`

输出	`loop times = 3`

37. 设置相对偏移属性 robotServiceSetMoveRelativeParam

```
int  robotServiceSetMoveRelativeParam(const aubo_robot_namespace::MoveRelative &relativeMoveOnBase);
```

函数功能	Base 坐标系下的相对偏移属性设置

参数描述	`typedef struct`
	`{`
	`bool  ena;  //是否使能偏移`

	float relativePosition[3]; //偏移量 x,y,z
	Ori relativeOri; //姿态偏移
	}MoveRelative;
	包括使能、位置偏移和姿态偏移

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：在 Base 坐标系下沿 Z 方向偏移 0.1m。
	
	double wp1[6] = {};
	wp1[0] = 60.443151*M_PI/180;
	wp1[1] = 42.275463*M_PI/180;
	wp1[2] = -97.679737*M_PI/180;
	wp1[3] = -49.990510*M_PI/180;
	wp1[4] = -90.007372*M_PI/180;
	wp1[5] = 62.567046*M_PI/180;
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);
	
	//设置 Base 坐标系下的相对偏移
	aubo_robot_namespace::MoveRelative relativeOnBase;
	relativeOnBase.ena = true;
	relativeOnBase.relativeOri.w = 1;
	relativeOnBase.relativeOri.x = 0;
	relativeOnBase.relativeOri.y = 0;
	relativeOnBase.relativeOri.z = 0;

	``
	relativeOnBase.relativePosition[0] = 0;
	relativeOnBase.relativePosition[1] = 0;
	relativeOnBase.relativePosition[2] = 0;
	
	//偏移运动测试
	for(int i = 0; i < 2; i++)
	{
	//wp1 基础上，Base Z 方向偏移 0
	relativeOnBase.relativePosition[2] = 0;
	robotService.robotServiceSetMoveRelativeParam(relativeOnBase);
	robotService.robotServiceJointMove(wp1,true);
	//wp1 基础上，Base Z 方向上偏移 0.1m
	relativeOnBase.relativePosition[2] = 0.1;
	robotService.robotServiceSetMoveRelativeParam(relativeOnBase);
	robotService.robotServiceJointMove(wp1,true);
	}
	``

输出	

38.设置相对偏移属性 robotServiceSetMoveRelativeParam

```
int  robotServiceSetMoveRelativeParam(const aubo_robot_namespace::MoveRelative   &relativeMoveOnUserCoord,
                                      const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord);
```

函数功能	User 坐标系或工具坐标系下的相对偏移属性设置

参数描述	relativeMoveOnUserCoord：包括使能、位置偏移和姿态偏移
	``
	typedef struct
	{
	bool ena; //是否使能偏移
	float relativePosition[3]; //偏移量 x,y,z
	Ori relativeOri; //姿态偏移
	}MoveRelative;
	``

	userCoord：用户坐标系或者工具坐标系
	``
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	``

```
    JointParam   wayPointArray[3];  //用于标定用户坐标系的3个点(关节角)该参数只在coordType==WorldCoordinate时有效
    ToolInEndDesc    toolDesc;           //工具描述  当coordType==EndCoordinate时，该参数表示工具参数
                                        //当coordType==WorldCoordinat时，该参数表示标定用户坐标系的3个点时，使用
                                          工具描述。
}CoordCalibrateByJointAngleAndTool;
```

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：在 Base 坐标系下沿 X+方向偏移 0.1m。

```
 double wp1[6] = {};
wp1[0] = 60.443151*M_PI/180;
wp1[1] = 42.275463*M_PI/180;
wp1[2] = -97.679737*M_PI/180;
wp1[3] = -49.990510*M_PI/180;
wp1[4] = -90.007372*M_PI/180;
wp1[5] = 62.567046*M_PI/180;

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
m_set_jointacc.jointPara[0] = 30*M_PI/180;
m_set_jointacc.jointPara[1] = 30*M_PI/180;
m_set_jointacc.jointPara[2] = 30*M_PI/180;
m_set_jointacc.jointPara[3] = 30*M_PI/180;
m_set_jointacc.jointPara[4] = 30*M_PI/180;
m_set_jointacc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
m_set_jointvelc.jointPara[0] = 15*M_PI/180;
m_set_jointvelc.jointPara[1] = 15*M_PI/180;
m_set_jointvelc.jointPara[2] = 15*M_PI/180;
m_set_jointvelc.jointPara[3] = 15*M_PI/180;
m_set_jointvelc.jointPara[4] = 15*M_PI/180;
m_set_jointvelc.jointPara[5] = 15*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

//设置Base坐标系下的相对偏移
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

aubo_robot_namespace::MoveRelative relativeOnBase;
relativeOnBase.ena = true;
```

```
    relativeOnBase.relativeOri.w = 1;
    relativeOnBase.relativeOri.x = 0;
    relativeOnBase.relativeOri.y = 0;
    relativeOnBase.relativeOri.z = 0;
    relativeOnBase.relativePosition[0] = 0;
    relativeOnBase.relativePosition[1] = 0;
    relativeOnBase.relativePosition[2] = 0;

    //偏移运动测试
    for(int i = 0; i < 2; i++)
    {
        //wp1 基础上，Base X 方向偏移 0
        relativeOnBase.relativePosition[0] = 0;
        robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
        robotService.robotServiceJointMove(wp1,true);
        //wp1 基础上，Base X 方向上偏移 0.1m
        relativeOnBase.relativePosition[0] = 0.1;
        robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
        robotService.robotServiceJointMove(wp1,true);
    }
```

输出

示例 2

示例讲解：工具坐标系（{0,0,0.45},{1,0,0,0}）下沿 X+方向相对偏移 0.1m。

```
     double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
```

```
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置相对偏移属性
    aubo_robot_namespace::MoveRelative relativeOnBase;
    relativeOnBase.ena = true;
    relativeOnBase.relativeOri.w = 1;
    relativeOnBase.relativeOri.x = 0;
    relativeOnBase.relativeOri.y = 0;
    relativeOnBase.relativeOri.z = 0;
    relativeOnBase.relativePosition[0] = 0;
    relativeOnBase.relativePosition[1] = 0;
    relativeOnBase.relativePosition[2] = 0;

    //设置 End 坐标系相对偏移
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::EndCoordinate;
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    //偏移运动测试
    for(int i = 0; i < 2; i++)
    {
        //wp1 基础上，Base X 方向偏移 0
        relativeOnBase.relativePosition[0] = 0;
        robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
        robotService.robotServiceJointMove(wp1,true);
        //wp1 基础上，Base X 方向上偏移 0.1m
        relativeOnBase.relativePosition[0] = 0.1;
```

```
        robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
        robotService.robotServiceJointMove(wp1,true);
    }
```

输出

示例 3

示例讲解：User 坐标系（p1）下沿 X+方向偏移 0.1m。

```
     double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置相对偏移属性
    aubo_robot_namespace::MoveRelative relativeOnBase;
    relativeOnBase.ena = true;
    relativeOnBase.relativeOri.w = 1;
    relativeOnBase.relativeOri.x = 0;
    relativeOnBase.relativeOri.y = 0;
    relativeOnBase.relativeOri.z = 0;
    relativeOnBase.relativePosition[0] = 0;
```

```
    relativeOnBase.relativePosition[1] = 0;
    relativeOnBase.relativePosition[2] = 0;

    /**** 设置 User 坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    //偏移运动测试
    for(int i = 0; i < 2; i++)
    {
        //wp1 基础上，Base X 方向偏移 0
        relativeOnBase.relativePosition[0] = 0;
```

	robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
	robotService.robotServiceJointMove(wp1,true);
	//wp1 基础上，Base X 方向上偏移 0.1m
	relativeOnBase.relativePosition[0] = 0.1;
	robotService.robotServiceSetMoveRelativeParam(relativeOnBase,userCoord);
	robotService.robotServiceJointMove(wp1,true);
	}

输出	

39. 设置无提前到位 robotServiceSetNoArrivalAhead

```
int  robotServiceSetNoArrivalAhead();
```

函数功能	设置无提前到位
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

```
double wp1[6] = {};
wp1[0] = 60.443151*M_PI/180;
wp1[1] = 42.275463*M_PI/180;
wp1[2] = -97.679737*M_PI/180;
wp1[3] = -49.990510*M_PI/180;
wp1[4] = -90.007372*M_PI/180;
wp1[5] = 62.567046*M_PI/180;

double wp2[6] = {};
wp2[0] = 83.411541*M_PI/180;
wp2[1] = 39.625360*M_PI/180;
wp2[2] = -103.796807*M_PI/180;
wp2[3] = -53.491856*M_PI/180;
wp2[4] = -90.021641*M_PI/180;
wp2[5] = 85.530279*M_PI/180;

double wp3[6] = {};
wp3[0] = 81.206455*M_PI/180;
wp3[1] = 28.381980*M_PI/180;
wp3[2] = -129.233955*M_PI/180;
wp3[3] = -67.700289*M_PI/180;
wp3[4] = -90.019516*M_PI/180;
wp3[5] = 83.325883*M_PI/180;

//关节运动
robotService.robotServiceJointMove(wp1,true);

/*设置无提前到位*/
robotService.robotServiceSetNoArrivalAhead();
/*运动测试*/
for(int i = 0; i < 2; i++)
{
    robotService.robotServiceJointMove(wp2,true);
    robotService.robotServiceLineMove(wp3,true);
    robotService.robotServiceJointMove(wp1,true);
}
```

输出

40.设置提前到位距离模式 robotServiceSetArrivalAheadDistanceMode

```
int  robotServiceSetArrivalAheadDistanceMode(double distance /*米*/);
```

函数功能	设置提前到位距离模式
参数描述	distance：距离。单位 m
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
```

	``
	    wp2[2] = -103.796807*M_PI/180;
	    wp2[3] = -53.491856*M_PI/180;
	    wp2[4] = -90.021641*M_PI/180;
	    wp2[5] = 85.530279*M_PI/180;
	
	    double wp3[6] = {};
	    wp3[0] = 81.206455*M_PI/180;
	    wp3[1] = 28.381980*M_PI/180;
	    wp3[2] = -129.233955*M_PI/180;
	    wp3[3] = -67.700289*M_PI/180;
	    wp3[4] = -90.019516*M_PI/180;
	    wp3[5] = 83.325883*M_PI/180;
	
	    //关节运动
	    robotService.robotServiceJointMove(wp1,true);
	
	    /*设置提前到位距离模式*/
	    robotService.robotServiceSetArrivalAheadDistanceMode(0.05);
	    /*运动测试*/
	    for(int i = 0; i < 2; i++)
	    {
	        robotService.robotServiceJointMove(wp2,true);
	        robotService.robotServiceJointMove(wp3,true);
	        robotService.robotServiceJointMove(wp1,true);
	    }
	``

输出	

41.设置提前到位时间模式 robotServiceSetArrivalAheadTimeMode

```int robotServiceSetArrivalAheadTimeMode(double second /*秒*/);```	
函数功能	设置提前到位时间模式
参数描述	second：时间。单位 s
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```
	    //初始化运动属性
	    robotService.robotServiceInitGlobalMoveProfile();
	
	    //设置关节运动最大加速度
	``

```
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
    wp2[2] = -103.796807*M_PI/180;
    wp2[3] = -53.491856*M_PI/180;
    wp2[4] = -90.021641*M_PI/180;
    wp2[5] = 85.530279*M_PI/180;

    double wp3[6] = {};
    wp3[0] = 81.206455*M_PI/180;
    wp3[1] = 28.381980*M_PI/180;
    wp3[2] = -129.233955*M_PI/180;
    wp3[3] = -67.700289*M_PI/180;
    wp3[4] = -90.019516*M_PI/180;
    wp3[5] = 83.325883*M_PI/180;

    //关节运动
```

	``
	robotService.robotServiceJointMove(wp1,true);
	
	/*设置提前到位距离模式*/
	robotService.robotServiceSetArrivalAheadTimeMode(0.5);
	/*运动测试*/
	for(int i = 0; i < 2; i++)
	{
	robotService.robotServiceJointMove(wp2,true);
	robotService.robotServiceJointMove(wp3,true);
	robotService.robotServiceJointMove(wp1,true);
	}
	``

输出	

42. 设置提前到位交融半径模式 robotServiceSetArrivalAheadBlendDistanceMode

```
int  robotServiceSetArrivalAheadBlendDistanceMode(double distance /*米*/);
```

函数功能	设置提前到位交融半径模式
参数描述	distance：交融半径。单位 m
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	``

```
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
    wp2[2] = -103.796807*M_PI/180;
    wp2[3] = -53.491856*M_PI/180;
    wp2[4] = -90.021641*M_PI/180;
    wp2[5] = 85.530279*M_PI/180;

    double wp3[6] = {};
    wp3[0] = 81.206455*M_PI/180;
    wp3[1] = 28.381980*M_PI/180;
    wp3[2] = -129.233955*M_PI/180;
    wp3[3] = -67.700289*M_PI/180;
    wp3[4] = -90.019516*M_PI/180;
    wp3[5] = 83.325883*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    /*设置提前到位距离模式*/
    robotService.robotServiceSetArrivalAheadBlendDistanceMode(0.05);
    /*运动测试*/
    for(int i = 0; i < 2; i++)
    {
        robotService.robotServiceJointMove(wp2,true);
        robotService.robotServiceLineMove(wp3,true);
        robotService.robotServiceJointMove(wp1,true);
    }
```

输出

43.设置示教坐标系 robotServiceSetTeachCoordinateSystem

```
int robotServiceSetTeachCoordinateSystem(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &coordSystem);
```

函数功能	设置示教运动的坐标系

参数描述	**坐标系描述**
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效
	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    /**** 设置 User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
```

	``
	    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;
	
	    aubo_robot_namespace::ToolInEndDesc tool1;
	    tool1.toolInEndPosition.x = 0;
	    tool1.toolInEndPosition.y = 0;
	    tool1.toolInEndPosition.z = 0.45;
	    tool1.toolInEndOrientation.w = 1;
	    tool1.toolInEndOrientation.x = 0;
	    tool1.toolInEndOrientation.y = 0;
	    tool1.toolInEndOrientation.z = 0;
	
	    userCoord.toolDesc = tool1;
	
	    //设置示教坐标系
	    robotService.robotServiceSetTeachCoordinateSystem(userCoord);
	``

输出	

44.关节运动 robotServiceJointMove

```int robotServiceJointMove(aubo_robot_namespace::wayPoint_S &wayPoint, bool IsBolck);```	
函数功能	关节（轴动）运动至目标点位

参数描述	路点信息 wayPoint
	`` typedef struct
	{
	    cartesianPos_U cartPos; //机械臂的位置信息 X,Y,Z
	    Ori orientation; //机械臂姿态信息 通过四元素表示,可以通过工具函数实现与欧拉角互转
	    double jointpos[ARM_DOF]; //机械臂关节角信息
	}wayPoint_S;```
	
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	``` //初始化运动属性
	    robotService.robotServiceInitGlobalMoveProfile();
	
	    //设置关节运动最大加速度```

```
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //定义路点 wp1
    aubo_robot_namespace::wayPoint_S wp1;
    wp1.jointpos[0] = 60.443151*M_PI/180;
    wp1.jointpos[1] = 42.275463*M_PI/180;
    wp1.jointpos[2] = -97.679737*M_PI/180;
    wp1.jointpos[3] = -49.990510*M_PI/180;
    wp1.jointpos[4] = -90.007372*M_PI/180;
    wp1.jointpos[5] = 62.567046*M_PI/180;
    wp1.cartPos.position.x = -0.199819;
    wp1.cartPos.position.y = -0.598529;
    wp1.cartPos.position.z = -0.431393;
    aubo_robot_namespace::Rpy rpy_tmp;
    rpy_tmp.rx = 179.977173*M_PI/180;
    rpy_tmp.ry = -0.027977*M_PI/180;
    rpy_tmp.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(rpy_tmp,wp1.orientation);

    //关节运动
    robotService.robotServiceJointMove(wp1,true);
```

输出	运动到位后：



45.关节运动 robotServiceJointMove

```
int robotServiceJointMove(double jointAngle[aubo_robot_namespace::ARM_DOF], bool IsBolck);
```

函数功能	关节（轴动）运动至目标点位
参数描述	jointAngle：以关节角表示路点。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
```

	``
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);
	
	//定义路点 wp1
	double wp1[6] = {};
	wp1[0] = 60.443151*M_PI/180;
	wp1[1] = 42.275463*M_PI/180;
	wp1[2] = -97.679737*M_PI/180;
	wp1[3] = -49.990510*M_PI/180;
	wp1[4] = -90.007372*M_PI/180;
	wp1[5] = 62.567046*M_PI/180;
	
	//关节运动
	robotService.robotServiceJointMove(wp1,true);
	``

输出	运动到位后： 

46. 跟随模式轴动 robotServiceFollowModeJointMove

```int robotServiceFollowModeJointMove(double jointAngle[aubo_robot_namespace::ARM_DOF]);```	
函数功能	基于跟随模式的轴动接口
参数描述	`jointAngle`：以关节角表示路点。
返回值	调用成功返回 `ErrnoSucc`;错误返回错误号

示例	```
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	``

```
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;


    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    std::cout << "move Joint succ" << std::endl;

    /*基于跟随模式的关节运动*/
    for(int i = 0; i < 1000; i++)
    {
        wp1[0] = wp1[0] + 0.0001;
        std::cout << wp1[0] << std::endl;

        robotService.robotServiceFollowModeJointMove(wp1);
        usleep(1000*5);
    }
```

输出

47.直线运动 robotServiceLineMove

```
int robotServiceLineMove(aubo_robot_namespace::wayPoint_S   &wayPoint,      bool IsBolck);
```

函数功能	直线运动至目标点位

参数描述	路点信息 wayPoint
	`typedef struct`
	`{`
	`  cartesianPos_U cartPos;  //机械臂的位置信息 X,Y,Z`
	`  Ori orientation;  //机械臂姿态信息  通过四元素表示,可以通过工具函数实现与欧拉角互转`
	`  double jointpos[ARM_DOF];  //机械臂关节角信息`
	`}wayPoint_S;`
	
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//初始化运动属性`
	`robotService.robotServiceInitGlobalMoveProfile();`
	
	`//设置末端型运动最大加速度`
	`double m_set_end_lineacc = 0.2;`
	`robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);`
	`//设置末端型运动最大速度`
	`double m_set_end_linevelc = 0.2;`
	`robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);`
	
	`//定义路点 wp1`
	`aubo_robot_namespace::wayPoint_S wp1;`
	`wp1.jointpos[0] = 60.443151*M_PI/180;`
	`wp1.jointpos[1] = 42.275463*M_PI/180;`
	`wp1.jointpos[2] = -97.679737*M_PI/180;`
	`wp1.jointpos[3] = -49.990510*M_PI/180;`
	`wp1.jointpos[4] = -90.007372*M_PI/180;`
	`wp1.jointpos[5] = 62.567046*M_PI/180;`
	`wp1.cartPos.position.x = -0.199819;`
	`wp1.cartPos.position.y = -0.598529;`
	`wp1.cartPos.position.z = -0.431393;`
	`aubo_robot_namespace::Rpy rpy_tmp;`

	``
	rpy_tmp.rx = 179.977173*M_PI/180;
	rpy_tmp.ry = -0.027977*M_PI/180;
	rpy_tmp.rz = -92.123917*M_PI/180;
	robotService.RPYToQuaternion(rpy_tmp,wp1.orientation);
	
	//定义路点 wp2
	double wp2[6] = {};
	wp2[0] = 83.411541*M_PI/180;
	wp2[1] = 39.625360*M_PI/180;
	wp2[2] = -103.796807*M_PI/180;
	wp2[3] = -53.491856*M_PI/180;
	wp2[4] = -90.021641*M_PI/180;
	wp2[5] = 85.530279*M_PI/180;
	
	//关节运动
	robotService.robotServiceJointMove(wp2,true);
	
	//直线运动
	robotService.robotServiceLineMove(wp1,true);
	``

输出	运动到位后： 

48. 直线运动 robotServiceLineMove

```int robotServiceLineMove(double jointAngle[aubo_robot_namespace::ARM_DOF], bool IsBolck);```	
函数功能	直线运动至目标点位
参数描述	jointAngle：以关节角表示路点。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。

	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    //定义路点 wp1
    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //定义路点 wp2
    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
    wp2[2] = -103.796807*M_PI/180;
    wp2[3] = -53.491856*M_PI/180;
    wp2[4] = -90.021641*M_PI/180;
    wp2[5] = 85.530279*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp2,true);

    //直线运动
    robotService.robotServiceLineMove(wp1,true);
```

输出	运动到位后：



49. 旋转运动 robotServiceRotateMove

```
int robotServiceRotateMove(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const double rotateAxisOnUserCoord[3], double rotateAngle,  bool IsBolck);
```

函数功能	保持当前位置变换姿态做旋转运动

参数描述	参数 1：userCoord 坐标系
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效
	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	参数 2：rotateAxisOnUserCoord。转轴[x,y,z] 当绕 X 转，[1,0,0]
	参数 3：rotateAngle。转角，单位 rad。
	参数 4：是否阻塞。
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：flange_center 绕 Base 坐标系 X 轴旋转 30°。
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();

```
    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    //设置 Base 坐标系
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    //rotate 旋转运动
    double rotateAxis[3] = {0,0,1};
    double rotateAngle1 = 30*M_PI/180;
    double rotateAngle2 = -30*M_PI/180;
    for( int i = 0; i < 2; i++)
    {
        robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle1,true);
        sleep(1);
        robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle2,true);
        sleep(1);
    }
```

输出

示例 2

示例讲解：flange_center 绕 User 坐标系（p1）的 Z 轴旋转 30°。

```
      //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    double wp1[6] = {};
```

```
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    /**** 设置 User 坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
```

```
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    //rotate 旋转运动
    double rotateAxis[3] = {0,0,1};
    double rotateAngle1 = 30*M_PI/180;
    double rotateAngle2 = -30*M_PI/180;
    for( int i = 0; i < 2; i++)
    {
        robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle1,true);
        sleep(1);
        robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle2,true);
        sleep(1);
    }
```

输出

示例 3

示例讲解：工具坐标系（tcp2）绕 User 坐标系（p1）的 Z 轴旋转 30°。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
```

```
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    /**** 设置User坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    /* 设置工具 */
    aubo_robot_namespace::ToolKinematicsParam toolkin;
    toolkin.toolInEndOrientation = {1,0,0,0};
    toolkin.toolInEndPosition = {0,0,0.45};
    robotService.robotServiceSetToolKinematicsParam(toolkin);
```

	``
	//rotate 旋转运动
	double rotateAxis[3] = {0,0,1};
	double rotateAngle1 = 30*M_PI/180;
	double rotateAngle2 = -30*M_PI/180;
	for(int i = 0; i < 2; i++)
	{
	robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle1,true);
	sleep(1);
	robotService.robotServiceRotateMove(userCoord,rotateAxis,rotateAngle2,true);
	sleep(1);
	}
	``

输出	
示例 4	示例讲解：绕工具坐标系（tcp2）的 X 轴旋转 10°。

50.轨迹运动 robotServiceTrackMove

`int robotServiceTrackMove(aubo_robot_namespace::move_track subMoveMode, bool IsBolck);`	
函数功能	轨迹运动
参数描述	/**** subMoveMode：轨迹类型 当 subMoveMode==ARC_CIR， CARTESIAN_MOVEP， CARTESIAN_CUBICSPLINE， CARTESIAN_UBSPLINEINTP 时，该运动属于末端型运动； 当 subMoveMode==JIONT_CUBICSPLINE，JOINT_UBSPLINEINTP 时，该运动属于关节型运动； 当 subMoveMode==ARC_CIR 表示圆或者圆弧 当圆的圈数属性（CircularLoopTimes）为 0 时，表示圆弧轨迹， 当圆的圈数属性（CircularLoopTimes）大于 0 时，表示圆轨迹。 当 subMoveMode==CARTESIAN_MOVEP 表示 MOVEP 轨迹，需要用户这只交融半径的属性。 当 subMoveMode==JOINT_UBSPLINEINTP 轨迹复现接口 ***/ IsBolck ：是否阻塞。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：圆运动。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
    wp2[2] = -103.796807*M_PI/180;
    wp2[3] = -53.491856*M_PI/180;
    wp2[4] = -90.021641*M_PI/180;
    wp2[5] = 85.530279*M_PI/180;

    double wp3[6] = {};
    wp3[0] = 81.206455*M_PI/180;
    wp3[1] = 28.381980*M_PI/180;
    wp3[2] = -129.233955*M_PI/180;
    wp3[3] = -67.700289*M_PI/180;
    wp3[4] = -90.019516*M_PI/180;
    wp3[5] = 83.325883*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    //清空全局路点容器
    robotService.robotServiceClearGlobalWayPointVector();
    //添加路点
    robotService.robotServiceAddGlobalWayPoint(wp1);
```

```
    //添加路点
    robotService.robotServiceAddGlobalWayPoint(wp2);
    //添加路点
    robotService.robotServiceAddGlobalWayPoint(wp3);
    //设置圆运动圈数
    robotService.robotServiceSetGlobalCircularLoopTimes(3);

    //圆弧运动
    robotService.robotServiceTrackMove(aubo_robot_namespace::ARC_CIR,true);
```

输出

示例 2

示例讲解：圆弧运动。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    double wp2[6] = {};
    wp2[0] = 83.411541*M_PI/180;
    wp2[1] = 39.625360*M_PI/180;
    wp2[2] = -103.796807*M_PI/180;
    wp2[3] = -53.491856*M_PI/180;
    wp2[4] = -90.021641*M_PI/180;
    wp2[5] = 85.530279*M_PI/180;

    double wp3[6] = {};
    wp3[0] = 81.206455*M_PI/180;
    wp3[1] = 28.381980*M_PI/180;
    wp3[2] = -129.233955*M_PI/180;
```

	wp3[3] = -67.700289*M_PI/180;
	wp3[4] = -90.019516*M_PI/180;
	wp3[5] = 83.325883*M_PI/180;
	
	//关节运动
	robotService.robotServiceJointMove(wp1,true);
	
	//清空全局路点容器
	robotService.robotServiceClearGlobalWayPointVector();
	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp1);
	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp2);
	//添加路点
	robotService.robotServiceAddGlobalWayPoint(wp3);
	
	//设置交融半径
	float blendradius = 0.05;
	std::cout << "set blendradius ret is " << robotService.robotServiceSetGlobalBlendRadius(blendradius) << std::endl;
	//MOVEP 运动
	robotService.robotServiceTrackMove(aubo_robot_namespace::CARTESIAN_MOVEP,true);

输出	

51.保持当前姿态通过相对位移直线运动至目标位置 robotMoveLineToTargetPositionByRelative

```
int robotMoveLineToTargetPositionByRelative(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord,
                    const aubo_robot_namespace::MoveRelative    &relativeMoveOnUserCoord,      //目标位置相对当前位置的偏移
                    bool IsBolck);          //是否阻塞
```

函数功能	保持当前位姿通过直线运动的方式运动到目标位置,其中目标位置是通过相对当前位置的偏移给出

参数描述	参数 1：参考坐标系
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效

	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	参数 2：参考坐标系下的偏移
	typedef struct
	{
	bool ena; //是否使能偏移
	float relativePosition[3]; //偏移量 x,y,z
	Ori relativeOri; //姿态偏移
	}MoveRelative;
	参数 3：IsBolck 是否阻塞。
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：Base 坐标系下 X 方向偏移 0.1m。
	
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);
	
	//设置末端型运动最大加速度

```
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    //设置参考坐标系,Base
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
    moverelative.relativeOri.w = 1;
    moverelative.relativeOri.x = 0;
    moverelative.relativeOri.y = 0;
    moverelative.relativeOri.z = 0;
    moverelative.relativePosition[0] = 0.1;
    moverelative.relativePosition[1] = 0;
    moverelative.relativePosition[2] = 0;

    /*直线运动至目标位置*/
    robotService.robotMoveLineToTargetPositionByRelative(userCoord,moverelative,true);
```

输出

示例 2

示例讲解：User 坐标系（p1）下 Z 方向偏移 0.05m。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
```

```
    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    /**** 设置User坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
```

```
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;


    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
    moverelative.relativeOri.w = 1;
    moverelative.relativeOri.x = 0;
    moverelative.relativeOri.y = 0;
    moverelative.relativeOri.z = 0;
    moverelative.relativePosition[0] = 0;
```

```
    moverelative.relativePosition[1] = 0;
    moverelative.relativePosition[2] = 0.05;

    /*直线运动至目标位置*/
    robotService.robotMoveLineToTargetPositionByRelative(userCoord,moverelative,true);
```

输出

示例 3

示例讲解：tool 坐标系（tcp1）下 Z 方向偏移 0.1m。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
```

```
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    /**** 设置 Tool 坐标系(tcp1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::EndCoordinate;
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
    moverelative.relativeOri.w = 1;
    moverelative.relativeOri.x = 0;
    moverelative.relativeOri.y = 0;
    moverelative.relativeOri.z = 0;
    moverelative.relativePosition[0] = 0;
    moverelative.relativePosition[1] = 0;
    moverelative.relativePosition[2] = 0.1;

    /*直线运动至目标位置*/
    robotService.robotMoveLineToTargetPositionByRelative(userCoord,moverelative,true);
```

输出

52.保持当前姿态通过相对位移关节运动至目标位置 robotMoveJointToTargetPositionByRelative

```
int robotMoveJointToTargetPositionByRelative(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord,
                const aubo_robot_namespace::MoveRelative    &relativeMoveOnUserCoord,      //目标位置相对当前位置的偏移
                bool IsBolck = false);     //是否阻塞
```

函数功能	保持当前位姿通过关节运动的方式运动到目标位置,其中目标位置是通过相对当前位置的偏移给出

参数描述	参数 1：参考坐标系
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效
	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	参数 2：参考坐标系下的偏移
	typedef struct
	{
	bool ena; //是否使能偏移
	float relativePosition[3]; //偏移量 x,y,z
	Ori relativeOri; //姿态偏移
	}MoveRelative;
	参数 3：IsBolck 是否阻塞。
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：Base 坐标系下 X 方向偏移 0.1m。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
```

```
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    //设置参考坐标系,Base
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
```

```
    moverelative.relativeOri.w = 1;
    moverelative.relativeOri.x = 0;
    moverelative.relativeOri.y = 0;
    moverelative.relativeOri.z = 0;
    moverelative.relativePosition[0] = 0.1;
    moverelative.relativePosition[1] = 0;
    moverelative.relativePosition[2] = 0;

    /*关节运动至目标位置*/
    robotService.robotMoveJointToTargetPositionByRelative(userCoord,moverelative,true);
```

输出

示例 2

示例讲解：User 坐标系（p1）下 Z 方向偏移 0.05m。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);
```

```
    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    /**** 设置 User 坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
```

```
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;


    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
    moverelative.relativeOri.w = 1;
    moverelative.relativeOri.x = 0;
    moverelative.relativeOri.y = 0;
    moverelative.relativeOri.z = 0;
    moverelative.relativePosition[0] = 0;
    moverelative.relativePosition[1] = 0;
    moverelative.relativePosition[2] = 0.05;

    /*关节运动至目标位置*/
    robotService.robotMoveJointToTargetPositionByRelative(userCoord,moverelative,true);
```

输出

示例 3

示例讲解：tool 坐标系（tcp1）下 Z 方向偏移 0.1m。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
```

```
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    sleep(2);

    /**** 设置 Tool 坐标系(tcp1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::EndCoordinate;
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    /*设置偏移量*/
    aubo_robot_namespace::MoveRelative moverelative;
    moverelative.ena = true;
    moverelative.relativeOri.w = 1;
```

	moverelative.relativeOri.x = 0;
	moverelative.relativeOri.y = 0;
	moverelative.relativeOri.z = 0;
	moverelative.relativePosition[0] = 0;
	moverelative.relativePosition[1] = 0;
	moverelative.relativePosition[2] = 0.1;
	
	/*关节运动至目标位置*/
	robotService.robotMoveJointToTargetPositionByRelative (userCoord,moverelative,true);

输出	

53. 直线方式运动至给定位置 robotMoveLineToTargetPosition

```
int robotMoveLineToTargetPosition(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool  &userCoord,
                                  const aubo_robot_namespace::Pos             &toolEndPositionOnUserCoord,
                                  const aubo_robot_namespace::ToolInEndDesc   &toolInEndDesc,
                                  bool IsBolck = false);
```

函数功能	保持当前姿态通过直线运动的方式运动到目标位置

参数描述	参数 1：参考坐标系
	typedef struct
	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效
	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	参数 2：工具坐标系在参考坐标系下的位置参数
	struct Pos
	{
	double x;
	double y;
	double z;
	};
	参数 3：工具坐标系
	typedef struct

	{
	Pos toolInEndPosition; //工具相对末端坐标系的位置
	Ori toolInEndOrientation; //工具相对末端坐标系的姿态
	}ToolInEndDesc;
	
	参数 4：IsBolck 是否阻塞。
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：flange_center 在 Base 下的某个位置。
	
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);
	
	//设置末端型运动最大加速度
	double m_set_end_lineacc = 0.2;
	robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
	//设置末端型运动最大速度
	double m_set_end_linevelc = 0.2;
	robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

```
    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    //设置参考坐标系
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    //设置工具坐标系
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool1;

    /*设置目标位置*/
    aubo_robot_namespace::Pos posOnBase;
    posOnBase.x = -0.2;
    posOnBase.y = -0.6;
    posOnBase.z = 0;

    /*直线运动至目标位置*/
    robotService.robotMoveLineToTargetPosition(userCoord,posOnBase,tool1,true);
```

输出



示例 2

示例讲解：运动至工具坐标系（{0,0,0.45},{1,0,0,0}）在参考坐标系（p1）下的位置。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
    m_set_jointacc.jointPara[0] = 30*M_PI/180;
    m_set_jointacc.jointPara[1] = 30*M_PI/180;
    m_set_jointacc.jointPara[2] = 30*M_PI/180;
    m_set_jointacc.jointPara[3] = 30*M_PI/180;
    m_set_jointacc.jointPara[4] = 30*M_PI/180;
    m_set_jointacc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
    m_set_jointvelc.jointPara[0] = 15*M_PI/180;
    m_set_jointvelc.jointPara[1] = 15*M_PI/180;
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    double wp1[6] = {};
```

```
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    /**** 设置User 坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    /*工具参数*/
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
```

```
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;


    /*设置目标位置*/
    aubo_robot_namespace::Pos posOnBase;
    posOnBase.x = 0;
    posOnBase.y = 0.2;
    posOnBase.z = -0.6;

    /*直线运动至目标位置*/
    robotService.robotMoveLineToTargetPosition(userCoord,posOnBase,tool1,true);
```

输出



54. 轴动方式运动至给定位置 robotMoveJointToTargetPosition

```
int robotMoveJointToTargetPosition(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool  &userCoord,
                                   const aubo_robot_namespace::Pos             &toolEndPositionOnUserCoord,
                                   const aubo_robot_namespace::ToolInEndDesc   &toolInEndDesc,
                                   bool IsBolck = false);
```

函数功能	保持当前姿态通过轴动运动的方式运动到目标位置

参数描述	参数 1：参考坐标系
	`typedef struct`
	`{`
	`coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系`
	`CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效`
	`JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效`

	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	参数 2：工具坐标系在参考坐标系下的位置参数
	struct Pos
	{
	double x;
	double y;
	double z;
	};
	参数 3：工具坐标系
	typedef struct
	{
	Pos toolInEndPosition; //工具相对末端坐标系的位置
	Ori toolInEndOrientation; //工具相对末端坐标系的姿态
	}ToolInEndDesc;
	
	参数 4：IsBolck 是否阻塞。
	IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。
	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	示例讲解：轴动运动至 flange_center 在 Base 下的位置。
	
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;

```
    m_set_jointvelc.jointPara[2] = 15*M_PI/180;
    m_set_jointvelc.jointPara[3] = 15*M_PI/180;
    m_set_jointvelc.jointPara[4] = 15*M_PI/180;
    m_set_jointvelc.jointPara[5] = 15*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);

    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;

    //关节运动
    robotService.robotServiceJointMove(wp1,true);

    /*工具参数*/
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;


    /*设置目标位置*/
    aubo_robot_namespace::Pos posOnBase;
    posOnBase.x = -0.2;
    posOnBase.y = -0.6;
    posOnBase.z = 0;

    /*直线运动至目标位置*/
    robotService.robotMoveJointToTargetPosition(userCoord,posOnBase,tool1,true);
```

输出	

55.示教运动开始 robotServiceTeachStart

```
int robotServiceTeachStart(aubo_robot_namespace::teach_mode mode, bool direction);
```

函数功能	开始示教

参数描述	参数 1：mode 示教模式。
	示教关节:JOINT1, JOINT2, JOINT3, JOINT4, JOINT5, JOINT6,
	位置示教:MOV_X, MOV_Y, MOV_Z
	姿态示教:ROT_X, ROT_Y, ROT_Z;
	`enum teach_mode`
	`{`
	`NO_TEACH = 0,`
	`JOINT1,`
	`JOINT2,`
	`JOINT3,`
	`JOINT4,`
	`JOINT5,`
	`JOINT6,`
	`MOV_X,`
	`MOV_Y,`
	`MOV_Z,`
	`ROT_X,`
	`ROT_Y,`
	`ROT_Z`
	`};`
	参数 2：direction 方向，方向正方向 true 反方向 false

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：关节示教。

	``
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置关节运动最大加速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointacc;
	m_set_jointacc.jointPara[0] = 30*M_PI/180;
	m_set_jointacc.jointPara[1] = 30*M_PI/180;
	m_set_jointacc.jointPara[2] = 30*M_PI/180;
	m_set_jointacc.jointPara[3] = 30*M_PI/180;
	m_set_jointacc.jointPara[4] = 30*M_PI/180;
	m_set_jointacc.jointPara[5] = 30*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxAcc(m_set_jointacc);
	//设置关节运动最大速度
	aubo_robot_namespace::JointVelcAccParam m_set_jointvelc;
	m_set_jointvelc.jointPara[0] = 15*M_PI/180;
	m_set_jointvelc.jointPara[1] = 15*M_PI/180;
	m_set_jointvelc.jointPara[2] = 15*M_PI/180;
	m_set_jointvelc.jointPara[3] = 15*M_PI/180;
	m_set_jointvelc.jointPara[4] = 15*M_PI/180;
	m_set_jointvelc.jointPara[5] = 15*M_PI/180;
	robotService.robotServiceSetGlobalMoveJointMaxVelc(m_set_jointvelc);
	
	//设置试教模式，JOINT1
	aubo_robot_namespace::teach_mode teachmode;
	teachmode = aubo_robot_namespace::JOINT1;
	bool teachdir = true;
	//开始示教
	robotService.robotServiceTeachStart(teachmode,teachdir);
	sleep(2);
	//结束示教
	robotService.robotServiceTeachStop();
	``

输出	
示例 2	示例讲解：Base 下位置示教。

	``
	//初始化运动属性
	robotService.robotServiceInitGlobalMoveProfile();
	
	//设置末端型运动最大加速度
	``

```
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    //设置试教模式，Base_X
    aubo_robot_namespace::teach_mode teachmode;
    teachmode = aubo_robot_namespace::MOV_X;

    for(int i = 0; i < 2; i++)
    {
        //开始示教
        robotService.robotServiceTeachStart(teachmode,true);
        sleep(1);
        //结束示教
        robotService.robotServiceTeachStop();
        //开始示教
        robotService.robotServiceTeachStart(teachmode,false);
        sleep(1);
        //结束示教
        robotService.robotServiceTeachStop();
    }
```

输出

示例 3

示例讲解：用户坐标系（p1）下位置示教。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    /**** 设置User 坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
```

```
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;

    //设置示教坐标系
    robotService.robotServiceSetTeachCoordinateSystem(userCoord);

    //设置试教模式，User_Y
    aubo_robot_namespace::teach_mode teachmode;
    teachmode = aubo_robot_namespace::MOV_Y;

    for(int i = 0; i < 2; i++)
    {
        //开始示教
        robotService.robotServiceTeachStart(teachmode,true);
```

```
        sleep(1);
        //结束示教
        robotService.robotServiceTeachStop();
        //开始示教
        robotService.robotServiceTeachStart(teachmode,false);
        sleep(1);
        //结束示教
        robotService.robotServiceTeachStop();
    }
```

输出

示例 4

示例讲解：工具坐标系（{0,0,0.45}，{1,0,0,0}）在 Base 坐标系下姿态示教。

```
    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置末端型运动最大加速度
    double m_set_end_lineacc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineAcc(m_set_end_lineacc);
    //设置末端型运动最大速度
    double m_set_end_linevelc = 0.2;
    robotService.robotServiceSetGlobalMoveEndMaxLineVelc(m_set_end_linevelc);

    /**** 设置User坐标系(p1)下的相对偏移 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;
    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
```

```
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    userCoord.toolDesc = tool1;

    //设置示教坐标系(p1)
    robotService.robotServiceSetTeachCoordinateSystem(userCoord);

    //设置工具坐标系(tcp1)
    aubo_robot_namespace::ToolKinematicsParam toolkin;
    toolkin.toolInEndPosition.x = 0;
    toolkin.toolInEndPosition.y = 0;
    toolkin.toolInEndPosition.z = 0.45;
    toolkin.toolInEndOrientation.w = 1;
    toolkin.toolInEndOrientation.x = 0;
    toolkin.toolInEndOrientation.y = 0;
    toolkin.toolInEndOrientation.z = 0;
    robotService.robotServiceSetToolKinematicsParam(toolkin);

    //设置试教模式，User_Tool_RZ
    aubo_robot_namespace::teach_mode teachmode;
    teachmode = aubo_robot_namespace::ROT_Z;

    for(int i = 0; i < 2; i++)
    {
        //开始示教
        std::cout << robotService.robotServiceTeachStart(teachmode,true) << std::endl;
        sleep(1);
        //结束示教
        robotService.robotServiceTeachStop();
```

	//开始示教
	robotService.robotServiceTeachStart(teachmode,false);
	sleep(1);
	//结束示教
	robotService.robotServiceTeachStop();
	}

输出	

56.示教停止 robotServiceTeachStop

int robotServiceTeachStop();	
函数功能	示教停止
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	//结束示教
	robotService.robotServiceTeachStop();

输出	

57.运动的启动、停止、暂停和继续控制 rootServiceRobotMoveControl

int rootServiceRobotMoveControl(aubo_robot_namespace::RobotMoveControlCommand cmd);	
函数功能	机械臂运动控制 停止,暂停,继续
参数描述	cmd: 控制命令。 typedef enum { RobotMoveStop = 0, RobotMovePause = 1, RobotMoveContinue = 2, }RobotMoveControlCommand;

	注意：RobotMoveControl 需要在与 move 不同的线程中调用。且 robotMoveStop 调用后需要停止 move 线程，否则会接着执行下一个 move 指令。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：机械臂暂停操作。

	`/*机械臂暂停*/`
	`ret = robotService.rootServiceRobotMoveControl(aubo_robot_namespace::RobotMovePause);`
	`std::cout << "move control ret is : " << ret << std::endl;`

输出	
示例 2	示例讲解：机械臂继续操作。

	`/*机械臂继续运动*/`
	`ret = robotService.rootServiceRobotMoveControl(aubo_robot_namespace::RobotMoveContinue);`
	`std::cout << "move control ret is : " << ret << std::endl;`

输出	
示例 3	示例讲解：机械臂停止操作。

	`/*机械臂停止*/`
	`ret = robotService.rootServiceRobotMoveControl(aubo_robot_namespace::RobotMoveStop);`
	`std::cout << "move control ret is : " << ret << std::endl;`

输出	

58.快速停止 robotMoveFastStop

`int robotMoveFastStop();`	
函数功能	机械臂快速停止
参数描述	注意：robotMoveFastStop 需要在与 move 不同的线程中调用。且 robotMoveFastStop 调用后需要停止 move 线程，否则会接着执行下一个 move 指令。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：机械臂快速停止。
	`/*机械臂快速停止*/` `ret = robotService.robotMoveFastStop();`

	`std::cout << "move control ret is : " << ret << std::endl;`
输出	

离线轨迹运动

59.添加离线轨迹路点 robotServiceOfflineTrackWaypointAppend

60.清空离线轨迹路点 robotServiceOfflineTrackWaypointClear

61.启动离线轨迹运行 robotServiceOfflineTrackMoveStartup

62.停止离线轨迹运行 robotServiceOfflineTrackMoveStop

TCP 转 CAN 透传

63.进入 tcp 转 can 透传模式 robotServiceEnterTcp2CanbusMode

`int  robotServiceEnterTcp2CanbusMode();`	
函数功能	进入 can 透传模式
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	`robotService.robotServiceEnterTcp2CanbusMode();`
输出	

64. 退出 tcp 转 can 透传模式 robotServiceLeaveTcp2CanbusMode

int robotServiceLeaveTcp2CanbusMode();	
函数功能	退出 can 透传模式
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	robotService.robotServiceEnterTcp2CanbusMode();
	
	robotService.robotServiceLeaveTcp2CanbusMode();

输出	

65. 发送坐标数据到关节 can 总线 robotServiceSetRobotPosData2Canbus

int robotServiceSetRobotPosData2Canbus(double jointAngle[aubo_robot_namespace::ARM_DOF]);	
函数功能	通过透传将关节角度信息发送至驱动器
参数描述	jointAngle：六个关节角，单位 rad
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`double wp1[6] = {};`
	`wp1[0] = 60.443151*M_PI/180;`
	`wp1[1] = 42.275463*M_PI/180;`
	`wp1[2] = -97.679737*M_PI/180;`
	`wp1[3] = -49.990510*M_PI/180;`
	`wp1[4] = -90.007372*M_PI/180;`
	`wp1[5] = 62.567046*M_PI/180;`
	
	`//关节运动`
	`robotService.robotServiceJointMove(wp1,true);`
	
	`/*CAN 透传*/`
	`robotService.robotServiceEnterTcp2CanbusMode();`

	``
	for(int i = 0; i < 1000; i++)
	{
	wp1[0] = wp1[0] + 0.0001;
	std::cout << wp1[0] << std::endl;
	//将关节角透传到关节驱动器
	robotService.robotServiceSetRobotPosData2Canbus(wp1);
	usleep(1000*5);
	}
	
	robotService.robotServiceLeaveTcp2CanbusMode();
	``

输出	

工具接口

66. 正解 robotServiceRobotFk

```int robotServiceRobotFk(const double *jointAngle, int size, aubo_robot_namespace::wayPoint_S &wayPoint);```	
函数功能	此函数为正解函数，已知关节角求对应位置的位置和姿态
参数描述	jointAngle:六个关节的关节角，输入参数，单位 rad。 size：关节角数组长度，输入参数 wayPoint：输出参数。
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```
	/*求正解FK*/
	double wp1[6] = {};
	wp1[0] = 60.443151*M_PI/180;
	wp1[1] = 42.275463*M_PI/180;
	wp1[2] = -97.679737*M_PI/180;
	wp1[3] = -49.990510*M_PI/180;
	wp1[4] = -90.007372*M_PI/180;
	wp1[5] = 62.567046*M_PI/180;
	
	aubo_robot_namespace::wayPoint_S waypointFK;
	
	robotService.robotServiceRobotFk(wp1,6,waypointFK);
	
	std::cout<<std::endl<<"-------------路点信息---------------"<<std::endl;
	``

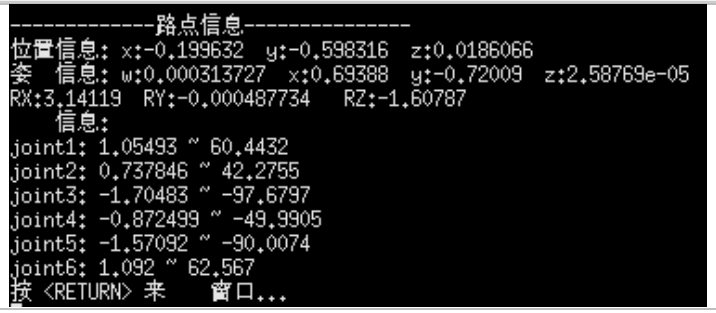
```
    //位置信息
    std::cout<<"位置信息: ";
    std::cout<<"x:"<<waypointFK.cartPos.position.x<<"  ";
    std::cout<<"y:"<<waypointFK.cartPos.position.y<<"  ";
    std::cout<<"z:"<<waypointFK.cartPos.position.z<<std::endl;

    //姿态信息
    std::cout<<"姿态信息: ";
    std::cout<<"w:"<<waypointFK.orientation.w<<"  ";
    std::cout<<"x:"<<waypointFK.orientation.x<<"  ";
    std::cout<<"y:"<<waypointFK.orientation.y<<"  ";
    std::cout<<"z:"<<waypointFK.orientation.z<<std::endl;

    aubo_robot_namespace::Rpy tempRpy;
    robotService.quaternionToRPY(waypointFK.orientation,tempRpy);
    std::cout<<"RX:"<<tempRpy.rx<<"  RY:"<<tempRpy.ry<<"   RZ:"<<tempRpy.rz<<std::endl;

    //关节信息
    std::cout<<"关节信息: "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout<<"joint"<<i+1<<": "<<waypointFK.jointpos[i]<<" ~
"<<waypointFK.jointpos[i]*180.0/M_PI<<std::endl;
    }
```

输出



67.逆解 robotServiceRobotIk

```
int robotServiceRobotIk(const double *startPointJointAngle,const aubo_robot_namespace::Pos &position, const aubo_robot_namespace::Ori &ori, aubo_robot_namespace::wayPoint_S &wayPoint);
```

函数功能	此函数为机械臂逆解函数，根据位置信息(x,y,z)和对应位置的参考姿态(w,x,y,z)得到对应位置的关节角信息
参数描述	startPointJointAngle:参考点六个关节的关节角，输入参数，单位 rad。 position：目标路点的位置，输入参数，单位 m。 ori：目标路点的参考姿态，输入参数，单位四元数。 wayPoint：输出参数。
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    /*求逆解IK*/
    /* 目标关节角
    double wp1[6] = {};
    wp1[0] = 60.443151*M_PI/180;
    wp1[1] = 42.275463*M_PI/180;
    wp1[2] = -97.679737*M_PI/180;
    wp1[3] = -49.990510*M_PI/180;
    wp1[4] = -90.007372*M_PI/180;
    wp1[5] = 62.567046*M_PI/180;
    */
    //参考点关节角
    double startpoint[6] = {65.443151*M_PI/180,
                            44.275463*M_PI/180,
                            -108.679737*M_PI/180,
                            -20.990510*M_PI/180,
                            -60.007372*M_PI/180,
                            30.567046*M_PI/180};
    //目标位置
    aubo_robot_namespace::Pos targetpos;
    targetpos.x = -0.199632;
    targetpos.y = -0.598316;
    targetpos.z = 0.018606;
    //目标姿态
    aubo_robot_namespace::Ori targetori;
    aubo_robot_namespace::Rpy targetrpy;
    targetrpy.rx = 179.977173*M_PI/180;
    targetrpy.ry = -0.027977*M_PI/180;
```

```
    targetrpy.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(targetrpy,targetori);

    aubo_robot_namespace::wayPoint_S waypointIK;
    robotService.robotServiceRobotIk(startpoint,targetpos,targetori,waypointIK);

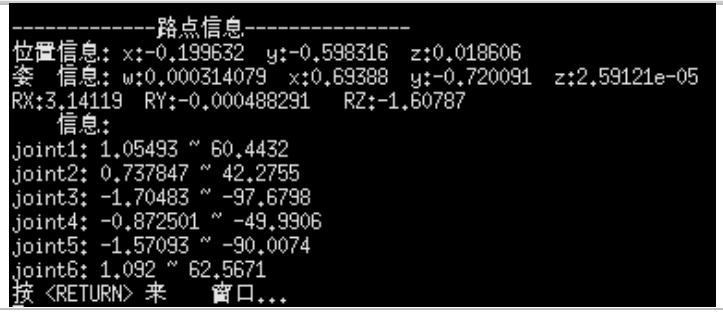
    std::cout<<std::endl<<"-------------路点信息---------------"<<std::endl;
    //位置信息
    std::cout<<"位置信息: ";
    std::cout<<"x:"<<waypointIK.cartPos.position.x<<"  ";
    std::cout<<"y:"<<waypointIK.cartPos.position.y<<"  ";
    std::cout<<"z:"<<waypointIK.cartPos.position.z<<std::endl;

    //姿态信息
    std::cout<<"姿态信息: ";
    std::cout<<"w:"<<waypointIK.orientation.w<<"  ";
    std::cout<<"x:"<<waypointIK.orientation.x<<"  ";
    std::cout<<"y:"<<waypointIK.orientation.y<<"  ";
    std::cout<<"z:"<<waypointIK.orientation.z<<std::endl;

    aubo_robot_namespace::Rpy tempRpy;
    robotService.quaternionToRPY(waypointIK.orientation,tempRpy);
    std::cout<<"RX:"<<tempRpy.rx<<"  RY:"<<tempRpy.ry<<"   RZ:"<<tempRpy.rz<<std::endl;

    //关节信息
    std::cout<<"关节信息: "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout<<"joint"<<i+1<<": "<<waypointIK.jointpos[i]<<" ~
"<<waypointIK.jointpos[i]*180.0/M_PI<<std::endl;
    }
```

输出



68.工具标定 robotServiceToolCalibration

69.检查用户坐标系是否合理 robotServiceCheckUserCoordinate

```
int robotServiceCheckUserCoordinate(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool  &coordSystem);
```

函数功能	检查提供的参数是否标定出一个坐标系

参数描述	`typedef struct`
	`{`
	`coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系`
	`CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效`
	`JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效`
	`ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数`
	`//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。`
	`}CoordCalibrateByJointAngleAndTool;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	
输出	

70.用户坐标系标定 robotServiceUserCoordinateCalibration

71.robotServiceOriMatrixToQuaternion

72.Base 坐标系转 User 坐标系 baseToUserCoordinate

```
static int baseToUserCoordinate(
        const aubo_robot_namespace::Pos          &flangeCenterPositionOnBase,    //基于基座标系的法兰盘中心位置信息
        const aubo_robot_namespace::Ori          &flangeCenterOrientationOnBase, //基于基座标系的姿态信息
```

```
        const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord,  //用户坐标系
        const aubo_robot_namespace::ToolInEndDesc  &toolInEndDesc,                 //工具参数
        aubo_robot_namespace::Pos                  &toolEndPositionOnUserCoord,    //基于用户座标系的工具末端位置信息
        aubo_robot_namespace::Ori                  &toolEndOrientationOnUserCoord  //基于用户座标系的工具末端姿态信息
);
```

函数功能	flange_center 在 Base 下坐标转工具坐标系在 User 下的坐标。

参数描述	参数 1：flangeCenterPositionOnBase，flange_center 在 Base 下的位置参数
	参数 2：flangeCenterOrientationOnBase，flange_center 在 Base 下的姿态参数
	参数 3：userCoord，参考坐标系
	`typedef struct`
	`{`
	`coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系`
	`CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效`
	`JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效`
	`ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数`
	`//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。`
	`}CoordCalibrateByJointAngleAndTool;`
	参数 4：toolInEndDesc，工具参数
	参数 5：toolEndPositionOnUserCoord，输出参数，工具在 User 下的位置参数
	参数 5：toolEndOrientationOnUserCoord ，输出参数，工具在 User 下的姿态参数

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：flange_center 在 Base 下坐标转 Tool(tcp1)在 Base 下坐标。

```
    /*F_B 转 T_B*/
    //F_B 的位置参数
    aubo_robot_namespace::Pos f_b_pos;
    f_b_pos.x = -0.199632;
    f_b_pos.y = -0.598316;
    f_b_pos.z = 0.018606;
    //F_B 的姿态参数
    aubo_robot_namespace::Ori f_b_ori;
    aubo_robot_namespace::Rpy f_b_rpy;
    f_b_rpy.rx = 179.977173*M_PI/180;
    f_b_rpy.ry = -0.027977*M_PI/180;
    f_b_rpy.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(f_b_rpy,f_b_ori);
```

```
    /**** 设置Base 坐标系 *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    //设置工具参数
    aubo_robot_namespace::ToolInEndDesc tool1;
    tool1.toolInEndPosition.x = 0;
    tool1.toolInEndPosition.y = 0;
    tool1.toolInEndPosition.z = 0.45;
    tool1.toolInEndOrientation.w = 1;
    tool1.toolInEndOrientation.x = 0;
    tool1.toolInEndOrientation.y = 0;
    tool1.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos t_u_pos;
    aubo_robot_namespace::Ori t_u_ori;

    robotService.baseToUserCoordinate(f_b_pos,f_b_ori,userCoord,tool1,t_u_pos,t_u_ori);

    std::cout << "*********  T_U_POS  *********" << std::endl;
    std::cout <<"POS X = " << t_u_pos.x << std::endl;
    std::cout <<"POS Y = " << t_u_pos.y << std::endl;
    std::cout <<"POS Z = " << t_u_pos.z << std::endl;
    std::cout << "*********  T_U_ORI  *********" << std::endl;
    std::cout << "ORI w = " << t_u_ori.w << std::endl;
    std::cout << "ORI x = " << t_u_ori.x << std::endl;
    std::cout << "ORI y = " << t_u_ori.y << std::endl;
    std::cout << "ORI z = " << t_u_ori.z << std::endl;
    std::cout << "*********  T_U_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(t_u_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



示例 2

示例讲解：flange_center 在 Base 下坐标转 flange_center 在 User(p1)下坐标。

```
/*F_B 转 T_B*/
//F_B 的位置参数
aubo_robot_namespace::Pos f_b_pos;
f_b_pos.x = -0.199632;
f_b_pos.y = -0.598316;
f_b_pos.z = 0.018606;
//F_B 的姿态参数
aubo_robot_namespace::Ori f_b_ori;
aubo_robot_namespace::Rpy f_b_rpy;
f_b_rpy.rx = 179.977173*M_PI/180;
f_b_rpy.ry = -0.027977*M_PI/180;
f_b_rpy.rz = -92.123917*M_PI/180;
robotService.RPYToQuaternion(f_b_rpy,f_b_ori);

/**** 设置 User 坐标系(p1) *****/
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
```

```
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;

    //设置工具参数
    aubo_robot_namespace::ToolInEndDesc tool_robot;
    tool_robot.toolInEndPosition.x = 0;
    tool_robot.toolInEndPosition.y = 0;
    tool_robot.toolInEndPosition.z = 0;
    tool_robot.toolInEndOrientation.w = 1;
    tool_robot.toolInEndOrientation.x = 0;
    tool_robot.toolInEndOrientation.y = 0;
    tool_robot.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos t_u_pos;
    aubo_robot_namespace::Ori t_u_ori;

    robotService.baseToUserCoordinate(f_b_pos,f_b_ori,userCoord,tool_robot,t_u_pos,t_u_ori);

    std::cout << "*********  T_U_POS  *********" << std::endl;
    std::cout <<"POS X = " << t_u_pos.x << std::endl;
    std::cout <<"POS Y = " << t_u_pos.y << std::endl;
    std::cout <<"POS Z = " << t_u_pos.z << std::endl;
    std::cout << "*********  T_U_ORI  *********" << std::endl;
    std::cout << "ORI w = " << t_u_ori.w << std::endl;
    std::cout << "ORI x = " << t_u_ori.x << std::endl;
    std::cout << "ORI y = " << t_u_ori.y << std::endl;
```

```
    std::cout << "ORI z = " << t_u_ori.z << std::endl;
    std::cout << "*********  T_U_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(t_u_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



示例 3

示例讲解：flange_center 在 Base 下坐标转 Tool(tcp1)在 User(p1)下坐标。

```
    /*F_B 转 T_B*/
    //F_B 的位置参数
    aubo_robot_namespace::Pos f_b_pos;
    f_b_pos.x = -0.199632;
    f_b_pos.y = -0.598316;
    f_b_pos.z = 0.018606;
    //F_B 的姿态参数
    aubo_robot_namespace::Ori f_b_ori;
    aubo_robot_namespace::Rpy f_b_rpy;
    f_b_rpy.rx = 179.977173*M_PI/180;
    f_b_rpy.ry = -0.027977*M_PI/180;
    f_b_rpy.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(f_b_rpy,f_b_ori);

    /**** 设置 User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
```

```
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;

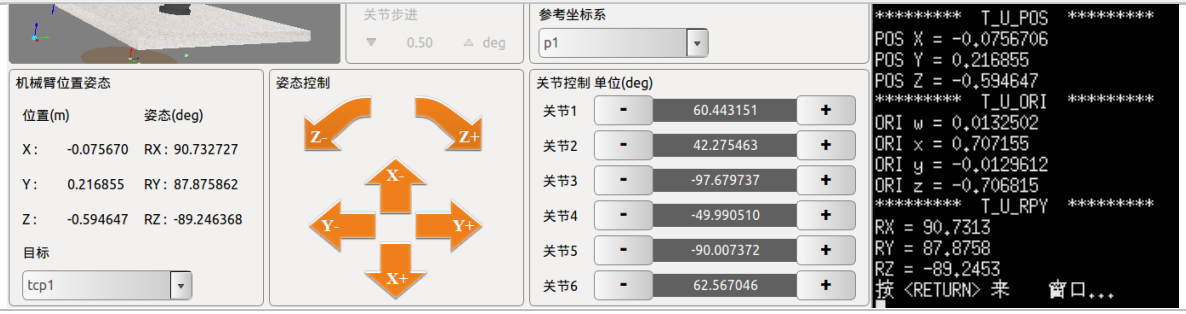
    //设置工具参数
    aubo_robot_namespace::ToolInEndDesc tool_robot;
    tool_robot.toolInEndPosition.x = 0;
    tool_robot.toolInEndPosition.y = 0;
    tool_robot.toolInEndPosition.z = 0.45;
    tool_robot.toolInEndOrientation.w = 1;
    tool_robot.toolInEndOrientation.x = 0;
    tool_robot.toolInEndOrientation.y = 0;
    tool_robot.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos t_u_pos;
    aubo_robot_namespace::Ori t_u_ori;

    robotService.baseToUserCoordinate(f_b_pos,f_b_ori,userCoord,tool_robot,t_u_pos,t_u_ori);
```

```
    std::cout << "*********  T_U_POS  *********" << std::endl;
    std::cout <<"POS X = " << t_u_pos.x << std::endl;
    std::cout <<"POS Y = " << t_u_pos.y << std::endl;
    std::cout <<"POS Z = " << t_u_pos.z << std::endl;
    std::cout << "*********  T_U_ORI  *********" << std::endl;
    std::cout << "ORI w = " << t_u_ori.w << std::endl;
    std::cout << "ORI x = " << t_u_ori.x << std::endl;
    std::cout << "ORI y = " << t_u_ori.y << std::endl;
    std::cout << "ORI z = " << t_u_ori.z << std::endl;
    std::cout << "*********  T_U_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(t_u_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



73. F_B 坐标系转 T_B 坐标

```
static int baseToBaseAdditionalTool(
        const aubo_robot_namespace::Pos      &flangeCenterPositionOnBase,   //基于基座标系的法兰盘中心位置信息
         const aubo_robot_namespace::Ori           &flangeCenterOrientationOnBase, //基于基座标系的法兰盘姿态信息
         const aubo_robot_namespace::ToolInEndDesc  &toolInEndDesc,                //工具信息
         aubo_robot_namespace::Pos                 &toolEndPositionOnBase,    //基于用户座标系的工具末端位置信息
         aubo_robot_namespace::Ori                 &toolEndOrientationOnBase  //基于用户座标系的工具末端姿态信息
);
```

函数功能	flange_center 在 Base 下坐标转工具坐标系在 Base 下的坐标。
参数描述	参数 1：flangeCenterPositionOnBase，flange_center 在 Base 下的位置参数

	参数 2：flangeCenterOrientationOnBase，flange_center 在 Base 下的姿态参数 参数 3：toolInEndDesc，工具参数 参数 4：toolEndPositionOnUserCoord，输出参数，工具在 Baser 下的位置参数 参数 5：toolEndOrientationOnUserCoord ，输出参数，工具在 Base 下的姿态参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	示例讲解：flange_center 在 Base 下坐标系转 Tool(tcp1)在 Base 下坐标。

```
/*F_B 转 T_B*/
//F_B 的位置参数
aubo_robot_namespace::Pos f_b_pos;
f_b_pos.x = -0.199632;
f_b_pos.y = -0.598316;
f_b_pos.z = 0.018606;
//F_B 的姿态参数
aubo_robot_namespace::Ori f_b_ori;
aubo_robot_namespace::Rpy f_b_rpy;
f_b_rpy.rx = 179.977173*M_PI/180;
f_b_rpy.ry = -0.027977*M_PI/180;
f_b_rpy.rz = -92.123917*M_PI/180;
robotService.RPYToQuaternion(f_b_rpy,f_b_ori);


//设置工具参数
aubo_robot_namespace::ToolInEndDesc tool_robot;
tool_robot.toolInEndPosition.x = 0;
tool_robot.toolInEndPosition.y = 0;
tool_robot.toolInEndPosition.z = 0.45;
tool_robot.toolInEndOrientation.w = 1;
tool_robot.toolInEndOrientation.x = 0;
tool_robot.toolInEndOrientation.y = 0;
tool_robot.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos t_b_pos;
aubo_robot_namespace::Ori t_b_ori;

robotService.baseToBaseAdditionalTool(f_b_pos,f_b_ori,tool_robot,t_b_pos,t_b_ori);

std::cout << "*********  T_B_POS  *********" << std::endl;
std::cout <<"POS X = " << t_b_pos.x << std::endl;
std::cout <<"POS Y = " << t_b_pos.y << std::endl;
std::cout <<"POS Z = " << t_b_pos.z << std::endl;
```

```
    std::cout << "*********  T_B_ORI  *********" << std::endl;
    std::cout << "ORI w = " << t_b_ori.w << std::endl;
    std::cout << "ORI x = " << t_b_ori.x << std::endl;
    std::cout << "ORI y = " << t_b_ori.y << std::endl;
    std::cout << "ORI z = " << t_b_ori.z << std::endl;
    std::cout << "*********  T_B_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(t_b_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



74. User 坐标系转 Base 坐标系 userToBaseCoordinate

```
static int userToBaseCoordinate(
        const aubo_robot_namespace::Pos            &toolEndPositionOnUserCoord,    //基于用户座标系的工具末端位置信息
        const aubo_robot_namespace::Ori            &toolEndOrientationOnUserCoord, //基于用户座标系的工具末端姿态信息
        const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord,  //用户坐标系
        const aubo_robot_namespace::ToolInEndDesc  &toolInEndDesc,                 //工具信息
        aubo_robot_namespace::Pos                  &flangeCenterPositionOnBase,    //基于基座标系的法兰盘中心位置信息
        aubo_robot_namespace::Ori                  &flangeCenterOrientationOnBase  //基于基座标系的法兰盘中心姿态信息
);
```

函数功能	工具坐标系在 User 下的坐标转 flange_center 在 Base 下坐标。
参数描述	参数 1：toolEndPositionOnUserCoord，工具在 User 下的位置参数 参数 2：toolEndOrientationOnUserCoord，工具在 User 下的姿态参数 参数 3：userCoord，参考坐标系 typedef struct

	{
	coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系
	CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效
	JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效
	ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数
	//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。
	}CoordCalibrateByJointAngleAndTool;
	不支持 userCoord==EndCoordinate,如果 userCoord==EndCoordinate 会报参数错误(ErrCode_ParamError)
	参数 4：toolInEndDesc，工具参数
	参数 5：flangeCenterPositionOnBase，输出参数，flange_center 在 Base 下的位置参数
	参数 5：flangeCenterOrientationOnBase ，输出参数，flange_center 在 Base 下的姿态参数

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	示例讲解：tcp1 在 Base 下坐标转 flange_center 在 Base 下坐标。

```
/*T_U 转 F_B*/
//T_U 的位置参数(tcp1 在 Base 下)
aubo_robot_namespace::Pos t_u_pos;
t_u_pos.x = -0.199819;
t_u_pos.y = -0.598529;
t_u_pos.z = -0.431394;
//T_U 的姿态参数(tcp1 在 Base 下)
aubo_robot_namespace::Ori t_u_ori;
aubo_robot_namespace::Rpy t_u_rpy;
t_u_rpy.rx = 179.977*M_PI/180;
t_u_rpy.ry = -0.027977*M_PI/180;
t_u_rpy.rz = -92.1239*M_PI/180;
robotService.RPYToQuaternion(t_u_rpy,t_u_ori);

/**** 设置 Base 坐标系 *****/
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置工具参数
aubo_robot_namespace::ToolInEndDesc tool_robot;
tool_robot.toolInEndPosition.x = 0;
tool_robot.toolInEndPosition.y = 0;
tool_robot.toolInEndPosition.z = 0.45;
tool_robot.toolInEndOrientation.w = 1;
tool_robot.toolInEndOrientation.x = 0;
```

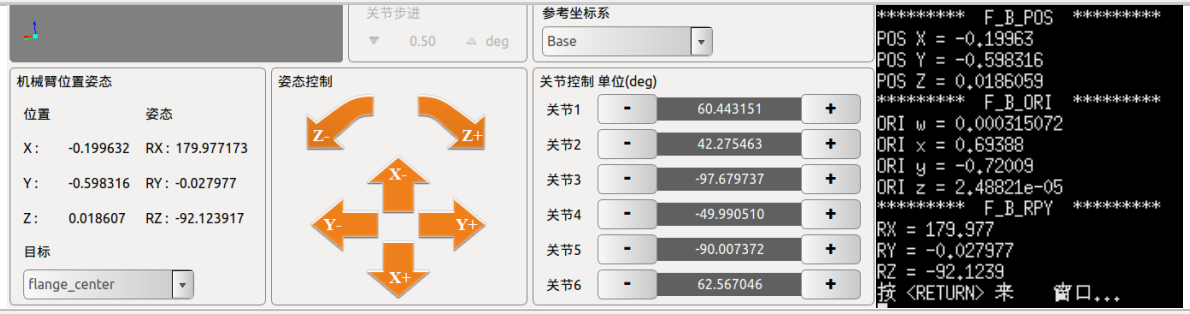
```
    tool_robot.toolInEndOrientation.y = 0;
    tool_robot.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos f_b_pos;
    aubo_robot_namespace::Ori f_b_ori;

    robotService.userToBaseCoordinate(t_u_pos,t_u_ori,userCoord,tool_robot,f_b_pos,f_b_ori);

    std::cout << "*********  F_B_POS  *********" << std::endl;
    std::cout <<"POS X = " << f_b_pos.x << std::endl;
    std::cout <<"POS Y = " << f_b_pos.y << std::endl;
    std::cout <<"POS Z = " << f_b_pos.z << std::endl;
    std::cout << "*********  F_B_ORI  *********" << std::endl;
    std::cout << "ORI w = " << f_b_ori.w << std::endl;
    std::cout << "ORI x = " << f_b_ori.x << std::endl;
    std::cout << "ORI y = " << f_b_ori.y << std::endl;
    std::cout << "ORI z = " << f_b_ori.z << std::endl;
    std::cout << "*********  F_B_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(f_b_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



示例 2

示例讲解：flange_center 在 p1 下的坐标转 flange_center 在 Base 下坐标。

```
    /*T_U 转 F_B*/
    //T_U 的位置参数(flange_center 在 p1 下)
    aubo_robot_namespace::Pos t_u_pos;
    t_u_pos.x = 0.374329;
    t_u_pos.y = 0.217043;
    t_u_pos.z = -0.594434;
    //T_U 的姿态参数(flange_center 在 p1 下)
```

```
    aubo_robot_namespace::Ori t_u_ori;
    aubo_robot_namespace::Rpy t_u_rpy;
    t_u_rpy.rx = 90.7313*M_PI/180;
    t_u_rpy.ry = 87.8758*M_PI/180;
    t_u_rpy.rz = -89.2453*M_PI/180;
    robotService.RPYToQuaternion(t_u_rpy,t_u_ori);

    /**** 设置User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;
```

```cpp
//设置工具参数
aubo_robot_namespace::ToolInEndDesc tool_robot;
tool_robot.toolInEndPosition.x = 0;
tool_robot.toolInEndPosition.y = 0;
tool_robot.toolInEndPosition.z = 0;
tool_robot.toolInEndOrientation.w = 1;
tool_robot.toolInEndOrientation.x = 0;
tool_robot.toolInEndOrientation.y = 0;
tool_robot.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos f_b_pos;
aubo_robot_namespace::Ori f_b_ori;

robotService.userToBaseCoordinate(t_u_pos,t_u_ori,userCoord,tool_robot,f_b_pos,f_b_ori);

std::cout << "*********  F_B_POS  *********" << std::endl;
std::cout <<"POS X = " << f_b_pos.x << std::endl;
std::cout <<"POS Y = " << f_b_pos.y << std::endl;
std::cout <<"POS Z = " << f_b_pos.z << std::endl;
std::cout << "*********  F_B_ORI  *********" << std::endl;
std::cout << "ORI w = " << f_b_ori.w << std::endl;
std::cout << "ORI x = " << f_b_ori.x << std::endl;
std::cout << "ORI y = " << f_b_ori.y << std::endl;
std::cout << "ORI z = " << f_b_ori.z << std::endl;
std::cout << "*********  F_B_RPY  *********" << std::endl;
aubo_robot_namespace::Rpy rpytmp;
robotService.quaternionToRPY(f_b_ori,rpytmp);
std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



示例 3

示例讲解：tcp1 在 p1 下坐标转 flange_center 在 Base 下坐标。

```
    /*T_U 转 F_B*/
    //T_U 的位置参数(tcp1 在 p1 下)
    aubo_robot_namespace::Pos t_u_pos;
    t_u_pos.x = -0.075670;
    t_u_pos.y = 0.216855;
    t_u_pos.z = -0.594647;
    //T_U 的姿态参数(tcp1 在 p1 下)
    aubo_robot_namespace::Ori t_u_ori;
    aubo_robot_namespace::Rpy t_u_rpy;
    t_u_rpy.rx = 90.732727*M_PI/180;
    t_u_rpy.ry = 87.875862*M_PI/180;
    t_u_rpy.rz = -89.246368*M_PI/180;
    robotService.RPYToQuaternion(t_u_rpy,t_u_ori);

    /**** 设置 User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
```

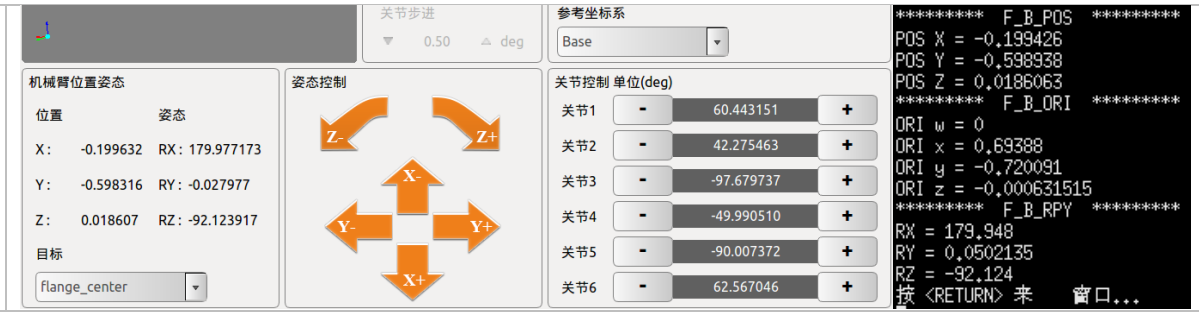
```
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;

    //设置工具参数
    aubo_robot_namespace::ToolInEndDesc tool_robot;
    tool_robot.toolInEndPosition.x = 0;
    tool_robot.toolInEndPosition.y = 0;
    tool_robot.toolInEndPosition.z = 0.45;
    tool_robot.toolInEndOrientation.w = 1;
    tool_robot.toolInEndOrientation.x = 0;
    tool_robot.toolInEndOrientation.y = 0;
    tool_robot.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos f_b_pos;
    aubo_robot_namespace::Ori f_b_ori;

    robotService.userToBaseCoordinate(t_u_pos,t_u_ori,userCoord,tool_robot,f_b_pos,f_b_ori);

    std::cout << "*********  F_B_POS  *********" << std::endl;
    std::cout <<"POS X = " << f_b_pos.x << std::endl;
    std::cout <<"POS Y = " << f_b_pos.y << std::endl;
    std::cout <<"POS Z = " << f_b_pos.z << std::endl;
    std::cout << "*********  F_B_ORI  *********" << std::endl;
    std::cout << "ORI w = " << f_b_ori.w << std::endl;
    std::cout << "ORI x = " << f_b_ori.x << std::endl;
    std::cout << "ORI y = " << f_b_ori.y << std::endl;
    std::cout << "ORI z = " << f_b_ori.z << std::endl;
    std::cout << "*********  F_B_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(f_b_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出	

75.User 坐标系下位置参数转 Base 坐标系 userCoordPointToBasePoint

```
static int userCoordPointToBasePoint(const aubo_robot_namespace::Pos &userCoordPoint,
                                      const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoordSystem,
                                      aubo_robot_namespace::Pos &basePoint);
```

函数功能	将空间内一个基于用户坐标系的位置信息(x,y,z)转换成基于基座标下的位置信息(x,y,z),同一个工具参数。

参数描述	参数 1：userCoordPoint，User 下的位置参数
	参数 2：userCoord，参考坐标系
	`typedef struct`
	`{`
	`coordinate_refer coordType; //坐标系类型:基座坐标系,末端坐标系或工具坐标系,用户坐标系`
	`CoordCalibrateMathod methods; //坐标系标定方法 该参数只在 coordType==WorldCoordinate 时有效`
	`JointParam wayPointArray[3]; //用于标定用户坐标系的 3 个点(关节角)该参数只在 coordType==WorldCoordinate 时有效`
	`ToolInEndDesc toolDesc; //工具描述 当 coordType==EndCoordinate 时，该参数表示工具参数`
	`//当 coordType==WorldCoordinat 时，该参数表示标定用户坐标系的 3 个点时，使用工具描述。`
	`}CoordCalibrateByJointAngleAndTool;`
	参数 3：basePoint，输出参数，Base 下的位置参数

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例 1	示例讲解：tcp1 在 p1 下的(x,y,z)转为 tcp1 在 base 下的(x,y,z)。
	`/*T_U 转 T_B*/`
	`//T_U 的位置参数(tcp1 在 p1 下)`
	`aubo_robot_namespace::Pos t_u_pos;`

```
    t_u_pos.x = -0.075670;
    t_u_pos.y = 0.216855;
    t_u_pos.z = -0.594647;

    /**** 设置User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;

    aubo_robot_namespace::Pos t_b_pos;
    robotService.userCoordPointToBasePoint(t_u_pos,userCoord,t_b_pos);
```

	`std::cout << "*********  T_B_POS  *********" << std::endl;`
	`std::cout <<"POS X = " << t_b_pos.x << std::endl;`
	`std::cout <<"POS Y = " << t_b_pos.y << std::endl;`
	`std::cout <<"POS Z = " << t_b_pos.z << std::endl;`

输出	

76. flange 姿态转 Tool 姿态 endOrientation2ToolOrientation

```
static int endOrientation2ToolOrientation(aubo_robot_namespace::Ori &tcpOriInEnd, const aubo_robot_namespace::Ori &endOri,  aubo_robot_namespace::Ori &toolOri);
```

函数功能	将 flange_center 姿态转为 tool 姿态。
参数描述	参数 1：tcpOriInEnd，工具姿态参数 参数 2：endOri，flange_center 姿态 参数 3：toolOri int，输出参数，tool 姿态
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	示例讲解：flange_center 在 Base 下姿态转 tcp2 在 base 姿态。

```
    /*F_O 转 T_O*/
    //F_B 的姿态参数
    aubo_robot_namespace::Ori f_b_ori;
    aubo_robot_namespace::Rpy f_b_rpy;
    f_b_rpy.rx = 179.977173*M_PI/180;
    f_b_rpy.ry = -0.027977*M_PI/180;
    f_b_rpy.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(f_b_rpy,f_b_ori);

    //设置工具姿态参数
```

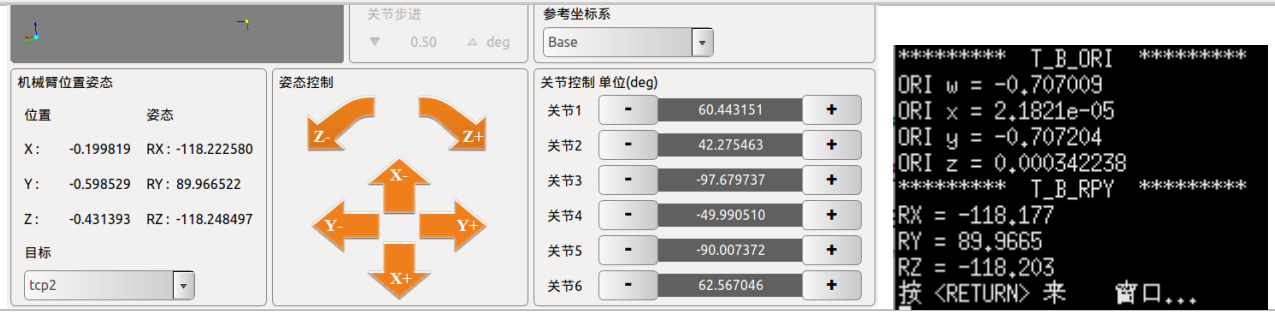
```
    aubo_robot_namespace::Ori tool_ori;
    aubo_robot_namespace::Rpy tool_rpy;
    tool_rpy.rx = 163.507258*M_PI/180;
    tool_rpy.ry = -89.991616*M_PI/180;
    tool_rpy.rz = -75.605219*M_PI/180;
    robotService.RPYToQuaternion(tool_rpy,tool_ori);

    aubo_robot_namespace::Ori t_b_ori;

    robotService.endOrientation2ToolOrientation(tool_ori,f_b_ori,t_b_ori);

    std::cout << "*********  T_B_ORI  *********" << std::endl;
    std::cout << "ORI w = " << t_b_ori.w << std::endl;
    std::cout << "ORI x = " << t_b_ori.x << std::endl;
    std::cout << "ORI y = " << t_b_ori.y << std::endl;
    std::cout << "ORI z = " << t_b_ori.z << std::endl;
    std::cout << "*********  T_U_RPY  *********" << std::endl;
    aubo_robot_namespace::Rpy rpytmp;
    robotService.quaternionToRPY(t_b_ori,rpytmp);
    std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
    std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
    std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出



77. Tool 姿态转 flange 姿态 toolOrientation2EndOrientation

```
static int toolOrientation2EndOrientation(aubo_robot_namespace::Ori &tcpOriInEnd, const aubo_robot_namespace::Ori &toolOri, aubo_robot_namespace::Ori &endOri);
```

函数功能	将 tool 姿态转为 flange_center 姿态。

参数描述	参数 1：tcpOriInEnd，工具姿态参数 参数 2：toolOri，tool 姿态 参数 3：endOri，输出参数，flange_center 姿态
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	示例讲解：tcp2 在 Base 下姿态转 flange_center 在 Base 下姿态。

```
/*T_O 转 F_O*/
//T_B 的姿态参数
aubo_robot_namespace::Ori t_b_ori;
aubo_robot_namespace::Rpy t_b_rpy;
t_b_rpy.rx = -118.222580*M_PI/180;
t_b_rpy.ry = 89.966522*M_PI/180;
t_b_rpy.rz = -118.248497*M_PI/180;
robotService.RPYToQuaternion(t_b_rpy,t_b_ori);

//设置工具姿态参数
aubo_robot_namespace::Ori tool_ori;
aubo_robot_namespace::Rpy tool_rpy;
tool_rpy.rx = 163.507258*M_PI/180;
tool_rpy.ry = -89.991616*M_PI/180;
tool_rpy.rz = -75.605219*M_PI/180;
robotService.RPYToQuaternion(tool_rpy,tool_ori);

aubo_robot_namespace::Ori f_b_ori;

robotService.toolOrientation2EndOrientation(tool_ori,t_b_ori,f_b_ori);

std::cout << "*********  F_B_ORI  *********" << std::endl;
std::cout << "ORI w = " << f_b_ori.w << std::endl;
std::cout << "ORI x = " << f_b_ori.x << std::endl;
std::cout << "ORI y = " << f_b_ori.y << std::endl;
std::cout << "ORI z = " << f_b_ori.z << std::endl;
std::cout << "*********  F_B_RPY  *********" << std::endl;
aubo_robot_namespace::Rpy rpytmp;
robotService.quaternionToRPY(f_b_ori,rpytmp);
std::cout << "RX = " << rpytmp.rx*180/M_PI << std::endl;
std::cout << "RY = " << rpytmp.ry*180/M_PI << std::endl;
std::cout << "RZ = " << rpytmp.rz*180/M_PI << std::endl;
```

输出	

78.根据位置获取目标路点信息 getTargetWaypointByPosition

```
static int getTargetWaypointByPosition(const aubo_robot_namespace::wayPoint_S       &sourceWayPointOnBaseCoord,
                                       const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoordSystem,
                                       const aubo_robot_namespace::Pos              &toolEndPosition,
                                       const aubo_robot_namespace::ToolInEndDesc    &toolInEndDesc,
                                       aubo_robot_namespace::wayPoint_S             &targetWayPointOnBaseCoord);
```

函数功能	根据位置参数求得目标路点信息（姿态同源路点）
参数描述	参数 1：tcpOriInEnd，工具姿态参数 参数 2：toolOri，tool 姿态 参数 3：endOri，输出参数，flange_center 姿态
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*GetTargetWaypointByPos*/`
	`//定义源路点`
	`aubo_robot_namespace::wayPoint_S sourcewaypoint;`
	`sourcewaypoint.jointpos[0] = 68.701547*M_PI/180;`
	`sourcewaypoint.jointpos[1] = 16.160485*M_PI/180;`
	`sourcewaypoint.jointpos[2] = -115.288498*M_PI/180;`
	`sourcewaypoint.jointpos[3] = -41.482864*M_PI/180;`
	`sourcewaypoint.jointpos[4] = -90.012365*M_PI/180;`
	`sourcewaypoint.jointpos[5] = 70.825442*M_PI/180;`
	`sourcewaypoint.cartPos.position.x = -0.067683;`
	`sourcewaypoint.cartPos.position.y = -0.508058;`
	`sourcewaypoint.cartPos.position.z = 0.147545;`
	`aubo_robot_namespace::Rpy rpy_source;`
	`rpy_source.rx = 179.977173*M_PI/180;`

```
    rpy_source.ry = -0.027977*M_PI/180;
    rpy_source.rz = -92.123917*M_PI/180;
    robotService.RPYToQuaternion(rpy_source,sourcewaypoint.orientation);

    /**** 设置User 坐标系(p1) *****/
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis;

    userCoord.wayPointArray[0].jointPos[0] = -16.418907*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 11.715555*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -133.748860*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -55.472799*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -89.999982*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -14.340675*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -55.333572*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 11.813817*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -138.234095*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -38.422666*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -75.692136*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -55.999864*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -81.686194*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 26.339385*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -126.337995*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -66.550139*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -65.208444*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -78.755974*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc tool_usercoord;
    tool_usercoord.toolInEndPosition.x = 0;
    tool_usercoord.toolInEndPosition.y = 0;
    tool_usercoord.toolInEndPosition.z = 0.45;
    tool_usercoord.toolInEndOrientation.w = 1;
    tool_usercoord.toolInEndOrientation.x = 0;
    tool_usercoord.toolInEndOrientation.y = 0;
    tool_usercoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = tool_usercoord;

    //设置工具参数
    aubo_robot_namespace::ToolInEndDesc tool_robot;
    tool_robot.toolInEndPosition.x = 0;
```

```
    tool_robot.toolInEndPosition.y = 0;
    tool_robot.toolInEndPosition.z = 0.45;
    tool_robot.toolInEndOrientation.w = 1;
    tool_robot.toolInEndOrientation.x = 0;
    tool_robot.toolInEndOrientation.y = 0;
    tool_robot.toolInEndOrientation.z = 0;

    //目标位置参数
    aubo_robot_namespace::Pos targetpos;
    targetpos.x = -0.075670;
    targetpos.y = 0.216855;
    targetpos.z = -0.594647;

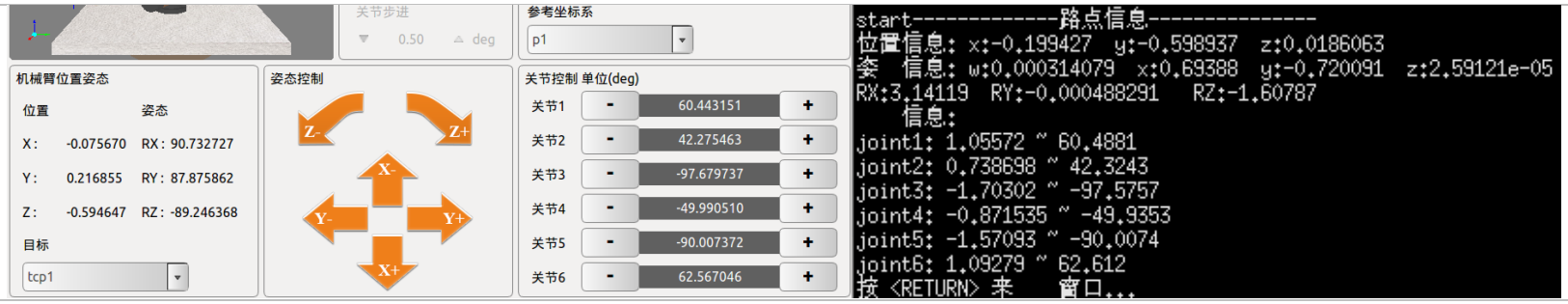
    aubo_robot_namespace::wayPoint_S targetwaypoint;
    robotService.getTargetWaypointByPosition(sourcewaypoint,userCoord,targetpos,tool_robot,targetwaypoint);

    std::cout<<std::endl<<"start-------------路点信息---------------"<<std::endl;
    //位置信息
    std::cout<<"位置信息: ";
    std::cout<<"x:"<<targetwaypoint.cartPos.position.x<<"  ";
    std::cout<<"y:"<<targetwaypoint.cartPos.position.y<<"  ";
    std::cout<<"z:"<<targetwaypoint.cartPos.position.z<<std::endl;

    //姿态信息
    std::cout<<"姿态信息: ";
    std::cout<<"w:"<<targetwaypoint.orientation.w<<"  ";
    std::cout<<"x:"<<targetwaypoint.orientation.x<<"  ";
    std::cout<<"y:"<<targetwaypoint.orientation.y<<"  ";
    std::cout<<"z:"<<targetwaypoint.orientation.z<<std::endl;

//    aubo_robot_namespace::Rpy tempRpy;
    robotService.quaternionToRPY(targetwaypoint.orientation,tempRpy);
    std::cout<<"RX:"<<tempRpy.rx<<"  RY:"<<tempRpy.ry<<"   RZ:"<<tempRpy.rz<<std::endl;

    //关节信息
    std::cout<<"关节信息: "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout<<"joint"<<i+1<<": "<<targetwaypoint.jointpos[i]<<" ~
"<<targetwaypoint.jointpos[i]*180.0/M_PI<<std::endl;
    }
```

输出	

79. 四元数转欧拉角 quaternionToRPY

```
int quaternionToRPY(const aubo_robot_namespace::Ori &ori, aubo_robot_namespace::Rpy &rpy);
```

函数功能	四元素转欧拉角
参数描述	ori：四元数，输入参数 rpy：欧拉角，输出参数，单位 rad
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`aubo_robot_namespace::Ori ori_tmp;`
	`ori_tmp.w = 0.000314;`
	`ori_tmp.x = 0.69388;`
	`ori_tmp.y = -0.720091;`
	`ori_tmp.z = 0.000026;`
	
	`aubo_robot_namespace::Rpy rpy_tmp;`
	`robotService.quaternionToRPY(ori_tmp,rpy_tmp);`
	`std::cout << "*********  RPY  *********" << std::endl;`
	`std::cout << "RX = " << rpy_tmp.rx*180/M_PI << std::endl;`
	`std::cout << "RY = " << rpy_tmp.ry*180/M_PI << std::endl;`
	`std::cout << "RZ = " << rpy_tmp.rz*180/M_PI << std::endl;`

输出	********* RPY ********* RX = 179.977 RY = -0.0279774 RZ = -92.1239

80.欧拉角转四元数 RPYToQuaternion

int RPYToQuaternion(const aubo_robot_namespace::Rpy &rpy, aubo_robot_namespace::Ori &ori);	
函数功能	欧拉角转四元数
参数描述	rpy：欧拉角，输入参数，单位 rad ori：四元数，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`aubo_robot_namespace::Rpy rpy_tmp;`
	`rpy_tmp.rx = 179.977173*M_PI/180;`
	`rpy_tmp.ry = -0.027977*M_PI/180;`
	`rpy_tmp.rz = -92.123917*M_PI/180;`
	
	`aubo_robot_namespace::Ori ori_tmp;`
	`robotService.RPYToQuaternion(rpy_tmp,ori_tmp);`
	
	`std::cout << "*********  ORI  *********" << std::endl;`
	`std::cout << "ORI w = " << ori_tmp.w << std::endl;`
	`std::cout << "ORI x = " << ori_tmp.x << std::endl;`
	`std::cout << "ORI y = " << ori_tmp.y << std::endl;`
	`std::cout << "ORI z = " << ori_tmp.z << std::endl;`

输出	********* ORI ********* ORI w = 0.000314079 ORI x = 0.69388 ORI y = -0.720091 ORI z = 2.59121e-05

81.根据错误号返回错误信息 getErrDescByCode

std::string getErrDescByCode(aubo_robot_namespace::RobotErrorCode code);	
函数功能	根据错误号获取错误信息
参数描述	code:错误号
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`aubo_robot_namespace::RobotErrorCode errorcode;`
	`errorcode = aubo_robot_namespace::ErrCode_IkFailed;`
	
	`std::cout << "err desc is : " << robotService.getErrDescByCode(errorcode) << std::endl;`

输出	err desc is : inverse kin failed.

机械臂控制接口

82.机械臂控制 rootServiceRobotControl

83.返回电源状态 robotServicePowerControl

84.机械臂释放刹车 robotServiceReleaseBrake

末端工具接口

85.设置无工具动力学参数 robotServiceSetNoneToolDynamicsParam

```
int  robotServiceSetNoneToolDynamicsParam();
```

函数功能	设置无工具的动力学参数
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置无工具动力学参数*/`
	`ret = robotService.robotServiceSetNoneToolDynamicsParam();`
	`std::cout << "set none tool dynamic param ret is : " << ret << std::endl;`

输出	set none tool dynamic param ret is : 0

86.设置工具的动力学参数 robotServiceSetToolDynamicsParam

```
int  robotServiceSetToolDynamicsParam(const aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam);
```

函数功能	设置工具的动力学参数

参数描述	toolDynamicsParam：工具动力学参数
	`typedef struct`
	`{`
	`  double positionX;  //工具重心的X坐标`
	`  double positionY;  //工具重心的Y坐标`
	`  double positionZ;  //工具重心的Z坐标`
	`  double payload;  //工具重量`
	`  ToolInertia toolInertia;  //工具惯量  预留`
	`}ToolDynamicsParam;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置工具动力学参数*/`
	`aubo_robot_namespace::ToolDynamicsParam m_set_tool_dyn;`
	`m_set_tool_dyn.payload = 0.5;`
	`m_set_tool_dyn.positionX = 0;`
	`m_set_tool_dyn.positionY = 0;`
	`m_set_tool_dyn.positionZ = 0.05;`
	`m_set_tool_dyn.toolInertia.xx = 0;`
	`m_set_tool_dyn.toolInertia.xy = 0;`
	`m_set_tool_dyn.toolInertia.xz = 0;`
	`m_set_tool_dyn.toolInertia.yy= 0;`
	`m_set_tool_dyn.toolInertia.yz = 0;`
	`m_set_tool_dyn.toolInertia.zz = 0;`
	`ret = robotService.robotServiceSetToolDynamicsParam(m_set_tool_dyn);`
	`std::cout << "set tool dynamic param ret is : " << ret << std::endl;`

输出	set tool dynamic param ret is : 0

87.获取工具的动力学参数 robotServiceGetToolDynamicsParam

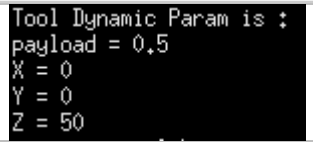
```
int  robotServiceGetToolDynamicsParam(aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam);
```

函数功能	获取工具的动力学参数

参数描述	toolDynamicsParam：工具动力学参数，输出参数
	`typedef struct`
	`{`
	`  double positionX;  //工具重心的X坐标`
	`  double positionY;  //工具重心的Y坐标`
	`  double positionZ;  //工具重心的Z坐标`
	`  double payload;  //工具重量`
	`  ToolInertia toolInertia;  //工具惯量  预留`
	`}ToolDynamicsParam;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取工具动力学参数*/`
	`aubo_robot_namespace::ToolDynamicsParam m_get_tool_dyn;`
	`robotService.robotServiceGetToolDynamicsParam(m_get_tool_dyn);`
	`std::cout << "Tool Dynamic Param is :" << std::endl;`
	`std::cout << "payload = " << m_get_tool_dyn.payload << std::endl;`
	`std::cout << "X = " << m_get_tool_dyn.positionX << std::endl;`
	`std::cout << "Y = " << m_get_tool_dyn.positionY << std::endl;`
	`std::cout << "Z = " << m_get_tool_dyn.positionZ << std::endl;`

输出	

88.设置无工具运动学参数 robotServiceSetNoneToolKinematicsParam

```
int  robotServiceSetNoneToolKinematicsParam();
```

函数功能	设置无工具运动学参数
参数描述	

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置无工具运动学参数*/`
	`ret = robotService.robotServiceSetNoneToolKinematicsParam();`
	`std::cout << "set none tool kinematic param ret is : " << ret << std::endl;`

输出	`set none tool kinematic param ret is : 0`

89.设置工具的运动学参数 robotServiceSetToolKinematicsParam

```
int  robotServiceSetToolKinematicsParam(const aubo_robot_namespace::ToolKinematicsParam &toolKinematicsParam);
```

函数功能	设置工具的运动学参数

参数描述	toolKinematicsParam：工具运动学参数
	`typedef struct`
	`{`
	`  Pos  toolInEndPosition;  //工具相对末端坐标系的位置`
	`  Ori  toolInEndOrientation;  //工具相对末端坐标系的姿态`
	`}ToolInEndDesc;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置工具运动学参数*/`
	`aubo_robot_namespace::ToolKinematicsParam m_set_tool_kin;`
	`m_set_tool_kin.toolInEndPosition.x = 0;`
	`m_set_tool_kin.toolInEndPosition.y = 0.05;`
	`m_set_tool_kin.toolInEndPosition.z = 0.1;`
	`m_set_tool_kin.toolInEndOrientation.w = 1;`
	`m_set_tool_kin.toolInEndOrientation.x = 0;`
	`m_set_tool_kin.toolInEndOrientation.y = 0;`
	`m_set_tool_kin.toolInEndOrientation.z = 0;`
	`ret = robotService.robotServiceSetToolKinematicsParam(m_set_tool_kin);`
	`std::cout << "set tool kinematic param ret is : " << ret << std::endl;`

输出	`set tool kinematic param ret is : 0`

90.获取工具的运动学参数 robotServiceGetToolKinematicsParam

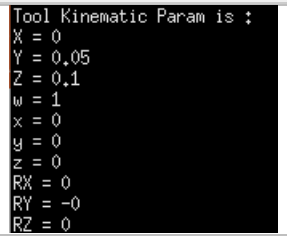
```
int  robotServiceGetToolKinematicsParam(aubo_robot_namespace::ToolKinematicsParam &toolKinematicsParam);
```

函数功能	获取工具的运动学参数

参数描述	toolKinematicsParam：工具运动学参数，输出参数
	`typedef struct`
	`{`
	`    Pos        toolInEndPosition;     //工具相对末端坐标系的位置`
	`    Ori        toolInEndOrientation;  //工具相对末端坐标系的姿态`
	`}ToolInEndDesc;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取工具运动学参数*/`
	`aubo_robot_namespace::ToolKinematicsParam m_get_tool_kin;`
	`robotService.robotServiceGetToolKinematicsParam(m_get_tool_kin);`
	`std::cout << "Tool Kinematic Param is :" << std::endl;`
	`std::cout << "X = " << m_get_tool_kin.toolInEndPosition.x << std::endl;`
	`std::cout << "Y = " << m_get_tool_kin.toolInEndPosition.y << std::endl;`
	`std::cout << "Z = " << m_get_tool_kin.toolInEndPosition.z << std::endl;`
	`std::cout << "w = " << m_get_tool_kin.toolInEndOrientation.w << std::endl;`
	`std::cout << "x = " << m_get_tool_kin.toolInEndOrientation.x << std::endl;`
	`std::cout << "y = " << m_get_tool_kin.toolInEndOrientation.y << std::endl;`
	`std::cout << "z = " << m_get_tool_kin.toolInEndOrientation.z << std::endl;`
	`aubo_robot_namespace::Rpy rpytmp;`
	`robotService.quaternionToRPY(m_get_tool_kin.toolInEndOrientation,rpytmp);`
	`std::cout << "RX = " << rpytmp.rx << std::endl;`
	`std::cout << "RY = " << rpytmp.ry << std::endl;`
	`std::cout << "RZ = " << rpytmp.rz << std::endl;`

输出	

机械臂相关属性获取与设置

91.获取机械臂当前工作模式 robotServiceGetRobotWorkMode

int robotServiceGetRobotWorkMode(aubo_robot_namespace::RobotWorkMode &mode);	
函数功能	获取机械臂当前工作模式
参数描述	传出参数，表示机械臂当前模式。0：仿真模式；1：真实模式。 typedef enum{ RobotModeSimulator, //机械臂仿真模式 RobotModeReal //机械臂真实模式 }RobotWorkMode;
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	//获取机械臂当前工作模式
	aubo_robot_namespace::RobotWorkMode m_get_workmode;
	robotService.robotServiceGetRobotWorkMode(m_get_workmode);
	std::cout << "robot work mode is " << m_get_workmode << std::endl;

输出	robot work mode is 0 仿真模式

92.设置机械臂当前工作模式 robotServiceSetRobotWorkMode

int robotServiceSetRobotWorkMode(aubo_robot_namespace::RobotWorkMode mode);	
函数功能	设置当前机械臂模式 仿真或真实
参数描述	typedef enum{ RobotModeSimulator, //机械臂仿真模式 RobotModeReal //机械臂真实模式 }RobotWorkMode;
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	//设置机械臂当前工作模式
	aubo_robot_namespace::RobotWorkMode m_set_workmode = aubo_robot_namespace::RobotModeSimulator;
	ret = robotService.robotServiceSetRobotWorkMode(m_set_workmode);

	`std::cout << "set robot work mode ret is " << ret << std::endl;`
输出	`set robot work mode ret is 0`

93.获取重力分量 robotServiceGetRobotGravityComponent

`int robotServiceGetRobotGravityComponent(aubo_robot_namespace::RobotGravityComponent &gravityComponent);`	
函数功能	获取重力分量
参数描述	gravityComponent：重力分量，输出参数 需连接真实机器人
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取重力分量*/`
	`aubo_robot_namespace::RobotGravityComponent gravity;`
	`robotService.robotServiceGetRobotGravityComponent(gravity);`
	`std::cout << "**** Gravity Component ****" << std::endl;`
	`std::cout << "x = " << gravity.x << std::endl;`
	`std::cout << "y = " << gravity.y << std::endl;`
	`std::cout << "z = " << gravity.z << std::endl;`

输出	`**** Gravity Component ****`
	`x = 0.0670606`
	`y = -0.201182`
	`z = -9.80771`

94.获取当前碰撞等级 robotServiceGetRobotCollisionCurrentService

`int robotServiceGetRobotCollisionCurrentService(int &collisionGrade);`	
函数功能	获取碰撞等级
参数描述	collisionGrade：碰撞等级，输出参数 需连接真实机器人
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取碰撞等级*/`
	`int collisiongrade;`
	`robotService.robotServiceGetRobotCollisionCurrentService(collisiongrade);`

	`std::cout << "collision grade is : " << collisiongrade << std::endl;`
输出	`collision grade is : 6`

95. 设置机械臂碰撞等级 robotServiceSetRobotCollisionClass

`int robotServiceSetRobotCollisionClass(int grade);`	
函数功能	设置碰撞等级
参数描述	碰撞等级：0~10
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置机械臂碰撞等级*/`
	`ret = robotService.robotServiceSetRobotCollisionClass(6);`
	`std::cout << "set robot collision ret is " << ret << std::endl;`

输出	`set robot collision ret is 0`

96. 获取设备信息 robotServiceGetRobotDevInfoService

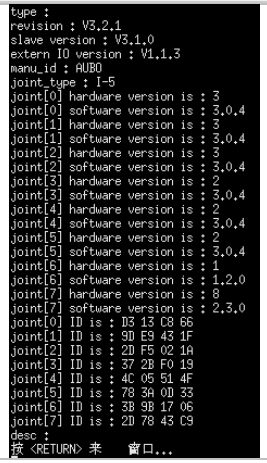
`int robotServiceGetRobotDevInfoService(aubo_robot_namespace::RobotDevInfo &devInfo);`	
函数功能	获取机器人设备信息（需连接真实机器人）
参数描述	devInfo：设备信息

```
typedef struct PACKED
{
    uint8 type;                         //设备型号、芯片型号：上位机主站：0x01  接口板 0x02
    char revision[16];                  //设备版本号，V1.0
    char manu_id[16];                   //厂家 ID，"OUR "的 ASCII 码 0x4F 55 52 00
    char joint_type[16];                //机械臂类型
    JointVersion joint_ver[8];          //机械臂关节及工具端信息
    char desc[64];                      //设备描述字符串以 0x00 结束
    JointProductID jointProductID[8];    //关节 ID 信息
    char slave_version[16];             //从设备版本号 - 字符串表示，如"V1.0.0
    char extio_version[16];             //IO 扩展板版本号 -字符串标志，如"V1.0.0
```

	}RobotDevInfo;
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    /*获取设备信息*/
    aubo_robot_namespace::RobotDevInfo robotdevinfo;
    robotService.robotServiceGetRobotDevInfoService(robotdevinfo);
    std::cout << "**** Robot Dev Info ****" << std::endl;
    std::cout << "type : " << robotdevinfo.type << std::endl;
    std::cout << "revision : " << robotdevinfo.revision << std::endl;
    std::cout << "slave version : " << robotdevinfo.slave_version << std::endl;
    std::cout << "extern IO version : " << robotdevinfo.extio_version << std::endl;
    std::cout << "manu_id : " << robotdevinfo.manu_id << std::endl;
    std::cout << "joint_type : " << robotdevinfo.joint_type << std::endl;
    for(int i = 0; i < 8; i++)
    {
        std::cout <<"joint[" << i << "] hardware version is : " << robotdevinfo.joint_ver[i].hw_version <<
std::endl;
        std::cout <<"joint[" << i << "] software version is : " << robotdevinfo.joint_ver[i].sw_version <<
std::endl;
    }
    for(int i = 0; i < 8; i++)
    {
        std::cout <<"joint[" << i << "] ID is : " << robotdevinfo.jointProductID[i].productID << std::endl;
    }
    std::cout << "desc : " << robotdevinfo.desc << std::endl;
```

输出



97.设置最大加速度 robotServiceSetRobotMaxACC

98.碰撞恢复 robotServiceCollisionRecover

99.获取机械臂当前运行状态 robotServiceGetRobotCurrentState

```
int robotServiceGetRobotCurrentState(aubo_robot_namespace::RobotState &state);
```

函数功能	获取机械臂当前运行状态
参数描述	state：运行状态，输出参数

	`/** 机械臂状态枚举  **/`
	`enum RobotState`
	`{`
	`  RobotStopped = 0,`
	`  RobotRunning,`
	`  RobotPaused,`
	`  RobotResumed`
	`};`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取机械臂当前运行状态*/`
	`aubo_robot_namespace::RobotState robotstate;`
	`robotService.robotServiceGetRobotCurrentState(robotstate);`
	`std::cout << "robot current state is : " << robotstate << std::endl;`

输出	robot current state is : 1

100. 获取 Mac 通信状态 robotServiceGetMacCommunicationStatus

int robotServiceGetMacCommunicationStatus(bool &value);	
函数功能	获取 MAC 通信状态
参数描述	value:连接状态，输出参数。true:连接成功 false：连接失败
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取 MAC 通信状态*/`
	`bool macconnectstate;`
	`robotService.robotServiceGetMacCommunicationStatus(macconnectstate);`
	`std::cout << "mac connect state is : " << macconnectstate << std::endl;`

输出	`mac connect state is : 1`

101. 判断真实机械臂是否存在 robotServiceGetIsRealRobotExist

int robotServiceGetIsRealRobotExist(bool &value);	
函数功能	获取是否存在真实机械臂
参数描述	为传出参数，true：存在真实机械臂　false：不存在真实机械臂
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`//判断真实臂是否存在`
	`bool isreal;`
	`robotService.robotServiceGetIsRealRobotExist(isreal);`
	`std::cout << "is real robot : " << isreal << std::endl;`

输出	`is real robot : 0`

102. 获取 Joint6 旋转 360°使能标志 robotServiceGetJoint6Rotate360EnableFlag

int robotServiceGetJoint6Rotate360EnableFlag(bool &value);	
函数功能	获取 6 关节旋转 360 使能标志

参数描述	value：J6 是否为 360 度版本。true：360 度版本；false:普通版本
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取 6 关节旋转 360 使能标志*/`
	`bool j6_360_flag;`
	`robotService.robotServiceGetJoint6Rotate360EnableFlag(j6_360_flag);`
	`std::cout << "joint 6 360 flag : " << j6_360_flag << std::endl;`

输出	joint 6 360 flag : 0

103. 获取机械臂关节状态 robotServiceGetRobotJointStatus

```
int robotServiceGetRobotJointStatus(aubo_robot_namespace::JointStatus *jointStatus, int size);
```

函数功能	获取机械臂关节状态

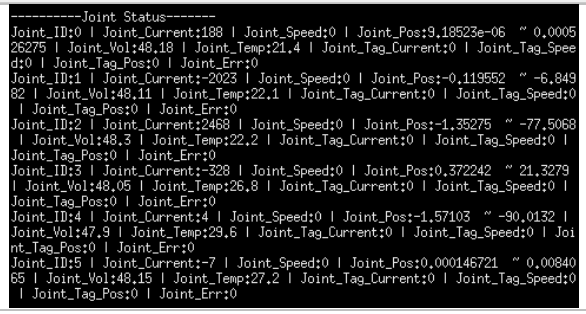
参数描述	jointStatus 关节状态缓冲区 输出参数；
	`typedef struct PACKED`
	`{`
	`  int  jointCurrentI;  /**< Current of driver  关节电流*/`
	`  int  jointSpeedMoto;  /**< Speed of driver  关节速度*/`
	`  float  jointPosJ;  /**< Current position in radian  关节角*/`
	`  float  jointCurVol;  /**< Rated voltage of motor. Unit: mV  关节电压*/`
	`  float  jointCurTemp;  /**< Current temprature of joint  当前温度*/`
	`  int  jointTagCurrentI;  /**< Target current of motor  电机目标电流*/`
	`  float  jointTagSpeedMoto;  /**< Target speed of motor  电机目标速度*/`
	`  float  jointTagPosJ;  /**< Target position of joint in radian 目标关节角 */`
	`  uint16 jointErrorNum;  /**< Joint error of joint num  关节错误码 */`
	`}JointStatus;`
	size 缓冲区大小，6

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取机械臂关节状态*/`
	`//缓冲区大小，自由度 6`
	`int size = 6;`
	`aubo_robot_namespace::JointStatus jointstatus[size];`
	`robotService.robotServiceGetRobotJointStatus(jointstatus,size);`
	`std::cout<<std::endl<<"----------Joint Status-------" << std::endl;`
	`for(int i=0; i<size; i++)`

```
    {
        std::cout<<"Joint_ID:"   <<i<<" | " ;
        std::cout<<"Joint_Current:"     <<jointstatus[i].jointCurrentI<<" | ";
        std::cout<<"Joint_Velc:"     <<jointstatus[i].jointSpeedMoto<<" | ";
        std::cout<<"Joint_Pos:"   <<jointstatus[i].jointPosJ<<" "<<" ~
"<<jointstatus[i].jointPosJ*180.0/3.1415926 << " | ";
        std::cout<<"Joint_Speed:"  <<jointstatus[i].jointCurVol<<" | ";
        std::cout<<"Joint_Temp:"  <<jointstatus[i].jointCurTemp<<" | ";
        std::cout<<"Joint_Tag_Current:"  <<jointstatus[i].jointTagCurrentI<<" | ";
        std::cout<<"Joint_Tag_Speed:" <<jointstatus[i].jointTagSpeedMoto<<" | ";
        std::cout<<"Joint_Tag_Pos:"  <<jointstatus[i].jointTagPosJ<<" | ";
        std::cout<<"Joint_Err:"  <<jointstatus[i].jointErrorNum <<std::endl;
    }
    std::cout<<std::endl;
```

输出



104. 获取机械臂诊断信息 robotServiceGetRobotDiagnosisInfo

```
int robotServiceGetRobotDiagnosisInfo(aubo_robot_namespace::RobotDiagnosis &robotDiagnosisInfo);
```

函数功能	获取机械臂诊断信息
参数描述	传出参数，机械臂诊断信息

```
/****机械臂诊断信息****/
typedef struct PACKED
{
    //CAN 通信状态:0x01~0x80：关节 CAN 通信错误（每个关节占用 1bit）
    //0x00：无错误 0xff：CAN 总线存在错误
    uint8 armCanbusStatus;
    float armPowerCurrent;    //机械臂 48V 电源当前电流
```

```
    float armPowerVoltage;    //机械臂48V电源当前电压
    bool  armPowerStatus;     //机械臂48V电源状态（开、关）
    char  contorllerTemp;     //控制箱温度
    uint8 contorllerHumidity; //控制箱湿度
    bool  remoteHalt;         //远程关机信号
    bool  softEmergency;      //机械臂软急停
    bool  remoteEmergency;    //远程急停信号
    bool  robotCollision;     //碰撞检测位
    bool  forceControlMode;   //机械臂进入力控模式标志位
    bool brakeStuats;         //刹车状态
    float robotEndSpeed;      //末端速度
    int robotMaxAcc;          //最大加速度
    bool orpeStatus;          //上位机软件状态位
    bool enableReadPose;      //位姿读取使能位
    bool robotMountingPoseChanged;    //安装位置状态
    bool encoderErrorStatus;          //磁编码器错误状态
    bool staticCollisionDetect;       //静止碰撞检测开关
    uint8 jointCollisionDetect;       //关节碰撞检测 每个关节占用1bit 0-无碰撞 1-存在碰撞
    bool encoderLinesError;           //光电编码器不一致错误 0-无错误 1-有错误
    bool jointErrorStatus;            //joint error status
    bool singularityOverSpeedAlarm;   //机械臂奇异点过速警告
    bool robotCurrentAlarm;           //机械臂电流错误警告
    uint8 toolIoError;                //tool error
    bool robotMountingPoseWarning;    //机械臂安装位置错位（只在力控模式下起作用）
    uint16 macTargetPosBufferSize;    //mac缓冲器长度
    uint16 macTargetPosDataSize;      //mac缓冲器有效数据长度
    uint8  macDataInterruptWarning;   //mac数据中断

}RobotDiagnosis;
```

返回值

调用成功返回 ErrnoSucc;错误返回错误号

示例

```
    /*获取机械臂诊断信息*/
    aubo_robot_namespace::RobotDiagnosis robotdiagnosis;
    robotService.robotServiceGetRobotDiagnosisInfo(robotdiagnosis);
    std::cout << "arm_Canbus_Status: " << robotdiagnosis.armCanbusStatus << std::endl;
    std::cout << "arm_Power_Current: " << robotdiagnosis.armPowerCurrent << std::endl;
    std::cout << "arm_Power_Status: " << robotdiagnosis.armPowerStatus << std::endl;
    std::cout << "arm_Power_Voltage: " << robotdiagnosis.armPowerVoltage << std::endl;
    std::cout << "brake_Status: " << robotdiagnosis.brakeStuats << std::endl;
    std::cout << "controller_Humidity: " << robotdiagnosis.contorllerHumidity << std::endl;
```

```
    std::cout << "controller_Temprature: " << robotdiagnosis.contorllerTemp << std::endl;
    std::cout << "encoderErrorStatus: " << robotdiagnosis.encoderErrorStatus << std::endl;
    std::cout << "encoderLinesError: " << robotdiagnosis.encoderLinesError << std::endl;
    std::cout << "forceControlMode: " << robotdiagnosis.forceControlMode << std::endl;
    std::cout << "jointCollisionDetect: " << robotdiagnosis.jointCollisionDetect << std::endl;
    std::cout << "jointErrorStatus: " << robotdiagnosis.jointErrorStatus << std::endl;
    std::cout << "macDataInterruptWarning: " << robotdiagnosis.macDataInterruptWarning << std::endl;
    std::cout << "macTargetPosBufferSize: " << robotdiagnosis.macTargetPosBufferSize << std::endl;
    std::cout << "macTargetPosDataSize: " << robotdiagnosis.macTargetPosDataSize << std::endl;
    std::cout << "orpeStatus: " << robotdiagnosis.orpeStatus << std::endl;
    std::cout << "remoteEmergency: " << robotdiagnosis.remoteEmergency << std::endl;
    std::cout << "remoteHalt: " << robotdiagnosis.remoteHalt << std::endl;
    std::cout << "robotCollision: " << robotdiagnosis.robotCollision << std::endl;
    std::cout << "robotCurrentAlarm: " << robotdiagnosis.robotCurrentAlarm << std::endl;
    std::cout << "robotEndSpeed: " << robotdiagnosis.robotEndSpeed << std::endl;
    std::cout << "robotMaxAcc: " << robotdiagnosis.robotMaxAcc << std::endl;
    std::cout << "robotMountingPoseChanged: " << robotdiagnosis.robotMountingPoseChanged << std::endl;
    std::cout << "robotMountingPoseWarning: " << robotdiagnosis.robotMountingPoseWarning << std::endl;
    std::cout << "singularityOverSpeedAlarm: " << robotdiagnosis.singularityOverSpeedAlarm << std::endl;
    std::cout << "softEmergency: " << robotdiagnosis.softEmergency << std::endl;
    std::cout << "staticCollisionDetect: " << robotdiagnosis.staticCollisionDetect << std::endl;
    std::cout << "toolIoError: " << robotdiagnosis.toolIoError << std::endl;
```

输出

```
arm_Canbus_Status:
arm_Power_Current: 0.8
arm_Power_Status: 1
arm_Power_Voltage: 0
brake_Status: 0
controller_Humidity: 0
controller_Temprature:
encoderErrorStatus: 0
encoderLinesError: 0
forceControlMode: 0
jointCollisionDetect:
jointErrorStatus: 0
macDataInterruptWarning:
macTargetPosBufferSize: 1200
macTargetPosDataSize: 0
orpeStatus: 0
remoteEmergency: 0
remoteHalt: 1
robotCollision: 0
robotCurrentAlarm: 0
robotEndSpeed: 0
robotMaxAcc: 1000
robotMountingPoseChanged: 0
robotMountingPoseWarning: 0
singularityOverSpeedAlarm: 0
softEmergency: 0
staticCollisionDetect: 0
toolIoError:
```

105. 获取机械臂当前关节角信息 robotServiceGetJointAngleInfo

```
int robotServiceGetJointAngleInfo(aubo_robot_namespace::JointParam &jointParam);
```

函数功能	获取机械臂当前关节角信息
参数描述	jointParam：路点信息，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    /*获取机械臂当前关节角信息*/
    aubo_robot_namespace::JointParam jointangle;
    robotService.robotServiceGetJointAngleInfo(jointangle);
    //关节信息
    std::cout<<"Joint angle : "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout << "joint" << i+1 << ": "<< jointangle.jointPos[i] << " ~ " <<
jointangle.jointPos[i]*180.0/M_PI << std::endl;
    }
```

输出

```
Joint angle :
joint1: 0.72785 ~ 41.7027
joint2: 0.257306 ~ 14.7425
joint3: -0.983386 ~ -56.3439
joint4: 0.269648 ~ 15.4497
joint5: -1.471 ~ -84.2821
joint6: -5.50204e-05 ~ -0.00315244
```

106. 获取实时路点信息 robotServiceGetCurrentWaypointInfo

```
int robotServiceGetCurrentWaypointInfo(aubo_robot_namespace::wayPoint_S &wayPoint);
```

函数功能	获取机械臂当前路点信息
参数描述	为传出参数，路点信息
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	

```
    /*获取实时路点信息*/
    aubo_robot_namespace::wayPoint_S wayPoint;
    robotService.robotServiceGetCurrentWaypointInfo(wayPoint);
    std::cout<<std::endl<<"-------------Current Waypoint---------------"<<std::endl;
```

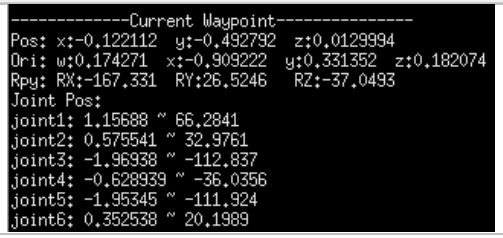
```cpp
    //位置信息
    std::cout<<"Pos: ";
    std::cout<<"x:"<<wayPoint.cartPos.position.x<<"  ";
    std::cout<<"y:"<<wayPoint.cartPos.position.y<<"  ";
    std::cout<<"z:"<<wayPoint.cartPos.position.z<<std::endl;

    //姿态信息
    std::cout<<"Ori: ";
    std::cout<<"w:"<<wayPoint.orientation.w<<"  ";
    std::cout<<"x:"<<wayPoint.orientation.x<<"  ";
    std::cout<<"y:"<<wayPoint.orientation.y<<"  ";
    std::cout<<"z:"<<wayPoint.orientation.z<<std::endl;

    aubo_robot_namespace::Rpy tempRpy;
    robotService.quaternionToRPY(wayPoint.orientation,tempRpy);
    std::cout<<"Rpy: ";
    std::cout<<"RX:"<<tempRpy.rx*180.0/M_PI<<"  RY:"<<tempRpy.ry*180.0/M_PI
                  <<"   RZ:"<<tempRpy.rz*180.0/M_PI<<std::endl;

    //关节信息
    std::cout<<"Joint Pos: "<<std::endl;
    for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
    {
        std::cout<<"joint"<<i+1<<": "<<wayPoint.jointpos[i]<<" ~ "<<wayPoint.jointpos[i]*180.0/M_PI<<std::endl;
    }
```

输出



安全 IO 相关

107. 使机械臂回初始位 robotServiceSetRobotAtOriginPose

108. 通知接口板上位机暂停状态 robotServiceSetRobotOrpePause

109. 通知接口板上位机停止状态 robotServiceSetRobotOrpeStop

110. 通知接口板上位机错误 robotServiceSetRobotOrpeError

111. 解除系统紧急停止输出信号 robotServiceClearSystemEmergencyStop

112. 解除缩减模式错误 robotServiceClearReducedModeError

113. 防护重置成功 robotServiceRobotSafetyguardResetSucc

接口板 IO

114. 获取接口板 IO 配置信息 robotServiceGetBoardIOConfig

```
int robotServiceGetBoardIOConfig(const std::vector<aubo_robot_namespace::RobotIoType> &ioType,
std::vector<aubo_robot_namespace::RobotIoDesc> &configVector);
```

函数功能	获取接口板指定 IO 集合的配置信息
参数描述	ioType：IO 类型的集合，输入参数 configVector：IO 配置信息的集合，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例 1	打印 U_DI 的配置信息 /*获取接口板 IO 配置信息*/

```
    std::vector<aubo_robot_namespace::RobotIoType> ioTypeVector;
    std::vector<aubo_robot_namespace::RobotIoDesc> ioDescVector;
    //打印User_DI 配置信息
    ioTypeVector.push_back(aubo_robot_namespace::RobotBoardUserDI);
    robotService.robotServiceGetBoardIOConfig(ioTypeVector,ioDescVector);
    std::cout << "ioTypeVector lenth = " << ioTypeVector.size() << std::endl;
    std::cout << "ioDescVector lenth = " << ioDescVector.size() << std::endl;
    for(int i = 0; i < ioDescVector.size(); i++)
    {
        std::cout << "U_DO_" << i << std::endl;
        std::cout << "ioID = " << ioDescVector[i].ioId << " | ";
        std::cout << "ioType = " << ioDescVector[i].ioType << " | ";
        std::cout << "ioName = " << ioDescVector[i].ioName << " | ";
        std::cout << "ioAddr = " << ioDescVector[i].ioAddr << " | ";
        std::cout << "ioValue = " << ioDescVector[i].ioValue << std::endl;
    }
```

输出

```
ioTypeVector lenth = 1
ioDescVector lenth = 22
U_DO_0
ioID = F1 | ioType = 4 | ioName = F1 | ioAddr = 30 | ioValue = -1
U_DO_1
ioID = F2 | ioType = 4 | ioName = F2 | ioAddr = 31 | ioValue = -1
U_DO_2
ioID = F3 | ioType = 4 | ioName = F3 | ioAddr = 32 | ioValue = -1
U_DO_3
ioID = F4 | ioType = 4 | ioName = F4 | ioAddr = 33 | ioValue = -1
U_DO_4
ioID = F5 | ioType = 4 | ioName = F5 | ioAddr = 34 | ioValue = -1
U_DO_5
ioID = F6 | ioType = 4 | ioName = F6 | ioAddr = 35 | ioValue = -1
U_DO_6
ioID = U_DI_00 | ioType = 4 | ioName = U_DI_00 | ioAddr = 36 | ioValue = -1
U_DO_7
ioID = U_DI_01 | ioType = 4 | ioName = U_DI_01 | ioAddr = 37 | ioValue = -1
U_DO_8
ioID = U_DI_02 | ioType = 4 | ioName = U_DI_02 | ioAddr = 38 | ioValue = -1
U_DO_9
ioID = U_DI_03 | ioType = 4 | ioName = U_DI_03 | ioAddr = 39 | ioValue = -1
```

示例 2

打印 U_DI 和 U_DO 的配置信息

```
    /*获取接口板 IO 配置信息*/
    std::vector<aubo_robot_namespace::RobotIoType> ioTypeVector;
    std::vector<aubo_robot_namespace::RobotIoDesc> ioDescVector;
    //打印User_DI 配置信息
    ioTypeVector.push_back(aubo_robot_namespace::RobotBoardUserDI);
    robotService.robotServiceGetBoardIOConfig(ioTypeVector,ioDescVector);
    std::cout << "ioTypeVector lenth = " << ioTypeVector.size() << std::endl;
    std::cout << "ioDescVector lenth = " << ioDescVector.size() << std::endl;
    for(int i = 0; i < ioDescVector.size(); i++)
    {
        std::cout << "U_DO_" << i << std::endl;
        std::cout << "ioID = " << ioDescVector[i].ioId << " | ";
        std::cout << "ioType = " << ioDescVector[i].ioType << " | ";
        std::cout << "ioName = " << ioDescVector[i].ioName << " | ";
```

```
        std::cout << "ioAddr = " << ioDescVector[i].ioAddr << " | ";
        std::cout << "ioValue = " << ioDescVector[i].ioValue << std::endl;
    }
    //打印User_DO配置信息
    ioTypeVector.pop_back();
    ioTypeVector.push_back(aubo_robot_namespace::RobotBoardUserDO);
    robotService.robotServiceGetBoardIOConfig(ioTypeVector,ioDescVector);
    std::cout << "ioTypeVector lenth = " << ioTypeVector.size() << std::endl;
    std::cout << "ioDescVector lenth = " << ioDescVector.size() << std::endl;
    for(int i = 0; i < ioDescVector.size(); i++)
    {
        std::cout << "U_DO_" << i << std::endl;
        std::cout << "ioID = " << ioDescVector[i].ioId << " | ";
        std::cout << "ioType = " << ioDescVector[i].ioType << " | ";
        std::cout << "ioName = " << ioDescVector[i].ioName << " | ";
        std::cout << "ioAddr = " << ioDescVector[i].ioAddr << " | ";
        std::cout << "ioValue = " << ioDescVector[i].ioValue << std::endl;
    }
```

输出

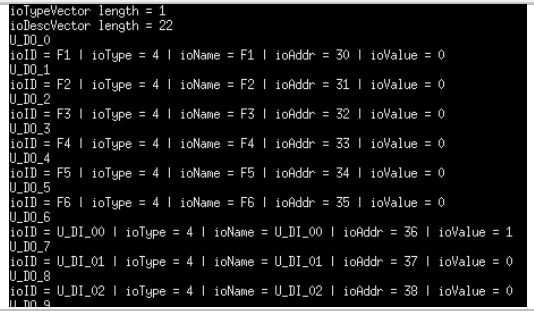
```
ioTypeVector lenth = 1
ioDescVector lenth = 16
U_DO_0
ioID = U_DO_00 | ioType = 5 | ioName = U_DO_00 | ioAddr = 32 | ioValue = -1
U_DO_1
ioID = U_DO_01 | ioType = 5 | ioName = U_DO_01 | ioAddr = 33 | ioValue = -1
U_DO_2
ioID = U_DO_02 | ioType = 5 | ioName = U_DO_02 | ioAddr = 34 | ioValue = -1
U_DO_3
ioID = U_DO_03 | ioType = 5 | ioName = U_DO_03 | ioAddr = 35 | ioValue = -1
U_DO_4
ioID = U_DO_04 | ioType = 5 | ioName = U_DO_04 | ioAddr = 36 | ioValue = -1
U_DO_5
ioID = U_DO_05 | ioType = 5 | ioName = U_DO_05 | ioAddr = 37 | ioValue = -1
U_DO_6
ioID = U_DO_06 | ioType = 5 | ioName = U_DO_06 | ioAddr = 38 | ioValue = -1
U_DO_7
ioID = U_DO_07 | ioType = 5 | ioName = U_DO_07 | ioAddr = 39 | ioValue = -1
U_DO_8
ioID = U_DO_10 | ioType = 5 | ioName = U_DO_10 | ioAddr = 40 | ioValue = -1
```

115. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus

```
int robotServiceGetBoardIOStatus(const std::vector<aubo_robot_namespace::RobotIoType>  ioType,
std::vector<aubo_robot_namespace::RobotIoDesc> &statusVector);
```

函数功能	获取接口板指定 IO 集合的状态信息
参数描述	ioType：IO 类型的集合，输入参数 configVector：IO 配置信息的集合，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```
	/*获取接口板 UserDI 状态信息*/
	std::vector<aubo_robot_namespace::RobotIoType> ioTypeVector;
	std::vector<aubo_robot_namespace::RobotIoDesc> ioDescVector;
	ioTypeVector.push_back(aubo_robot_namespace::RobotBoardUserDI);
	
	robotService.robotServiceGetBoardIOStatus(ioTypeVector,ioDescVector);
	std::cout << "ioTypeVector length = " << ioTypeVector.size() << std::endl;
	std::cout << "ioDescVector length = " << ioDescVector.size() << std::endl;
	for(int i = 0; i < ioDescVector.size(); i++)
	{
	    std::cout << "U_DO_" << i << std::endl;
	    std::cout << "ioID = " << ioDescVector[i].ioId << "
		";
		    std::cout << "ioType = " << ioDescVector[i].ioType << "
			";
			    std::cout << "ioName = " << ioDescVector[i].ioName << "
				";
				    std::cout << "ioAddr = " << ioDescVector[i].ioAddr << "
					";
					    std::cout << "ioValue = " << ioDescVector[i].ioValue << std::endl;
					}
					``

输出	

116. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus

```int robotServiceGetBoardIOStatus(aubo_robot_namespace::RobotIoType type, std::string name, double &value);```	
函数功能	根据接口板 IO 类型和名称获取 IO 状态
参数描述	type　　IO 类型 name　　IO 名称 value　　IO 状态，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	``` /*获取指定接口板 IO 的状态*/ ```

	``
	//获取U_DI_00的状态
	aubo_robot_namespace::RobotIoType iotype = aubo_robot_namespace::RobotBoardUserDI;
	std::string ioname = "U_DI_00";
	double iovalue;
	robotService.robotServiceGetBoardIOStatus(iotype,ioname,iovalue);
	std::cout << ioname << " = " << iovalue << std::endl;
	//获取U_DO_00的状态
	iotype = aubo_robot_namespace::RobotBoardUserDO;
	ioname = "U_DO_00";
	robotService.robotServiceGetBoardIOStatus(iotype,ioname,iovalue);
	std::cout << ioname << " = " << iovalue << std::endl;
	``

输出	``` U_DI_00 = 1 U_DO_00 = 1 ```

117. 获取接口板 IO 状态信息 robotServiceGetBoardIOStatus

```int robotServiceGetBoardIOStatus(aubo_robot_namespace::RobotIoType type, int addr, double &value);```	
函数功能	根据接口板 IO 类型和名称获取 IO 状态
参数描述	type IO 类型 addr IO 地址 F1:30 ~ F6:35 U_DI_00:36 ~ U_DI_17:51 U_DO_00:32 ~ U_DO_17:47 value IO 状态，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```
	/*获取指定接口板IO的状态*/
	//获取U_DI_00的状态
	aubo_robot_namespace::RobotIoType iotype = aubo_robot_namespace::RobotBoardUserDI;
	int ioaddr = 36;
	double iovalue;
	robotService.robotServiceGetBoardIOStatus(iotype,ioaddr,iovalue);
	std::cout << "addr " << ioaddr << " = " << iovalue << std::endl;
	//获取U_DO_00的状态
	iotype = aubo_robot_namespace::RobotBoardUserDO;
	ioaddr = 32;
	robotService.robotServiceGetBoardIOStatus(iotype,ioaddr,iovalue);
	``

	`std::cout << "addr " << ioaddr << " = " << iovalue << std::endl;`
输出	`addr 36 = 1` `addr 32 = 1`

118. 设置接口板 IO 状态 robotServiceSetBoardIOStatus

`int robotServiceSetBoardIOStatus(aubo_robot_namespace::RobotIoType type, std::string name, double value);`	
函数功能	根据接口板 IO 类型和名称设置 IO 状态
参数描述	type IO 类型 name IO 名称 value IO 状态，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置接口板 IO 状态*/`
	`aubo_robot_namespace::RobotIoType iotype = aubo_robot_namespace::RobotBoardUserDO;`
	`std::string ioname = "U_DO_00";`
	`double iovalue = 1;`
	`ret = robotService.robotServiceSetBoardIOStatus(iotype,ioname,iovalue);`
	`if(ret != aubo_robot_namespace::InterfaceCallSuccCode)`
	`{`
	`std::cerr << "set U_DO failed!" << std::endl;`
	`}`

输出	

119. 设置接口板 IO 状态 robotServiceSetBoardIOStatus

`int robotServiceSetBoardIOStatus(aubo_robot_namespace::RobotIoType type, int addr, double value);`	
函数功能	根据接口板 IO 类型和地址设置 IO 状态
参数描述	type IO 类型 addr IO 地址 F1:30 ~ F6:35 U_DI_00:36 ~ U_DI_17:51 U_DO_00:32 ~ U_DO_17:47

	value　IO 状态，输出参数
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置接口板 IO 状态*/`
	`aubo_robot_namespace::RobotIoType iotype = aubo_robot_namespace::RobotBoardUserDO;`
	`int ioaddr = 32;`
	`double iovalue = 0;`
	`ret = robotService.robotServiceSetBoardIOStatus(iotype,ioaddr,iovalue);`
	`if(ret != aubo_robot_namespace::InterfaceCallSuccCode)`
	`{`
	`std::cerr << "set U_DO failed!" << std::endl;`
	`}`

输出	

查看联机模式

120.　是否在联机模式 robotServiceIsOnlineMode

`int robotServiceIsOnlineMode(bool &isOnlineMode);`	
函数功能	返回当前机械臂是否运行在联机模式
参数描述	isOnlineMode 输出参数　true：联机 false：非联机
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*是否在联机模式*/`
	`bool isonlinemode;`
	`robotService.robotServiceIsOnlineMode(isonlinemode);`
	`std::cout << "robot is online mode : " << isonlinemode << std::endl;`

输出	`robot is online mode : 0`

121.　是否在联机主模式 robotServiceIsOnlineMasterMode

`int robotServiceIsOnlineMasterMode(bool &isOnlineMasterMode);`	
函数功能	返回当前机械臂是否运行在联机主模式

参数描述	isOnlineMode 输出参数 true：联机主模式/手动模式 false：联机从模式
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*是否在联机主模式*/`
	`bool isonlinemastermode;`
	`robotService.robotServiceIsOnlineMasterMode(isonlinemastermode);`
	`std::cout << "robot is online master mode : " << isonlinemastermode << std::endl;`

输出	`robot is online master mode : 1`

安全配置

122. 获取机械臂安全配置 robotServiceGetRobotSafetyConfig

`int robotServiceGetRobotSafetyConfig(aubo_robot_namespace::RobotSafetyConfig &safetyConfig);`

函数功能	获取机械臂安全配置

参数描述	safetyConfig：安全配置。
	`typedef struct PACKED`
	`{`
	`  uint16 robotReducedConfigJointSpeed[6];  //缩减配置 关节速度限制`
	`  uint32 robotReducedConfigTcpSpeed;  //缩减配置 TCP 速度限制`
	`  uint32 robotReducedConfigTcpForce;  //缩减配置 TCP 力（暂定为碰撞等级）`
	`  uint32 robotReducedConfigMomentum;  //缩减配置 动量`
	`  uint32 robotReducedConfigPower;  //缩减配置 功率`
	`  uint8 robotSafeguradResetConfig;  //防护重 置设置`
	`  uint8 robotOperationalModeConfig;  //操作模式设置`
	`}RobotSafetyConfig;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*获取机械臂安全配置*/`
	`aubo_robot_namespace::RobotSafetyConfig safeconfig;`
	`robotService.robotServiceGetRobotSafetyConfig(safeconfig);`
	`for(int i = 0; i < 6 ;i++)`
	`{`
	`  std::cout << "joint[" << i << "] speed = " << safeconfig.robotReducedConfigJointSpeed[i] << std::endl;`
	`}`
	`std::cout << "robotReducedConfigTcpSpeed = " << safeconfig.robotReducedConfigTcpSpeed << std::endl;`
	`std::cout << "robotReducedConfigTcpForce = " << safeconfig.robotReducedConfigTcpForce << std::endl;`

	std::cout << "robotReducedConfigMomentum = " << safeconfig.robotReducedConfigMomentum << std::endl;
	std::cout << "robotReducedConfigPower = " << safeconfig.robotReducedConfigPower << std::endl;
	std::cout << "robotSafeguradResetConfig = " << safeconfig.robotSafeguradResetConfig << std::endl;
	std::cout << "robotOperationalModeConfig = " << safeconfig.robotOperationalModeConfig << std::endl;

输出	joint[0] speed = 15 joint[1] speed = 15 joint[2] speed = 15 joint[3] speed = 15 joint[4] speed = 15 joint[5] speed = 15 robotReducedConfigTcpSpeed = 160 robotReducedConfigTcpForce = 1 robotReducedConfigMomentum = 0 robotReducedConfigPower = 0 robotSafeguradResetConfig = robotOperationalModeConfig =

123. 设置机械臂安全配置 robotServiceSetRobotSafetyConfig

124. 获取机械臂安全状态 robotServiceGetOrpeSafetyStatus

工具 IO 接口

125. 设置工具端电源电压类型 robotServiceSetToolPowerVoltageType

`int robotServiceSetToolPowerVoltageType (aubo_robot_namespace::ToolPowerType type);`	
函数功能	设置工具端电源电压类型
参数描述	type：工具电源类型。

	`typedef enum`
	`{`
	`  OUT_0V  = 0,`
	`  OUT_12V = 1,`
	`  OUT_24V = 2`
	`}ToolPowerType;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```/*设置工具电源电压类型*/
	aubo_robot_namespace::ToolPowerType m_set_toolpowertype;
	m_set_toolpowertype = aubo_robot_namespace::OUT_24V;
	robotService.robotServiceSetToolPowerVoltageType(m_set_toolpowertype);```

输出	

126. 获取工具端电源电压类型 robotServiceGetToolPowerVoltageType

```
int robotServiceGetToolPowerVoltageType  (aubo_robot_namespace::ToolPowerType &type);
```

函数功能	获取工具端电源电压类型
参数描述	type：工具电源类型，输出参数

	``typedef enum
	{
	OUT_0V = 0,
	OUT_12V = 1,
	OUT_24V = 2
	}ToolPowerType;```

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	```/*获取工具电源电压类型*/
	aubo_robot_namespace::ToolPowerType m_get_toolpowertype;
	robotService.robotServiceGetToolPowerVoltageType(m_get_toolpowertype);
	std::cout << "tool power voltage type is : " << m_get_toolpowertype << std::endl;```

输出	```tool power voltage type is : 2```

127. 获取工具端的电源电压 robotServiceGetToolPowerVoltageStatus

```
int robotServiceGetToolPowerVoltageStatus(double &value);
```

函数功能	获取工具端电源电压
参数描述	
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	
输出	

128. 设置工具端电源电压类型 and 所有数字量 IO 的类型 robotServiceSetToolPowerTypeAndDigitalIOType

```
int robotServiceSetToolPowerTypeAndDigitalIOType(aubo_robot_namespace::ToolPowerType type,
                                                 aubo_robot_namespace::ToolIOType io0,
                                                 aubo_robot_namespace::ToolIOType io1,
                                                 aubo_robot_namespace::ToolIOType io2,
                                                 aubo_robot_namespace::ToolIOType io3);
```

函数功能	设置工具端电源电压类型 and 所有数字量 IO 的类型

参数描述	参数 1：type，电源电压类型
	`typedef enum`
	`{`
	`  OUT_0V  = 0,`
	`  OUT_12V = 1,`
	`  OUT_24V = 2`
	`}ToolPowerType;`
	参数 2：io0，Tool IO 0 的类型。
	`typedef enum  //IO类型`
	`{  IO_IN = 0,  //输入`
	`  IO_OUT  //输出`
	`}ToolIOType;`
	参数 3：io1，Tool IO 1 的类型。
	参数 4：io2，Tool IO 2 的类型。
	参数 5：io3，Tool IO 3 的类型。

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*设置工具电源电压类型及所有数字量 IO 类型*/`
	`aubo_robot_namespace::ToolPowerType toolpowertype = aubo_robot_namespace::OUT_24V;`
	`aubo_robot_namespace::ToolIOType toolDIO_0type = aubo_robot_namespace::IO_IN;`
	`aubo_robot_namespace::ToolIOType toolDIO_1type = aubo_robot_namespace::IO_IN;`
	`aubo_robot_namespace::ToolIOType toolDIO_2type = aubo_robot_namespace::IO_OUT;`
	`aubo_robot_namespace::ToolIOType toolDIO_3type =aubo_robot_namespace::IO_OUT ;`
	`ret = robotService.robotServiceSetToolPowerTypeAndDigitalIOType(toolpowertype,`
	`  toolDIO_0type,`
	`  toolDIO_1type,`

	toolDIO_2type, toolDIO_3type); std::cout << "set tool IO type ret is : " << ret << std::endl;
输出	set tool IO type ret is : 0

129. 设置工具端数字量 IO 的类型 robotServiceSetToolDigitalIOType

int robotServiceSetToolDigitalIOType(aubo_robot_namespace::ToolDigitalIOAddr addr, aubo_robot_namespace::ToolIOType type);	
函数功能	设置工具端数字量 IO 的类型
参数描述	addr：IO 地址。
	typedef enum { TOOL_DIGITAL_IO_0 = 0, TOOL_DIGITAL_IO_1 = 1, TOOL_DIGITAL_IO_2 = 2, TOOL_DIGITAL_IO_3 = 3 }ToolDigitalIOAddr; type：类型。
	typedef enum //IO类型 { IO_IN = 0, //输入 IO_OUT //输出 }ToolIOType;
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	/*设置工具数字 IO 类型*/
	aubo_robot_namespace::ToolDigitalIOAddr toolIOaddr = aubo_robot_namespace::TOOL_DIGITAL_IO_0;
	aubo_robot_namespace::ToolIOType toolIOtype = aubo_robot_namespace::IO_IN;
	ret = robotService.robotServiceSetToolDigitalIOType(toolIOaddr,toolIOtype);
	std::cout << "set tool digital io type ret is : " << ret << std::endl;

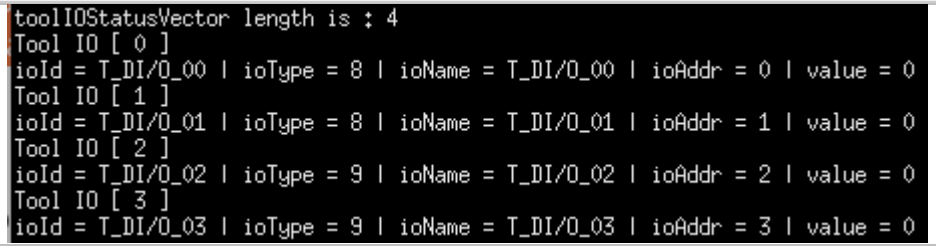
输出	set tool IO type ret is : 0

130. 获取工具端所有数字量 IO 的状态 robotServiceGetAllToolDigitalIOStatus

```
int robotServiceGetAllToolDigitalIOStatus(std::vector<aubo_robot_namespace::RobotIoDesc> &statusVector);
```

函数功能	获取工具端所有数字量 IO 的状态

参数描述	statusVector：IO 状态容器。输出参数。
	`typedef struct PACKED`
	`{`
	`  char  ioId[32];  //IO-ID 目前未使用`
	`  RobotIoType ioType;  //IO 类型`
	`  char  ioName[32];  //IO 名称`
	`  int  ioAddr;  //IO 地址`
	`  double  ioValue;  //IO 状态`
	`}RobotIoDesc;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	`/*获取所有工具端数字 IO 类型*/` `std::vector<aubo_robot_namespace::RobotIoDesc> toolIOStatusVector;` `robotService.robotServiceGetAllToolDigitalIOStatus(toolIOStatusVector);` `std::cout << "toolIOStatusVector length is : " << toolIOStatusVector.size() << std::endl;` `for(int i = 0; i < toolIOStatusVector.size(); i++)` `{` `  std::cout << "Tool IO [ " << i << " ]" << std::endl;` ` std::cout << "ioId = " << toolIOStatusVector[i].ioId << "	";` ` std::cout << "ioType = " << toolIOStatusVector[i].ioType << "	";` ` std::cout << "ioName = " << toolIOStatusVector[i].ioName << "	";` ` std::cout << "ioAddr = " << toolIOStatusVector[i].ioAddr << "	";` ` std::cout << "value = " << toolIOStatusVector[i].ioValue << std::endl;` `}`
输出	

131. 根据地址设置工具端数字量 IO 的状态 robotServiceSetToolDOStatus

```
int robotServiceSetToolDOStatus          (aubo_robot_namespace::ToolDigitalIOAddr addr, aubo_robot_namespace::IO_STATUS value);
```

函数功能	根据地址设置工具端数字量 IO 的状态
参数描述	addr：IO 地址。

	typedef enum
	{
	TOOL_DIGITAL_IO_0 = 0,
	TOOL_DIGITAL_IO_1 = 1,
	TOOL_DIGITAL_IO_2 = 2,
	TOOL_DIGITAL_IO_3 = 3
	
	}ToolDigitalIOAddr;
	value：IO 状态值。
	typedef enum //IO状态
	{
	IO_STATUS_INVALID = 0, //无效
	IO_STATUS_VALID //有效
	}IO_STATUS;

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	/*根据地址设置工具 IO 状态*/ robotService.robotServiceSetToolDOStatus(aubo_robot_namespace::TOOL_DIGITAL_IO_2,aubo_robot_namespace::IO_STATUS_VALID);
输出	

132. 根据名称设置工具端数字量 IO 的状态 robotServiceSetToolDOStatus

```
int robotServiceSetToolDOStatus(std::string name,       aubo_robot_namespace::IO_STATUS value);
```

函数功能	根据名称设置工具端数字量 IO 的状态
参数描述	name:IO 名称。

	T_DI/O_00
	T_DI/O_01
	T_DI/O_02
	T_DI/O_03
	T_AI_00
	T_AI_01
	value：IO 状态值。
	`typedef enum //ＩＯ状态`
	`{`
	`IO_STATUS_INVALID = 0, //无效`
	`IO_STATUS_VALID //有效`
	`}IO_STATUS;`

返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	`/*根据名称设置工具 IO 状态*/`
	`std::string ioname = "T_DI/O_02";`
	`robotService.robotServiceSetToolDOStatus(ioname,aubo_robot_namespace::IO_STATUS_INVALID);`

输出	

133. 根据名称获取工具端 IO 的状态 robotServiceGetToolIoStatus

`int robotServiceGetToolIoStatus(std::string name, double &value);`	
函数功能	根据名称获取工具端 IO 的状态
参数描述	name:IO 名称。

	T_DI/O_00
	T_DI/O_01
	T_DI/O_02
	T_DI/O_03
	T_AI_00
	T_AI_01
	value：IO 状态值。输出参数。
	`typedef enum //ＩＯ状态`
	`{`
	`IO_STATUS_INVALID = 0, //无效`
	`IO_STATUS_VALID //有效`

	}IO_STATUS;
返回值	调用成功返回 ErrnoSucc;错误返回错误号

示例	/*根据名称获取工具 IO 状态*/
	double d0_value;
	double d1_value;
	double d2_value;
	double d3_value;
	double a0_value;
	double a1_value;
	robotService.robotServiceGetToolIoStatus("T_DI/O_00",d0_value);
	robotService.robotServiceGetToolIoStatus("T_DI/O_01",d1_value);
	robotService.robotServiceGetToolIoStatus("T_DI/O_02",d2_value);
	robotService.robotServiceGetToolIoStatus("T_DI/O_03",d3_value);
	robotService.robotServiceGetToolIoStatus("T_AI_00",a0_value);
	robotService.robotServiceGetToolIoStatus("T_AI_01",a1_value);

输出	D_0 : 0 | D_1 : 0 | D_2 : 1 | D_3 : 0 | A_0 : 4.965 | A_1 : 4.97

134. 获取工具端所有 AI 的状态 robotServiceGetAllToolAIStatus

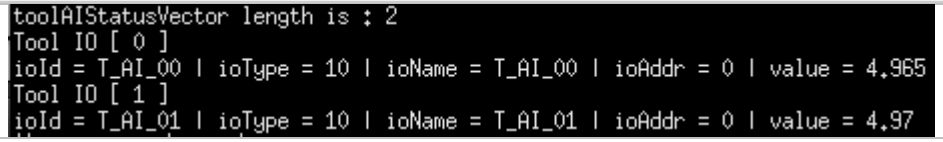
int robotServiceGetAllToolAIStatus (std::vector<aubo_robot_namespace::RobotIoDesc> &statusVector);	
函数功能	获取工具端所有 AI 的状态

参数描述	statusVector：Tool AI 容器，输出参数。
	typedef struct PACKED
	{
	char ioId[32]; //IO-ID 目前未使用
	RobotIoType ioType; //IO 类型
	char ioName[32]; //IO 名称
	int ioAddr; //IO 地址
	double ioValue; //IO 状态
	}RobotIoDesc;

返回值	调用成功返回 ErrnoSucc;错误返回错误号
示例	/*获取工具端所有 AI 的状态*/ std::vector<aubo_robot_namespace::RobotIoDesc> toolAIStatusVector; robotService.robotServiceGetAllToolAIStatus(toolAIStatusVector);

```cpp
    std::cout << "toolAIStatusVector length is : " << toolAIStatusVector.size() << std::endl;
    for(int i = 0; i < toolAIStatusVector.size(); i++)
    {
        std::cout << "Tool IO [ " << i << " ]" << std::endl;
        std::cout << "ioId = " << toolAIStatusVector[i].ioId << " | ";
        std::cout << "ioType = " << toolAIStatusVector[i].ioType << " | ";
        std::cout << "ioName = " << toolAIStatusVector[i].ioName << " | ";
        std::cout << "ioAddr = " << toolAIStatusVector[i].ioAddr << " | ";
        std::cout << "value = " << toolAIStatusVector[i].ioValue << std::endl;
    }
```

输出



固件升级

135. **robotServiceUpdateRobotBoardFirmware**

136. **robotServiceGetBoardFirmwareUpdateResultService**

137. **robotServiceGetRobotEthernetDeviceName**

设置关节补偿

138. 设置关节碰撞补偿 **robotServiceSetRobotJointOffset**

传送带跟踪

139. 设置编码器复位 **robotServiceSetConveyorEncoderReset**

140. 启动传送带 **robotServiceSetConveyorStartup**

141. 停止传送带 **robotServiceSetConveyorStop**

142. 设置传送带方向 **robotServiceSetConveyorDir**

143. 设置手眼标定结果关系 **robotServiceSetRobotCameraCalib**

144. 设置传送带线速度 **robotServiceSetConveyorVelc**

145. 设置编码器距离关系 **robotServiceSetEncoderValPerMeter**

146. 设置传送带起始窗口上限 **robotServiceSetStartWindowUpstream**

147. 设置传送带起始窗口下限 **robotServiceSetStartWindowDownstream**

148. 设置传送带跟踪轨迹下限 **robotServiceSetConveyorTrackDownstream**

149. **robotServiceAppendObject2ConveyorTrackQueue**

150. **robotServiceEnableConveyorTrack**

151. **robotServiceGetConveyorEncoderVal**

152. 设置传送带跟踪的最大速度 **robotServiceSetRobotConveyorTrackMaxVelc**

153. 设置传送带跟踪的最大加速度 **robotServiceSetRobotConveyorTrackMaxAcc**

154. 设置传送带跟踪的系统延时时间 **robotServiceSetRobotConveyorSystemDelay**

155. 设置机械臂工具 **robotServiceSetRobotTool**

八、 错误代码

错误号	错误代码	错误信息
0	InterfaceCallSuccCode	成功
10001	ErrCode_Failed	通用失败
10002	ErrCode_ParamError	参数错误

10003	ErrCode_ConnectSocketFailed	Socket 连接失败
10004	ErrCode_SocketDisconnect	Socket 断开连接
10005	ErrCode_CreateRequestFailed	创建请求失败
10006	ErrCode_RequestRelatedVariableError	请求相关的内部变量出错
10007	ErrCode_RequestTimeout	请求超时
10008	ErrCode_SendRequestFailed	发送请求信息失败
10009	ErrCode_ResponseInfoIsNULL	响应信息为空
10010	ErrCode_ResolveResponseFailed	解析响应失败
10011	ErrCode_FkFailed	正解出错
10012	ErrCode_IkFailed	逆解出错
10013	ErrCode_ToolCalibrateError	工具标定参数有错
10014	ErrCode_ToolCalibrateParamError	工具标定参数有错
10015	ErrCode_CoordinateSystemCalibrateError	坐标系标定失败
10016	ErrCode_BaseToUserConvertFailed	基坐标系转用户座标失败
10017	ErrCode_UserToBaseConvertFailed	用户坐标系转基座标失败
10018	ErrCode_MotionRelatedVariableError	运动相关的内部变量出错
10019	ErrCode_MotionRequestFailed	运动请求失败
10020	ErrCode_CreateMotionRequestFailed	生成运动请求失败
10021	ErrCode_MotionInterruptedByEvent	运动被事件中断
10022	ErrCode_MotionWaypointVetorSizeError	运动相关的路点容器的长度不符合规定
10023	ErrCode_ResponseReturnError	服务器响应返回错误
10024	ErrCode_RealRobotNoExist	真实机械臂不存在，因为有些接口只有在真是机械臂存在的情况下才可以被调用
10025	ErrCode_moveControlSlowStopFailed	调用缓停接口失败
10026	ErrCode_moveControlFastStopFailed	调用急停接口失败
10027	ErrCode_moveControlPauseFailed	调用暂停接口失败
10028	ErrCode_moveControlContinueFailed	调用继续接口失败